



UNIVERSIDAD DE GRANADA

TRABAJO FIN DE GRADO

INGENIERÍA INFORMÁTICA

ResurgeNet

Gestión de comercios afectados por una catástrofe

Autora

Ángela María Garrido Ruiz

Directores

María José Rodríguez Fortiz

José Luis Garrido Bullejos

Escuela Técnica Superior de Ingenierías Informática y de
Telecomunicación



ÍNDICE DE CONTENIDOS

CAPÍTULO 1 - Introducción	2
1.1. Motivación	2
1.2. Objetivos	2
1.3. Cronograma	3
1.4. Presupuesto	3
1.5. Organización del documento	5
CAPÍTULO 2 - Estado del Arte	6
2.1. Descripción general del dominio del problema	6
2.2. Tecnologías utilizadas actualmente	9
CAPÍTULO 3 - Propuesta y desarrollo	16
3.1. Descripción	16
3.2. Descripción de la metodología	17
3.3. Sprint 0	18
3.4. Elección de la tecnología	26
3.5. Iteración 1	28
3.6. Iteración 2	33
3.7. Iteración 3	38
3.8. Iteración 4	46
3.9. Iteración 5	48
3.10. Iteración 6	53
3.11. Iteración 7	60
3.12. Iteración 8	76
3.13. Iteración 9	85
3.14. Iteración 10	89
3.15. Iteración 11	106
3.16. Problemas encontrados y soluciones adquiridas	112
CAPÍTULO 4 - Conclusiones y trabajos futuro	114
4.1. Conclusiones	114
4.2. Trabajos futuros	115
CAPÍTULO 5 - Bibliografía	117
ANEXO	120
Matriz de problemas encontrados y soluciones adoptadas	120
Enlace al repositorio de GitHub	121

CAPÍTULO 1 - Introducción

1.1. Motivación

La Comunidad Valenciana sufrió a finales de octubre de 2024 una Depresión Aislada en Niveles Altos (DANA) que ha tenido un impacto devastador en los comercios. El Segundo Informe de Evaluación de Daños elaborado por la Oficina PATECO indica que los daños causados sobre el sector terciario son del 92,3% en la zona más perjudicada, conocida como Zona 0. El informe destaca que el 96% de las empresas en las zonas dañadas eran microempresas o autónomos, perfiles especialmente vulnerables.

Pasados cinco meses tras el desastre, el 71,7% de los locales afectados volvió a retomar su actividad, sin embargo el 21,9% seguían cerrados [1]. En la zona más crítica unos 3.000 negocios permanecían inactivos. Aunque la recuperación empresarial está en curso, avanza lentamente debido a la tardanza en el pago de las indemnizaciones así como la insuficiencia de ayudas públicas. Más allá de la destrucción física de los comercios, la DANA ha provocado una reducción de la demanda, lo que ralentiza la reactivación de los comercios. Reactivar la actividad económica de los comercios requiere medidas que vayan más allá de la reconstrucción física, incluyendo incentivos para hacer crecer la demanda.

Este escenario ha constituido la principal motivación **personal** para desarrollar este TFG, titulado ResurgeNet. Tras observar el impacto económico y social que este tipo de catástrofes produce en los comercios locales nació la idea de crear una plataforma web que permita mantener su economía, aunque no dispongan de un espacio físico para la venta, para así ayudarles a recuperarse en la mayor brevedad posible.

Otro de los aspectos que motivó el desarrollo del proyecto fue la importancia de diseñar una plataforma accesible para el mayor número de personas posible. Además, esta iniciativa tiene un carácter solidario, ya que busca facilitar un canal directo entre comercio y consumidor, favoreciendo el apoyo a los establecimientos damnificados.

De esa forma, este proyecto no solo pretende ofrecer una solución tecnológica sino también una herramienta accesible y útil para la sociedad.

Desde el punto de vista **académico y técnico** este proyecto supone una oportunidad para aplicar los conocimientos adquiridos durante el desarrollo de las especialidades de Ingeniería del Software y Sistemas de Información en un dominio de aplicación real.

1.2. Objetivos

El objetivo **general** de este proyecto es ayudar a la recuperación de los comercios que se han visto afectados por una catástrofe, proporcionando una plataforma de comercio electrónico y dándoles visibilidad para que continúen con su actividad económica. El trabajo persigue consolidar, profundizar y poner en práctica conocimientos, así como adquirir y mejorar destrezas en el diseño y organización de una web, la gestión de usuarios, diseño y tratamiento de la información almacenada en la base de datos y el desarrollo de interfaces accesibles. Asimismo, representa una oportunidad para aprender nuevas tecnologías utilizadas actualmente en el sector. Además, este proyecto plantea el reto de construir una solución con propiedades de calidad como escalabilidad y mantenibilidad.

Para alcanzar el objetivo principal se plantean los siguientes objetivos **específicos**:

1. Estudiar y revisar las plataformas y tecnologías existentes relacionadas con el comercio electrónico.
2. Aplicar los conocimientos adquiridos durante el grado para desarrollar una plataforma basada en tecnologías web y de soporte al e-commerce.
3. Aprender y utilizar tecnologías y herramientas de desarrollo web, soporte en la nube, y ayuda al desarrollo.
4. Poner en práctica metodologías ágiles como SCRUM para planificar y gestionar el proyecto.
5. Diseñar una solución arquitectónica mantenible, escalable y portable, capaz de evolucionar en futuras fases según las necesidades de los usuarios y de la propia plataforma.
6. Aplicar principios de accesibilidad web basados en las recomendaciones WCAG (Web Content Accessibility Guidelines).
7. Profundizar en competencias relacionadas con la extracción y el análisis de información.

1.3. Cronograma

La planificación temporal de este trabajo se muestra utilizando diagramas de Gantt. Con ellos podremos visualizar de forma clara las tareas, su duración y su secuencia temporal.

Contaremos con dos diagramas:

- **Diagrama general del proyecto:** en éste se mostrarán las fases principales del proyecto y sus subtareas considerando tareas como la revisión del estado del arte, el aprendizaje de tecnologías y la redacción de esta memoria.
- **Diagrama por iteraciones:** se crearán diagramas específicos de cada iteración del proyecto, detallando las tareas concretas que se realizan en cada periodo y la planificación de entregas. Con esto tendremos un seguimiento más fino del proceso.

En el siguiente enlace se pueden consultar dichos diagramas: [📅 Diagrama Gantt](#)

1.4. Presupuesto

El presupuesto que se presenta a continuación tiene como objetivo realizar una estimación de los recursos necesarios para la elaboración de este proyecto, tanto en términos de personal como de infraestructura tecnológica necesaria.

1.4.1. Personal

El coste de personal comprende el tiempo de trabajo dedicado al desarrollo del proyecto, incluyendo todos los aspectos de diseño, implementación, pruebas y redacción de la documentación final.

Se estima que el desarrollo completo del proyecto requerirá aproximadamente 27 semanas, distribuidas en las siguientes fases:

1. **Análisis, planificación y diseño de la arquitectura (4 semanas):** Incluye la definición de requisitos del sistema, funcionalidades, elaboración de las historias de usuario y diagramas necesarios para obtener una visión global del sistema.
2. **Implementación de la plataforma (20 semanas):** Codificación del frontend y backend del sistema, incluyendo la configuración inicial de la plataforma y la integración de todas las funcionalidades.

3. **Documentación (3 semanas):** Redacción de la memoria, elaboración de cronogramas y edición final del TFG.

Un TFG tiene una carga académica de **12 créditos** ECTS. Considerando una carga de 25 horas de trabajo por crédito, se estima un total de **300 horas** invertidas en el desarrollo del mismo. Tomando como referencia los datos de los portales de empleo de referencia, el sueldo base promedio para un perfil de desarrollador junior en España se sitúa en los **11,48 €/hora** [3]. Para calcular el coste real imputable al proyecto, a esta tarifa base se le debe aplicar un factor corrector estimado del 45% en concepto de costes de Seguridad Social a cargo de la empresa, prorratas de pagas extraordinarias, vacaciones y costes indirectos. De este modo, se establece un coste de personal definitivo de **17 €/hora**. El coste estimado de personal para la fase de desarrollo inicial asciende, por tanto, a **5.100 €**.

Para el mantenimiento anual posterior, calculando un 20% del tiempo de desarrollo (60 horas) [2] bajo la misma tarifa horaria, se estima un coste recurrente de **1.020 €** anuales.

1.4.2. Infraestructura

En este apartado vamos a incluir los recursos tecnológicos que pueden ser necesarios para la plataforma. En este caso, el proyecto se desarrollará de forma local en nuestro ordenador personal. Por tanto, no se requerirán costes adicionales aunque vamos a incluirlos para reflejar el coste en un entorno profesional. Considerando un valor actual aproximado de 1.200 € y una vida útil estimada de 5 años, el coste mensual amortizado sería de 20 €.

Posteriormente, si quisiéramos desplegar el programa para que se pudiera usar por usuarios reales necesitaremos cierta infraestructura como:

- **Servidor virtual para despliegue y pruebas:** un servidor virtual privado con recursos básicos ronda entre los 13 y 30€ al mes. Si nos quedamos con un plan intermedio pagaremos 25€ mensuales [4].
- **Implementación/Licencias:** se podrían utilizar software gratuito como editores de código abierto pero en caso de necesitar herramientas más avanzadas o licencias profesionales, los costes pueden estar en torno a los 200€ u 800€ al año. Si por ejemplo usamos Visual Studio con la tarifa de 'Profesional Estándar' tendríamos que pagar una mensualidad de 100€ (usando la prorratea de suscripción anual) [5]
- **Marketing y promoción de la aplicación:** se consideraría la contratación de servicios que incluyan SEO (posicionamiento web), redes sociales y publicidad. Estos servicios rondan entre los 400 y 3000€ mensuales dependiendo del alcance y los canales usados. Quedándonos con un precio intermedio pagaremos unos 1700€ por este servicio. [6]
- **Adquisición de dispositivos para la realización de pruebas:** con el objetivo de verificar el funcionamiento en otros entornos sería útil adquirir un teléfono o una tablet para la realización de pruebas. Suponiendo un coste conjunto de 800 € y una amortización de 5 años, el coste imputable al proyecto sería aproximadamente de 13,33€/mes.

Teniendo en cuenta lo anterior el coste sería:

$$20€/mes \text{ (ordenador desarrollo)} + 25€/mes \text{ (servidor)} + 100€/mes \text{ (implementación)} + 1700€/mes \text{ (marketing)} + 13,33€/mes \text{ (dispositivos prueba)} = 1858,33/mes$$

Teniendo en cuenta que la estimación de tiempo para la realización del proyecto es de 27 semanas, lo que equivale a 6,3 meses el **coste de infraestructura sería de 11.707,48€**.

Es importante remarcar que una vez finalizado el proyecto se necesitará un **mantenimiento** continuado del mismo si se desea que continúe operativo. Esto implicaría mantener el servidor virtual privado activo, renovar licencias y continuar con los servicios de publicidad para garantizar la visibilidad. Los costes calculados corresponden solo a la puesta en marcha inicial, si incluimos el mantenimiento, es decir, las renovaciones mencionadas anteriormente el coste sería de 1858,33€/mensuales x 12 meses = **22.300€/anuales**.

1.4.3. Resumen

A continuación, se presenta una tabla que sintetiza los costes estimados para este proyecto diferenciando la fase de desarrollo inicial y el coste posterior de mantenimiento anual:

Categoría/Concepto	Cálculo	Coste Desarrollo (6,3 meses)	Coste Mantenimiento (anual)
1. Personal (Junior)			
Desarrollo inicial	300H x 17€/h	5.100€	-
Soporte y actualizaciones	60H x 17€/h	-	1020€
2. Infraestructura			
Equipo de desarrollo	20€/mes	126€	-
Dispositivos Pruebas	13,33€/mes	84€	-
Servidor Virtual	25€/mes	157,50€	300€
Licencias	100€/mes	630€	1.200€
3. Difusión/Comercialización			
Marketing y promoción	1.700€/mes	10.710€	20.400€
TOTAL		16.503€	22.920€

Como puede observarse el coste total para su puesta en marcha y desarrollo asciende a 16.503€, mientras que su continuidad operativa y comercial requiere una inversión de 22.920€ anuales. Esta diferencia se debe al peso que representan las campañas de marketing para dar visibilidad a la herramienta en un entorno real.

1.5. Organización del documento

Para facilitar la lectura y comprensión de la presente memoria, el documento se estructura en los bloques que se detallan a continuación:

- **Capítulo 2 - Estado del Arte**, presenta un análisis del mercado actual en el ámbito del comercio electrónico, así como un estudio de las tecnologías actuales que potencialmente podrían usarse para el desarrollo del sistema.
- **Capítulo 3 - Propuesta**, detalla la especificación de los requisitos del sistema, las historias de usuario y el diseño de la arquitectura. Así mismo, describe el proceso de construcción e implementación de la webapp siguiendo la metodología Scrum.
- **Capítulo 4 - Conclusiones y trabajos futuros**, sintetiza los resultados obtenidos y el grado de consecución de los objetivos planteados inicialmente. Finalmente se analiza el trabajo realizado y se proponen mejoras/extensiones para su desarrollo futuro.

- **Anexo**, donde podemos encontrar una tabla resumen de los problemas encontrados y las soluciones aportadas así como el enlace al repositorio de GitHub donde está el código completo del proyecto.

CAPÍTULO 2 - Estado del Arte

2.1. Descripción general del dominio del problema

Gracias al uso de Internet, las empresas pueden comercializar productos y servicios de forma digital, dando lugar a lo que se conoce como comercio electrónico o e-commerce. El comercio electrónico hace referencia a las transacciones financieras y al intercambio de información realizados electrónicamente entre una organización y los distintos usuarios o entidades con los que interactúa [7].

La expansión de Internet ha transformado significativamente el marketing y los modelos de negocio tradicionales, permitiendo que millones de usuarios puedan acceder diariamente a productos y servicios desde cualquier localización. Esta capacidad de conexión global ha convertido al comercio electrónico en una herramienta fundamental para las empresas, ya que amplía su alcance comercial y facilita la digitalización de procesos de venta y atención al cliente.

Para ofrecer este tipo de servicios digitales, las plataformas de comercio electrónico requieren una infraestructura tecnológica capaz de gestionar procesos como la visualización de productos, autenticación de usuarios, procesamiento de pagos, gestión de pedidos y almacenamiento de información. Por ello, resulta necesario comprender la arquitectura y funcionamiento técnico sobre el que se construyen plataformas de comercio electrónico actuales.

2.2 Plataformas actuales para la creación y gestión de comercios

Vamos a analizar las plataformas existentes que permiten la creación y gestión de comercios de forma online, destacaremos las ventajas y limitaciones de dichas plataformas.

2.2.1 Hostinger (<https://www.hostinger.com/es>)

Hostinger es un proveedor destacado de soluciones de alojamiento web que opera en más de 150 países. El hosting es un servicio que permite que un sitio web sea accesible desde Internet, ya que al contratarlo se alquila un espacio en un servidor donde se almacenan todos los archivos y datos necesarios para su correcto funcionamiento. [10]

La misión de Hostinger es hacer que la presencia online sea accesible para todos. Para ello, ofrece herramientas como un creador de sitios web impulsado por inteligencia artificial y un panel de control (hPanel) que facilita tareas como la gestión, instalación y configuración de un sitio web de manera sencilla. [12]

Se trata de un servicio de pago que ofrece distintos planes —como Premium o Business— con precios y características variables según el tipo de hosting y la duración de la suscripción.

Entre sus principales **ventajas** destacan: [14]

- Interfaz intuitiva, fácil de usar y con diseño moderno.
- Precios reducidos en el primer contrato.
- Buen rendimiento en términos de velocidad y disponibilidad.
- Servicio de migración de sitios web incluido.

Entre las **limitaciones** se encuentran: [14]

- Espacio de almacenamiento restringido en algunos planes.
- Copias de seguridad diarias solo disponibles en el plan más completo, mientras que los demás planes ofrecen copias semanales.
- Funcionalidades avanzadas reservadas para los planes de mayor coste.

En conclusión, Hostinger es una empresa consolidada que proporciona servicios de alojamiento web con cobertura internacional, ofreciendo soluciones flexibles y un soporte que facilita la gestión de sitios web mediante sus distintos planes de pago. Sin embargo, el coste hace que no todos los comercios, como los pequeños, puedan utilizarla.

2.2.2 WooCommerce (<https://woocommerce.com/es/>)

WooCommerce es una plataforma de comercio electrónico flexible y de código abierto desarrollada por la empresa Woo, basada en WordPress. Desde 2017, la compañía ha centrado sus esfuerzos exclusivamente en ofrecer soluciones para el comercio electrónico. [16]

WooCommerce permite a pequeñas y medianas empresas crear tiendas online personalizadas para vender sus productos a través de Internet. La plataforma es gratuita y de código abierto, lo que facilita su acceso a cualquier usuario. Además, es compatible con diversos proveedores de alojamiento, como Bluehost, Nexcess o el propio WordPress.

Entre sus principales **ventajas** [15] destacan:

- Alta flexibilidad en cuanto al tipo de productos que se pueden vender.
- Seguridad sólida que protege la información y transacciones de los usuarios.
- Amplia comunidad de usuarios y desarrolladores, lo que genera recursos, tutoriales y soporte constante.
- Plataforma principalmente gratuita, accesible para negocios con distintos presupuestos.

Sin embargo, también presenta algunas **limitaciones** [15]:

- Solo es compatible con sitios desarrollados en WordPress.
- Algunas funcionalidades avanzadas requieren pagos adicionales, lo que puede generar costes ocultos.
- Puede resultar compleja para usuarios sin experiencia previa en WordPress o comercio electrónico.

En definitiva, esta empresa es una buena solución para la creación de tiendas online, especialmente indicada para pequeñas y medianas empresas que buscan flexibilidad y personalización. Sin embargo, su dependencia de WordPress y la necesidad de conocimientos técnicos para ciertas configuraciones representan retos que deben considerarse al momento de elegir esta plataforma para un proyecto de comercio electrónico.

2.2.3 Amazon (<https://developer.amazon.com/>)

Amazon es uno de los mayores referentes a nivel mundial en comercio electrónico, actuando tanto como plataforma de venta directa como marketplace. Su arquitectura permite dar

soporte a millones de usuarios y transacciones diarias a escala global. Esta plataforma se apoya en arquitecturas distribuidas y modelos basados en microservicios, lo que le permite escalar sus funcionalidades de forma eficiente. [9]

Los microservicios ofrecen un enfoque simplificado para el desarrollo software que acelera la implementación, fomenta la innovación, mejora el mantenimiento e impulsa la escalabilidad. Este método se basa en servicios pequeños y poco acoplados que se comunican a través de APIs y son gestionados por equipos autónomos.

Gran parte de los productos disponibles en Amazon no pertenecen a la propia multinacional, la mayoría son de empresas independientes o vendedores autónomos. Hay dos formas de vender a través de este canal:

- **Tienda propia en Amazon:** es la opción más adecuada si se desea tener más control sobre la identidad de la marca.
- **Usar los servicios existentes que proporciona:** es una opción adecuada para aquellos que quieren aprovechar la infraestructura sin invertir demasiado en desarrollo y mantenimiento de una tienda propia

Las principales **ventajas** [17] que presenta son:

- Gran audiencia global
- Confianza y reputación
- Logística avanzada
- Gestión sencilla y marketing, ya que podemos vender sin crear un e-commerce propio
- Proporciona análisis de datos a los vendedores sobre el rendimiento de los productos

Por otra parte, presenta los siguientes **inconvenientes** [17]:

- Gran competencia
- Cumplimiento con las políticas de Amazon que pueden cambiar y afectar a las operaciones comerciales
- Cumplimiento con las exigencias de calidad de servicio
- Comisiones y tarifas, se cobra comisión a los minoristas por cada producto vendido y, en el caso de crear una cuenta como vendedor, hay que abonar un pago mensual.

En conclusión, vender en Amazon puede representar una oportunidad para acceder a un mercado global y aprovechar una infraestructura tecnológica ya consolidada, aunque implica limitaciones como la alta competencia, las comisiones asociadas y la dependencia de las políticas de la plataforma.

2.2.4 Resumen comparativa plataformas

A continuación, se muestra una tabla comparativa de las plataformas vistas que nos ayudará a analizar la facilidad de uso que tiene cada una de ellas:

Tabla 1. Comparativa de las plataformas que ayudan a analizar la facilidad de uso para usuarios sin conocimientos técnicos

Características	Hostinger	WooCommerce	Amazon
Tipo de solución	Hosting + creador web	CMS de comercio electrónico	Marketplace global
Nivel de facilidad de uso	Alto	Medio	Muy alto

Conocimientos técnicos necesarios	Bajos	Medios	Ninguno
Personalización	Media	Alta	Baja
Control del negocio	Medio	Alto	Bajo
Coste inicial	Bajo	Bajo	Bajo
Escalabilidad	Media	Alta	Muy alta
Mantenimiento	Medio	Bajo	Nulo para el usuario
Dependencia de la plataforma	Media	Baja	Muy alto

A partir de esta comparativa podemos ver que cada una de las soluciones presenta un enfoque diferente en función del perfil de usuario y el grado de control que se desea tener sobre el comercio electrónico.

Desde el punto de vista técnico Amazon se posiciona como la opción más accesible, ya que no se requieren conocimientos técnicos para comenzar con la venta. Sin embargo, esta facilidad implica una alta dependencia con la plataforma, al igual que limitaciones con la personalización y el control del negocio.

Hostinger representa una solución intermedia orientada a usuarios sin conocimientos técnicos avanzados. Aun así, tenemos limitaciones en cuanto a la escalabilidad y el mantenimiento que depende más del usuario.

Finalmente, WooCommerce es la opción que más control de negocio y personalización ofrece permitiendo un control completo sobre la tienda online. No obstante, estas ventajas exigen mayor conocimiento técnico, requiere conocimientos básicos de gestión de WordPress, configuración de plugins y mantenimiento del sistema.

Definitivamente se identifica una barrera para el usuario sin conocimientos técnicos, ya que las soluciones comentadas presentan un equilibrio entre facilidad de uso, control y flexibilidad. Las opciones más sencillas limitan la personalización mientras que las más completas requieren competencias técnicas avanzadas. Esta situación pone de manifiesto la necesidad de soluciones que reduzcan la complejidad sin renunciar a funcionalidades esenciales como lo son la personalización y el control del comercio.

2.2. Tecnologías utilizadas actualmente

A continuación, se detallan las tecnologías que actualmente son más utilizadas para desarrollo web.

2.2.1 Lenguajes de programación

La elección del lenguaje de programación para el desarrollo del sistema está condicionada por la naturaleza del proyecto. En este caso, al tratarse de una aplicación web, es necesario utilizar tecnologías tanto para el desarrollo del frontend, encargado de la interfaz de usuario, como para el backend, que gestiona la lógica de negocio.

Por tanto, los lenguajes que sopesamos son:

- **HTML5:** Lenguaje de marcado utilizado para describir la estructura de las páginas web, siendo uno de los lenguajes frontend más fáciles de aprender [19]. Es accesible, ya que es de código abierto y gratuito, ampliamente utilizado y flexible. Como inconveniente

se presenta la compatibilidad de los navegadores y es necesario complementarlo con otros lenguajes para aportar dinamismo.

- **CSS:** Lenguaje utilizado para estilizar los elementos descritos en HTML, por lo que la relación entre estos dos lenguajes es muy fuerte [20]. Hace que la visualización de la página sea mucho más atractiva para el usuario final.
- **JavaScript (JS):** Complementado con HTML y CSS es uno de los lenguajes más usados para llevar a cabo el frontend de las páginas web, el 97,8% de las páginas lo utilizan para mejorar la interactividad de la misma. Es un lenguaje de código abierto, que funciona de manera adecuada con otros lenguajes y mejora la experiencia del usuario final. Por otro lado, el rendimiento entre navegadores es inestable, es decir, puede haber diferencias en la ejecución entre distintos navegadores. [18]
- **Python:** Lenguaje de alto nivel y de propósito general que es usado para diferentes tareas, una de ellas el desarrollo backend de una página web. Es más fácil de aprender que otros lenguajes, de código abierto y escalable. Sin embargo, es más lento y no es el ideal para el desarrollo de aplicaciones móviles. [18]
- **PHP:** Lenguaje de desarrollo backend para la web, el 78,1% de los sitios web lo utilizan al ser el lenguaje principal de WordPress. Además, este lenguaje ofrece una serie de frameworks, como Laravel o Symfony, que son de los mejores para el desarrollo de web y aplicaciones. Este lenguaje es de código abierto, está bien establecido para el desarrollo web y soporta OOP. Sin embargo, la creación de la web puede ser más lenta que con otros lenguajes y ofrece menos herramientas de depuración. [18]

A continuación, se muestra una tabla comparativa de los lenguajes mencionados anteriormente.

Tabla 2. Comparativa de lenguajes de programación web

Características	HTML5	CSS	JS	PHP	Python
Frontend	X	X	X	-	
Backend				X	X
Facilidad de aprendizaje	X	X	X	-	X
Compatibilidad con otros lenguajes	-	-	X	-	-
Mejora experiencia al usuario final	-	X	X	-	-
Popularidad	-	-	-	X	-
Más lento para plataformas web	-	-	-	-	X
Utilizado anteriormente	X	X	X	X	-

2.2.2 Frameworks [24]

Los frameworks o marcos de desarrollo web son un conjunto de herramientas, estilos y librerías para el desarrollo de aplicaciones web más escalables y sencillas de mantener. Durante el proceso de desarrollo de las implementaciones necesarias tanto para el cliente

como el servidor, la utilización de frameworks facilita nuestro trabajo ya que ahorramos tiempo, llevamos a cabo una codificación que es escalable y aporta seguridad.

Los marcos diseñados para el *frontend* nos aportan códigos ya escritos donde podemos seguir desarrollando funcionalidades. Dependiendo del lenguaje de programación contaremos con marcos de trabajo específicos:

- **React:** es una biblioteca de JavaScript que nos aporta código abierto para crear elementos de la interfaz de usuario. Entre las ventajas que nos ofrece su uso encontramos: un rendimiento uniforme y consistente con el uso de DOM, fácil reutilización de componentes y las herramientas que nos ofrece son útiles y avanzadas. Su principal desventaja es que necesitaremos usar bibliotecas adicionales para el enrutamiento y algunas funcionalidades que se encuentran en la parte del cliente.
- **Vuejs:** es un modelo-vista-vista-modelo, este es un modelo que nos ayuda a separar la lógica de presentación y negocios de la interfaz de usuario. Al igual que el anterior, es un marco de JavaScript para implementar aplicaciones de una sola página, estas aplicaciones son las que cargan una única vez la página principal desde el servidor y luego actualiza dinámicamente su contenido. Es un marco diseñado para mejorar el rendimiento y tratar aspectos que representan una dificultad en el desarrollo, a pesar de esto no es uno de los marcos más populares. Las ventajas que ofrece son: simplicidad, mayor flexibilidad y menos restricciones.
- **jQuery:** es uno de los marcos con más antigüedad y sigue siendo relevante en la tecnología de hoy en día. Ofrece facilidad de uso y simplicidad, nos permite usar códigos de JavaScript más cortos. Fundamentalmente su uso se centra en la manipulación de DOM, interfaz de programación para los documentos HTML, y CSS, lo que permite optimizar la interactividad y funcionalidades de un sitio web.

Vista esta información comparativa obtenemos la siguiente tabla.

Tabla 3. Comparativa de frameworks para desarrollo web en el *frontend*

Característica	React	Vuejs	jQuery
Uso de JS	X	X	X
Simplicidad		X	X
Facilidad de uso	-	-	X
Mejor reutilización de componentes	X	-	-
Mayor flexibilidad	-	X	-
Uso de bibliotecas adicionales	X	-	-
Menos popularidad	-	X	-
Utilizado anteriormente	X	-	-

Los marcos que podemos utilizar para el desarrollo del *backend* nos ayudan a crear la configuración del mismo para las aplicaciones web. Su uso permite a los desarrolladores suplir la necesidad de configurar y construir desde cero. Al igual que en el frontend, dependiendo del lenguaje de programación contaremos con unos marcos u otros:

- **Django:** es un marco de código abierto basado en Python, su objetivo principal es simplificar la creación de sitios web que se basan en bases de datos. Este marco nos aporta una interfaz administrativa que se configura con la ayuda de modelos de administración. Como ventajas nos encontramos con: la combinación de Django y Python son tecnologías claves utilizadas por referentes de la informática como Google, buena documentación, escalable, personalizable y seguro.
- **Laravel:** es un marco web PHP basado en Symfony cuyo código fuente se encuentra disponible en GitHub, sigue el patrón modelo-vista-controlador. Cuenta con las siguientes ventajas: simplifica la implementación del proceso de autenticación, tendremos aplicaciones web muy seguras ya que protege de la falsificación de solicitudes, inyección SQL y script entre sitios; ya contamos con la implementación del manejo de excepciones y errores y los trabajos de prueba están automatizados.

Igual que para los marcos frontend, vamos a ver la información como una tabla comparativa.

Tabla 4. Comparativa de frameworks para desarrollo web en el *backend*

Característica	Django	Laravel
Python	X	
PHP	-	X
Interfaz administrativa	X	-
Buena escalabilidad	X	X
Buena seguridad	X	X
Mayor facilidad de uso		X
Protección contra ataques comunes	-	X
Simplificación de procesos	-	X
Utilizado anteriormente	-	-

2.2.3 Bases de datos

En este punto vamos a distinguir principalmente entre dos enfoques para la construcción de la base de datos, el relacional y no relacional. [25]

- **BD relacional:** almacena los datos en formato tabular con filas (valores) y columnas (atributos del dato). Cada tabla contará con una clave primaria, que es el identificador de la misma. Se usa para establecer las distintas relaciones que se darán entre las tablas. Cuando tenemos las tablas relacionadas podemos obtener información mediante distintas consultas SQL.
 - Este modelo sigue ACID, un acrónimo que significa: atomicidad, consistencia, aislamiento y durabilidad, establece un conjunto de propiedades que garantizan la integridad de los datos.
 - Por otra parte, el rendimiento y la escalabilidad son un punto débil de este tipo de base de datos. En definitiva, esta forma de almacenamiento de datos es la indicada para datos que son predecibles en tamaño, frecuencia de acceso y estructura.

- **BD no relacional:** utilizan distintos modelos de datos que permiten acceder a los mismos y administrarlos. Son eficientes cuando tenemos que manejar grandes volúmenes de datos. Se puede llevar a cabo el almacén de datos de distintas maneras, por clave-valor, geográficos y de documentos.
 - Este modelo nos asegura la disponibilidad de los datos, pero no la coherencia de los mismos, no cumple el modelo ACID. Sin embargo, este tipo de bases de datos ofrecen un mayor rendimiento y son altamente escalables. Sabiendo esto, podemos decir que este tipo de almacenamiento de datos es el adecuado para datos flexibles o que pueden cambiar en un futuro.

A continuación, se muestra una tabla comparativa de ambas opciones.

Tabla 5. Comparativa de bases de datos

Característica	BD relacional	BD no relacional
Almacenamiento de datos	Tabular	No hay una forma fija
ACID	X	-
Mejor rendimiento	-	X
Mejor escalabilidad	-	X
Mayor experiencia de uso	X	-

Ya que sabemos los dos tipos de bases de datos con las que contamos para trabajar, vamos a comparar dos concretas de cada tipo. En la parte de bases de datos relacionales contamos con MySQL y PostgreSQL [21]; en la parte de no relacionales tenemos MongoDB y Apache Casandra [22]

Comenzamos por las bases de datos relacionales:

Tabla 6. Comparativa de bases de datos relacionales

Característica	MySQL	PostgreSQL
ACID	Parcial	Completo
Control de concurrencia en todas las versiones	Depende del motor	Nativo
Tipos de datos	Básicos	Muy avanzados
Extensibilidad	Media	Alta
Consumo de recursos	Alto	Medio
Mayor experiencia de uso	X	-

Finalmente, vamos a comparar MongoDB con Apache Casandra, nuestras bases de datos no relacionales a estudio:

Tabla 7. Comparativa de bases de datos no relacionales

Característica	Mongo DB	Apache Casandra
Modelo de datos	Columnas anchas	Documentos
Esquema	Fijo	Muy flexible
Escalabilidad	Excelente	Buena
Disponibilidad	Muy alta	Alta
Consultas	CQL (tipo SQL)	MQL (JSON)
Experiencia de uso	Baja	Nula

2.2.4 Servidores

Para desarrollar una aplicación web, necesitaremos un servidor web que cumplirá tres funciones clave: servir la interfaz al usuario final, actuar como proxy inverso, es decir, es un servidor que se encuentra antes del servidor web y reenvía las solicitudes del cliente a estos servidores; y garantizar la seguridad de nuestra plataforma.

Entre los más destacados se encuentran:

- **Apache:** es un servidor web gratuito y de código abierto que sirve contenidos a través del protocolo HTTP. Fue el más utilizado en los inicios de la World Wide Web y, aunque ha perdido el primer puesto, sigue siendo uno de los más populares, con aproximadamente un 30% de las webs en funcionamiento. Forma parte de la pila LAMP, un paquete de componentes de código abierto y gratuito que funcionan de manera conjunta con el fin de ayudar a los desarrolladores a crear, desplegar y gestionar apps web dinámicas, lo que lo hace compatible con diversos programas, lenguajes y frameworks web. Su sistema modular le proporciona alto rendimiento y flexibilidad, permitiendo la personalización según las necesidades del proyecto. Utiliza una arquitectura basada en procesos, creando un hilo para cada petición, lo que puede generar tiempos de carga más lentos y periodos de inactividad cuando debe gestionar un gran número de solicitudes simultáneamente. [26]
- **Nginx:** es un servidor web gratuito y de código abierto, reconocido por su fiabilidad, escalabilidad y velocidad. Su popularidad es comparable a la de Apache, aunque actualmente es ligeramente más utilizado. A diferencia de Apache, emplea una arquitectura basada en eventos, lo que le permite gestionar un mayor número de peticiones simultáneamente con mayor eficiencia. Además de funcionar como servidor web, puede desempeñar el papel de equilibrador de carga, mejorando la disponibilidad y eficiencia de los recursos, y también puede actuar como proxy inverso para optimizar la comunicación entre clientes y servidores backend. [26]

Como en los puntos anteriores vamos a ver la información expuesta antes en modo de tabla comparativa:

Tabla 8. Comparativa de servidores

Característica	Apache	Nginx
Arquitectura	Basada en procesos	Basada en eventos
Flexibilidad	X	-
Mejor rendimiento	-	X
Mejor escalabilidad	-	X
Mejor velocidad	-	X
Seguridad	Depende de la configuración	Depende de la configuración
Mayor experiencia de uso	X	-

2.2.5 Contenedorización y despliegue

Los contenedores los usamos para construir, compartir y ejecutar aplicaciones, por lo tanto, es un paquete de software que contiene los elementos necesarios para que una aplicación se ejecute de forma rápida y fiable. Hace que se puedan desplegar en distintos entornos sin problema.[27] Los contenedores más utilizados son:

- **Kubernetes:** fue diseñado por Google y es de código abierto. Se utiliza para automatizar el despliegue, escalado y gestión de aplicaciones en contenedores. Las ventajas que nos proporciona son: operaciones automatizadas, abstracción de la arquitectura y supervisión del estado de los servicios, es decir, se llevan a cabo comprobaciones de estado automáticas y se reinician los contenedores que o bien han fallado o bien se han detenido. No obstante, este tipo de contenedorización no es totalmente una PaaS, hay que tener en cuenta lo complejo que es la gestión de Kubernetes, por este motivo muchos de los clientes prefieren usar un servicio ya gestionado. [23]
- **Docker:** es una plataforma de contenedores de código abierto, se pueden ejecutar tanto en Linux, Windows y MacOS. Nos proporciona un modo de construir y ejecutar contenedores usando una arquitectura cliente-servidor. Entre las ventajas que nos aporta tenemos: automatización a través de una sola API, incluye un kit de herramientas para empaquetar aplicaciones como imágenes de contenedor pudiendo estas ejecutarse en cualquier plataforma que admita contenedores y nos permite hacer el proceso de forma eficaz. Pese a esto, la coordinación y programación en algunos servidores, la actualización y la supervisión del estado pueden plantear problemas [23]

Vamos a ver esta información en modo de tabla comparativa:

Tabla 9. Comparativa de contenedores

Característica	Kubernetes	Docker
Tipo de plataforma	Orquestador de contenedores	Plataforma de contenedores
Automatización	X	X
Supervisión	X	-
Compatibilidad	X	X
Facilidad de uso	-	X
Mejor escalabilidad	X	-
Popularidad	X	X
Mayor experiencia de uso		X

CAPÍTULO 3 - Propuesta y desarrollo

3.1. Descripción

Este proyecto consiste en la creación y administración de una plataforma que permita a los comercios afectados por una catástrofe continuar con su actividad económica mientras se recupera su punto de venta local. Este servicio estará disponible hasta que el comercio pueda volver a operar físicamente con normalidad.

Los objetivos de la plataforma serán:

- Visibilizar los comercios afectados que cuentan con venta online, facilitando su identificación rápida en la zona afectada. Se les dará publicidad en un listado con enlace a su propia web.
- Permitir que los comercios sin plataforma de venta online puedan realizar las ventas a través de nuestra web, incluyendo también a aquellos que aunque antes contaban con una han perdido los recursos para mantenerla.
- Gestionar adecuadamente el stock de los comercios, para lo cual se requerirá que, al solicitar el alta en la plataforma, las tiendas proporcionen un archivo con el inventario disponible. Así se garantizará que la venta de productos se pueda hacer de manera eficiente.

Se prestará especial atención a que la plataforma cuente con un asistente de accesibilidad para que personas con diversidad funcional puedan utilizarla.

Una vez sabemos de qué trata el proyecto vamos a ver quienes serán nuestros clientes y los actores principales del sistema:

- **Comercios:** son los afectados por el desastre que se publicitan a través de la web y a los que se les ofrecerá el punto de venta. Serán los encargados de gestionar su tienda dentro de la plataforma, es decir, de controlar el stock de los productos que desean vender. Tendrán un espacio de administración de su comercio, en el que actualizarán sus productos, eliminarán o subirán nuevos. Además contarán con un monedero donde tienen el dinero de las ventas acumuladas, este dinero estará disponible y podrán sacarlo de la plataforma pasándolo a su cuenta bancaria tras la revisión del administrador. Los dividiremos en tres grupos:
 - Comercios sin venta online propia
 - Comercios con venta online propia pero que deciden publicitarse para obtener mayor visibilidad y ser agrupados dentro de la zona dañada
 - Comercios que han perdido el punto de venta que previamente tenían
- **Consumidores:** con los usuarios finales que comprarán los productos con una intención solidaria, apoyando a los comercios afectados
- **Validador de comercios:** será la figura encargada de verificar si el comercio que solicita nuestros servicios reúne los requisitos para incorporarse a la plataforma.
- **Administrador:** será el encargado de la gestión de la plataforma, esto implica dar de alta a los comercios que han pasado la validación, darles de baja cuando no necesitan más el servicio, crear la lista de comercios afectados que será visible para los consumidores, revisar las retiradas de dinero efectuadas por los comercios, atender peticiones tanto de comercios como consumidores y hacer una gestión de los usuarios que tenemos en el sistema. Tendremos un único administrador del sistema.

Si recapitulamos, sabemos que lo que **ofrecemos a nuestros clientes** es: punto de venta online, publicidad y una plataforma solidaria de fácil acceso. Por tanto, **el cliente encontrará valioso lo que ofrecemos** ya que como comercio puede seguir teniendo ingresos, y si se trata de un consumidor, tendrá un canal directo de ayuda hacia las empresas damnificadas.

Sabiendo los clientes que tendremos, tenemos que pensar en cómo va a ser nuestra **interacción** con ellos. El comercio hará una solicitud de adhesión a la plataforma, que sería estudiada de forma manual, por un **validador de comercios**, para asegurar que cumple con los requisitos. Una vez que esté dado de alta, la gestión se realizará de forma online de manera automática, es decir, nos informará del tipo de servicio que quiere mediante la propia web. Por otra parte, con el consumidor las operaciones se realizarán de forma automática a través de la plataforma, como con el resto de puntos de venta online.

Además de comunicarnos con los distintos clientes por la plataforma, como se ha expuesto anteriormente, nos serán de utilidad las redes sociales y las plataformas de ONGs colaboradoras para promover la iniciativa. Así mismo, debemos tener **colaboradores** para poder ofrecer la propuesta, como he comentado anteriormente podemos colaborar con ONGs pero además deberíamos explorar alianzas con gobiernos locales y proveedores de servicios de entrega, ya que podrían facilitar la logística para los envíos de los productos.

Para poder ofrecer esta solución tenemos que llevar a cabo una serie de tareas como:

- Desarrollo y mantenimiento de la plataforma
- Actualización de la base de datos de los comercios
- Estrategias de marketing para atraer tanto a los comercios como a los consumidores
- Gestión de relaciones con ONGs y otras organizaciones
- Gestión con empresas de logística para todo lo relacionado con los envíos

No solo debemos contar con colaboradores sino también con una fuente de ingresos, en este caso la principal fuente será la mensualidad que nos pagarán las **marcas o empresas no afectadas por publicitarse en la plataforma**. Esta es una iniciativa solidaria por lo que los comercios afectados no deberán pagar nada durante su proceso de recuperación. La logística se gestionará mediante convenios con las ONGs que cuenten con venta online o aquellas empresas que estén interesadas a prestar el servicio. Además de estos recursos, se ofrecerán promociones, subastas y concursos a los consumidores.

Sabiendo esto, podemos ver claramente quiénes serán nuestros **pagadores**: las empresas publicitadas y los consumidores que participen en las promociones, subastas o concursos mencionados. También se dejará abierto un canal de crowdfunding, un método de financiación colectiva donde las personas que lo deseen podrán aportar dinero para impulsar la causa.

En los siguientes puntos se expone cuál será el diagrama de la arquitectura del sistema, con qué tecnologías se va a desarrollar y la metodología que seguiremos.

3.2. Descripción de la metodología

Para el desarrollo de este proyecto hemos decidido seguir un enfoque basado en la metodología ágil SCRUM, debido a que permite gestionar el trabajo de forma iterativa e incremental, lo que nos facilita la incorporación de cambios en el proceso de desarrollo.

Esta metodología se basa en ciclos de desarrollo denominados sprint, y en nuestro caso tendrán una duración de tres semanas cada uno, tal como se ha establecido en el cronograma del proyecto.

Durante cada sprint se van a llevar a cabo las siguientes actividades:

- Planificación del sprint, donde se definen los objetivos del mismo y se seleccionarán las historias de usuario que se van a llevar a cabo, comenzando por las de mayor prioridad
- Desglose de tareas a realizar en cada historia y pruebas de aceptación
- Diseño de prototipos, tanto de la interfaz como de la base de datos
- Implementación de cada uno de los componentes
- Realización de pruebas para asegurar un funcionamiento correcto de cada componente y que se satisfagan las condiciones de aceptación establecidas para cada historia
- Revisión del sprint al finalizar el mismo, donde se ven los avances y se revisa el cumplimiento de los objetivos establecidos en la planificación
- Retrospectiva del sprint, se identifican los puntos fuertes y débiles que ha habido a la hora de desarrollar las historias de usuario elegidas y se optimiza el proceso

Para llevar a cabo toda esta organización cuento con herramientas como Jira para llevar un control del estado de las tareas y GitHub para el control de versiones.

El uso de Scrum nos permitirá obtener resultados funcionales desde las primeras iteraciones y obtener un desarrollo ordenado y eficiente.

A continuación describiremos con detalle las actividades o tareas, las pruebas y los resultados obtenidos en cada iteración o sprint.

3.3. Sprint 0

En este sprint, que tendrá una duración de tres semanas, vamos a llevar a cabo las siguientes actividades:

- Planteamiento general de requisitos no funcionales
- Elaboración del diseño arquitectónico inicial del sistema
- Desarrollo de la lista priorizada y estimada de historias de usuario
- Estimación de la velocidad de trabajo
- Agrupación y descripción de las historias de usuario para la primera iteración

3.3.1. Requisitos no funcionales del sistema

Para garantizar que la plataforma sea una herramienta eficiente, segura y con capacidad de crecimiento, el sistema debe cumplir con una serie de atributos de calidad y requisitos no funcionales que son:

- Acceso multiplataforma, al estar basada en la web se podrá acceder desde cualquier dispositivo con diferentes sistemas operativos y navegadores con acceso a Internet
- Seguridad, toda la información sensible relacionada a los comercios afectados así como a los consumidores y los productos debe protegerse frente a accesos no autorizados, garantizando así la integridad de los datos.
- Soporte multiusuario, la plataforma debe tener la capacidad de gestionar múltiples peticiones simultáneas de consumidores y comercios de forma eficiente, evitando el empeoramiento del rendimiento del sistema en picos de uso.
- Modularidad y Mantenibilidad, el código y los componentes del sistema deben estar estructurados de forma que permitan realizar actualizaciones o correcciones de manera independiente y con un impacto mínimo en el resto del sistema.

- Interoperabilidad, el sistema debe ser capaz de integrarse con otras plataformas o aplicaciones externas, como pasarelas de pago, de forma sencilla mediante protocolos estándar.
- Escalabilidad y Portabilidad, el sistema debe estar preparado para incrementar sus capacidades y recursos a medida que aumente el volumen de comercios y consumidores registrados. Así como de tener la capacidad de ser utilizado en distintos entornos.

3.3.2.Diseño Arquitectónico

Dado que el sistema se desarrollará como una plataforma web, es necesario definir una arquitectura adecuada, y la más acertada es la arquitectura cliente-servidor basada en un entorno contenedorizado.

En esta arquitectura tenemos dos capas destacadas:

- El cliente, que se encargará de interactuar con la aplicación
- El servidor, que es el responsable de la lógica de negocio y de servir la información solicitada por el cliente. El servidor incluye el acceso de una base de datos centralizada, que será donde tengamos todos los datos de nuestro sistema.

Como se detalla más adelante, para facilitar el despliegue y asegurar que la aplicación funcione de manera idéntica en cualquier servidor toda la lógica del mismo se ha estructurado unificada dentro de un contenedor. Un contenedor es un entorno aislado que empaqueta los servicios y dependencias necesarios para que el sistema funcione de forma autónoma.

Los motivos para usar esta arquitectura, basándonos en el cumplimiento de los requisitos no funcionales expuestos en el punto 3.6.0, son:

- Separación de responsabilidades, al tener dos capas con sus funcionalidades bien diferenciadas como se ha explicado anteriormente. El cliente se encarga de la presentación y la interacción con el usuario, resolviendo el requisito de acceso multiplataforma desde cualquier navegador. Por su parte, el servidor gestiona la lógica de negocio y el acceso a los datos.
- Seguridad, los datos se gestionarán en el servidor evitando así que estén expuestos en el lado del cliente.
- Soporte a usuarios simultáneos, esta arquitectura permite gestionar múltiples peticiones concurrentes, así aseguramos que varios usuarios puedan interactuar con la plataforma sin que disminuya el rendimiento. Sin embargo, al solo tener un servidor si en un momento dado tenemos un gran volumen de peticiones podemos tener sobrecarga y esto nos disminuiría el rendimiento. Por tanto, es importante combinar esta capacidad con la escalabilidad vertical comentada anteriormente. También se podría pensar como trabajo futuro realizar escalabilidad horizontal, es decir, añadir servidores para mantener la calidad del servicio conforme se incrementa el número de usuarios.
- Modularidad y mantenibilidad, la separación entre frontend y backend favorece una arquitectura modular en la que cada componente se desarrolla, modifica o sustituye de forma independiente. Con esto conseguimos que la reutilización de componentes sea más sencilla además de facilitar las tareas de mantenimiento y futura evolución del sistema. Las modificaciones o correcciones de diseño en la interfaz tienen un impacto

nulo en la lógica interna del servidor, lo que simplifica las tareas de mantenimiento y la evolución del sistema.

- Interoperabilidad, la comunicación entre los distintos componentes se realiza mediante APIs y protocolos estándar como son HTTP/HTTPS, con esto se permite que diferentes tecnologías puedan interactuar entre sí de forma sencilla. Gracias a esto el sistema puede integrarse con otras plataformas o aplicaciones externas pensando en futuras versiones.
- Escalabilidad y Portabilidad, la arquitectura que tenemos nos permite ampliar los recursos del sistema conforme aumenten las necesidades de la plataforma. Como se comentó anteriormente, el sistema principalmente se basa en la escalabilidad vertical mediante la ampliación de recursos en la parte del servidor. Sin embargo, el hecho de tener el backend aislado dentro de un contenedor desde el inicio simplifica la transición hacia una arquitectura más distribuida y escalable, si el volumen de usuarios así lo requiera. Gracias al uso de contenedores podemos desplegar la aplicación en distintos entornos de forma consistente, independientemente del sistema operativo o la infraestructura, garantizando una alta portabilidad. Esto nos simplifica las tareas de despliegue, configuración y mantenimiento, además facilita la migración del sistema entre distintos servidores.

En la capa del **cliente** se desarrollará la interfaz de usuario, que permitirá la interacción con el sistema de forma sencilla e intuitiva. Esta capa será la encargada de mostrar la información y recibir las peticiones del usuario, adaptándose a diferentes dispositivos.

En la capa del **servidor** se gestionará la lógica de la aplicación y la comunicación con la base de datos. El servidor recibirá las solicitudes procedentes del cliente, las procesará y devolverá las respuestas correspondientes. La **base de datos** almacenará la información necesaria para el funcionamiento de la aplicación, garantizando la persistencia y consistencia de los datos.

Además, se utilizarán **contenedores** para aislar los diferentes componentes de la aplicación, facilitando su despliegue, portabilidad y escalabilidad.

Finalmente, este es el esbozo de la arquitectura del sistema:

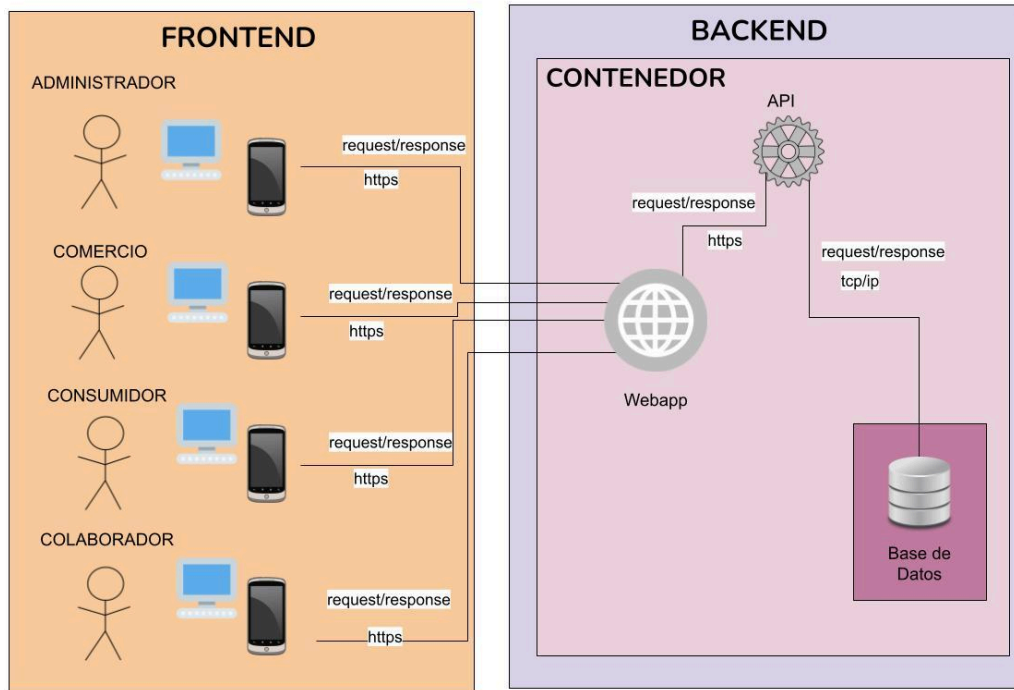


Figura 1: Esbozo de la arquitectura del sistema

En este esquema se puede ver como el sistema cuenta con dos componentes diferenciados interconectados entre sí. Además vemos los diferentes actores que intervienen en el sistema, en la parte del cliente, así como el flujo que tenemos en el servidor:

- Nos llegarán las peticiones del cliente a través de la webapp
- Con la ap:
 - Procesamos las peticiones
 - Consultamos la base de datos
 - Devolvemos el recurso solicitado

Dicho recurso devuelto será visible para los clientes gracias a la webapp

3.3.3. Lista priorizada y estimada de historias de usuario.

A continuación, se muestra el listado que contiene las historias de usuario, el product backlog, que son una descripción simple de una tarea concreta que aporta valor al usuario o al negocio.

Para determinar la prioridad vamos a usar la técnica **MoSCoW**, que se basa en determinar qué historias son imprescindibles para que el producto funcione (Must have), cuales debería tener (Should have), cuales podría tener (Could have) y cuales no tendrá en primera instancia (Won't have).

Para decidir la estimación de cada una, es decir, el tiempo que nos llevará realizar cada una de estas funcionalidades, vamos a usar los puntos de historia donde:

- 1-2 puntos serán para HU debajo esfuerzo o rápidas de realizar
- 3 puntos para funcionalidades sencillas pero que exigen más esfuerzo de programación
- 5 puntos para historias que requieren una mayor lógica de programación y validaciones
- 8 puntos para aquellas con complejidad alta

- 13 puntos para funcionalidades muy complejas

Teniendo en cuenta todo esto así quedaría nuestro listado:

ID	Descripción	Prioridad	Estimación
HU 0	Instalación y preparación del sistema	M	2
HU 1	Creación de la landing page de la webapp	M	1
HU 2	Como administrador quiero un inicio de sesión único para poder llevar a cabo mis funciones	M	2
HU 3	Como administrador quiero un cierre de sesión seguro para que los datos no sean de fácil acceso	M	1
HU 4	Como validador de comercios quiero contar con un inicio de sesión personalizado para poder llevar a cabo mis funciones	M	2
HU 5	Como comercio quiero tener un inicio de sesión personalizado para poder llevar a cabo mis funciones	M	2
HU6	Como consumidor quiero contar con una plataforma de registro donde poder darme de alta	M	2
HU 7	Como consumidor quiero tener un inicio de sesión personalizado para poder llevar a cabo mis funciones	M	2
HU 8	Como comercio quiero tener un espacio en el que subir los productos para empezar con su venta	M	3
HU 9	Como administrador quiero tener acceso a la lista de comercios que han pasado la validación para poder proceder al alta de los mismos	M	2
HU 10	Como administrador del sistema quiero poder dar de baja a los comercios	M	2
HU 11	Como administrador del sistema quiero poder dar de baja a los consumidores que realizarán compras en los comercios promocionados	M	2
HU 12	Como validador de comercios quiero contar con un buzón donde recibir las solicitudes de los comercios	M	3
HU 13	Como validador de comercios quiero contar con un buzón a través del que pueda ponerme en contacto con los comercios para hacerles saber el estado de su solicitud	M	3
HU14	Como comercio quiero tener acceso a la web de mi tienda de forma que pueda actualizar sus productos.	M	5
HU15	Como validador de comercios quiero un cierre de sesión seguro para que los datos no sean de fácil acceso	M	2
HU16	Como comercio quiero contar con un cierre de sesión seguro para que los datos no sean de fácil acceso	M	2
HU17	Como consumidor quiero contar con un cierre de sesión seguro para poder llevar a cabo mis funciones	M	2
HU 18	Como consumidor quiero poder modificar mis datos personales	M	3
HU 19	Como consumidor quiero tener una zona donde ver los pedidos que he realizado y consultar su estado	M	3

HU 20	Como consumidor quiero poder consultar qué productos puedo comprar y hacer un pedido	S	4
HU 21	Como consumidor quiero contar con una plataforma de pago segura para la realización de mis compras en los comercios	S	4
HU 22	Como consumidor quiero contar con un cierre de sesión seguro para poder llevar a cabo mis funciones	S	1
HU 23	Como administrador quiero contar con un apartado para publicar el listado de todos los comercios afectados, enlazando cada comercio a su web, ya sea propia o proporcionada por la plataforma.	S	3
HU 24	Como administrador quiero contar con un buzón de peticiones donde puedan escribirme tanto consumidores como comercios	S	3
HU 25	Como administrador quiero contar con un buzón a través del que pueda ponerme en contacto con los comercios	S	3
HU 26	Como consumidor quiero poder ver el seguimiento de mi pedido en tiempo real	S	4
HU 27	Como comercio quiero tener un monedero donde se ingresarán las ventas que se realicen en mi web	S	5
HU 28	Como comercio quiero poder retirar el dinero del monedero de forma segura a mi cuenta bancaria	S	5
HU 29	Como consumidor quiero elegir el tipo de entrega que se realizará (a domicilio o en un punto de recogida)	C	5
HU 30	Como usuario final quiero recuperar mi contraseña en caso de olvidarla	C	5
HU 31	Como usuario final (ya sea consumidor o comercio) quiero contar con un asistente de accesibilidad donde poner las características que deseo que tenga la página	C	8
HU 32	Como colaborador (ya sea empresa u ONG) quiero tener acceso a un apartado para incluir información sobre nosotros.	C	4
HU 33	Como colaborador quiero ver que se incluye mi logo en la plataforma	C	2

3.3.4. Estimación de la velocidad de trabajo

Tras haber realizado el listado anterior vamos a estimar la velocidad de trabajo.

El equipo de trabajo en este caso solo contará conmigo como miembro y las iteraciones serán de tres semanas. Los días de trabajo de cada iteración van a ir variando por lo que se tendrá que ajustar la velocidad para cada una de estas iteraciones en función del tiempo real disponible y la productividad observada.

Vamos a hacer una primera estimación para el periodo comprendido desde Octubre hasta finales de Enero de 2026. Definimos que:

- 1 punto de historia (PH) equivale a 1 día de trabajo (8 horas efectivas)

- 1 día ideal equivale a 3 días reales de trabajo, considerando que no trabajo exclusivamente en este proyecto, estimo que puedo dedicarle unas 2-3 horas reales durante los días que trabaje en él.

Como primera estimación, tendré **12 días reales de trabajo** (aproximadamente 3 días por semana en cada iteración).

Dado que:

$$12 \text{ días_trabajo_por_iter} / 3 \text{ días_trabajo_real} = 4 \text{ días ideales}$$

La velocidad inicial estimada será **4 PH por iteración**.

Vamos a hacer la estimación para el periodo comprendido desde Febrero hasta Junio ya que mi disponibilidad cambia en esos meses y puedo dedicar más tiempo al proyecto. En este caso, 1 día ideal equivaldría a 2 días de trabajo real. Trabajando en el proyecto 5 días a la semana nos quedarían 15 días de trabajo por iteración. Por tanto:

$$15 \text{ días_trabajo_por_iter} / 2 \text{ días_trabajo_real} = 7,5 \text{ días ideales}$$

La velocidad inicial estimada será **8 PH por iteración**.

Sabiendo la velocidad de trabajo, así quedaría el listado anterior añadiendo las iteraciones en las que se llevará a cabo cada historia de usuario:

ID	Descripción	Prioridad	Estimación	Iteración
HU 0	Instalación y preparación del sistema	M	2	I1
HU 1	Creación de la landing page de la webapp	M	1	I1
HU 2	Como administrador quiero un inicio de sesión único para poder llevar a cabo mis funciones	M	2	I1
HU 3	Como administrador quiero un cierre de sesión seguro para que los datos no sean de fácil acceso	M	2	I2
HU 4	Como validador de comercios quiero contar con un inicio de sesión personalizado para poder llevar a cabo mis funciones	M	2	I2
HU 5	Como comercio quiero tener un inicio de sesión personalizado para poder llevar a cabo mis funciones	M	2	I3
HU6	Como consumidor quiero contar con una plataforma de registro donde poder darme de alta	M	2	I3
HU 7	Como consumidor quiero tener un inicio de sesión personalizado para poder llevar a cabo mis funciones	M	2	I4
HU 8	Como comercio quiero tener un espacio en el que subir los productos para empezar con su venta	M	3	I5

HU 9	Como administrador quiero tener acceso a la lista de comercios que han pasado la validación para poder proceder al alta de los mismos	M	2	I6
HU 10	Como administrador del sistema quiero poder dar de baja a los comercios	M	2	I6
HU 11	Como administrador del sistema quiero poder dar de baja a los consumidores que realizarán compras en los comercios promocionados	M	2	I6
HU 12	Como validador de comercios quiero contar con un buzón donde recibir las solicitudes de los comercios	M	3	I7
HU 13	Como validador de comercios quiero contar con un buzón a través del que pueda ponerme en contacto con los comercios para hacerles saber el estado de su solicitud	M	3	I7
HU14	Como comercio quiero tener acceso a la web de mi tienda de forma que pueda actualizar sus productos.	M	5	I8
HU15	Como validador de comercios quiero un cierre de sesión seguro para que los datos no sean de fácil acceso	M	2	I9
HU16	Como comercio quiero contar con un cierre de sesión seguro para que los datos no sean de fácil acceso	M	2	I9
HU17	Como consumidor quiero contar con un cierre de sesión seguro para poder llevar a cabo mis funciones	M	2	I9
HU 18	Como consumidor quiero poder modificar mis datos	M	3	I10
HU 19	Como consumidor quiero tener una zona donde ver los pedidos que he realizado	M	3	I10
HU 20	Como consumidor quiero poder consultar qué productos puedo comprar y hacer un pedido	S	4	-
HU 21	Como consumidor quiero contar con una plataforma de pago segura para la realización de mis compras en los comercios	S	4	-
HU 22	Como comercio quiero contar con un espacio para controlar el stock de mi establecimiento	S	5	-
HU 23	Como administrador quiero contar con un apartado para publicar el listado de todos los comercios afectados. Enlazando cada comercio a su web, ya sea propia o proporcionada por la plataforma. que no necesitan soporte web	S	3	-
HU 24	Como administrador quiero contar con un buzón de peticiones donde puedan	S	3	-

	escribirme tanto consumidores como comercios			
HU 25	Como administrador quiero contar con un buzón a través del que pueda ponerme en contacto con los comercios	S	3	-
HU 26	Como consumidor quiero poder ver el seguimiento de mi pedido en tiempo real	S	4	-
HU 27	Como comercio quiero tener un monedero donde se ingresarán las ventas que se realicen en mi web	S	5	-
HU 28	Como comercio quiero poder retirar el dinero del monedero de forma segura a mi cuenta bancaria	S	5	-
HU 29	Como consumidor quiero elegir el tipo de entrega que se realizará (a domicilio o en un punto de recogida)	C	5	-
HU 30	Como usuario final quiero recuperar mi contraseña en caso de olvidarla	C	5	
HU 31	Como usuario final (ya sea consumidor o comercio) quiero contar con un asistente de accesibilidad donde poner las características que deseo que tenga la página	C	8	-
HU 32	Como colaborador (ya sea empresa u ONG) quiero tener acceso a un apartado para incluir información sobre nosotros.	C	4	-
HU 33	Como colaborador quiero ver que se incluye mi logo en la plataforma	C	2	-

Las historias no planificadas inicialmente son las menos prioritarias y tienen la última columna vacía, por lo que serán consideradas para una segunda fase del proyecto.

Para cada iteración vamos a presentar primero sus historias, cada una con sus tareas y pruebas de aceptación. A continuación mostraremos los resultados de la realización de las tareas de diseño, principalmente bocetos, capturas de pantalla y evolución del diagrama de la base de datos. Después explicaremos cómo se han llevado a cabo las tareas de implementación estática (frontend) y dinámica (backend) de las historias, y terminaremos con la revisión y retrospectiva de la iteración.

3.4. Elección de la tecnología

Una vez analizadas las diversas tecnologías disponibles para el desarrollo del proyecto (apartado 2.b), y teniendo en cuenta las características de la aplicación a desarrollar, vamos a definir cuáles serán finalmente las tecnologías elegidas:

- **Lenguajes de programación:**
 - **Frontend:** Se opta por la combinación de HTML5, CSS y JavaScript, ya que son los lenguajes más utilizados para el desarrollo de aplicaciones web. Además, su uso es complementario: HTML proporciona la estructura de la página, CSS se encarga de su diseño visual, y JavaScript permite agregar interactividad, mejorando así la experiencia del usuario final.
 - **Backend:** Se utilizará PHP debido a su popularidad y a la experiencia adquirida en proyectos anteriores con este lenguaje.

- **Frameworks:**
 - **Frontend:** Se elegirá jQuery, un framework basado en JavaScript con gran trayectoria y aún en uso hoy en día. Aunque no tengo experiencia previa con él, considero que su simplicidad y facilidad de uso permitirán una integración rápida, optimizando la interactividad y la funcionalidad del sitio web. jQuery se integrará directamente con HTML y CSS, ya que permite manipular el contenido y el estilo de la página de manera dinámica facilitando la creación de efectos visuales y animaciones sin necesidad de tener un código de JavaScript extenso.
 - **Backend:** Dado que PHP es el lenguaje elegido, el framework seleccionado será Laravel, ya que es el más comúnmente utilizado junto con este lenguaje de programación.
- **Base de datos:** Se ha optado por una base de datos relacional, debido a la experiencia previa con este tipo de bases de datos, lo que facilitará su implementación. Además, este modelo garantiza la integridad de los datos, un aspecto que considero relevante y adecuado ya que vamos a trabajar con datos estructurados. Tendremos distintas tablas con las respectivas relaciones entre ellas. En la base de datos se almacenará información de los usuarios registrados, los comercios participantes, productos disponibles, pedidos realizados etc.
- **Servidor:** Se utilizará Nginx, ya que proporciona un rendimiento, escalabilidad y velocidad superiores a la opción de Apache más tradicional. Además, es una tecnología con la que me gustaría familiarizarme, lo que representa una oportunidad para adquirir experiencia.
- **Contenerización y despliegue:** Se implementará Docker, lo que ofrecerá ventajas como la compatibilidad con múltiples plataformas. Los contenedores permiten aislar cada componente, facilitar su despliegue en distintos entornos y simplificar las dependencias, nos garantiza que el sistema se ejecute correctamente independientemente del sistema operativo o equipo que usemos. Hace que nuestro sistema sea portable y escalable. Al haber trabajado previamente con Docker, esta elección permitirá un desarrollo más ágil en esta parte del proyecto.

Sabiendo cuales son las tecnologías que se van a utilizar podemos ver un esquema de la arquitectura más detallado, incluyendo éstas:

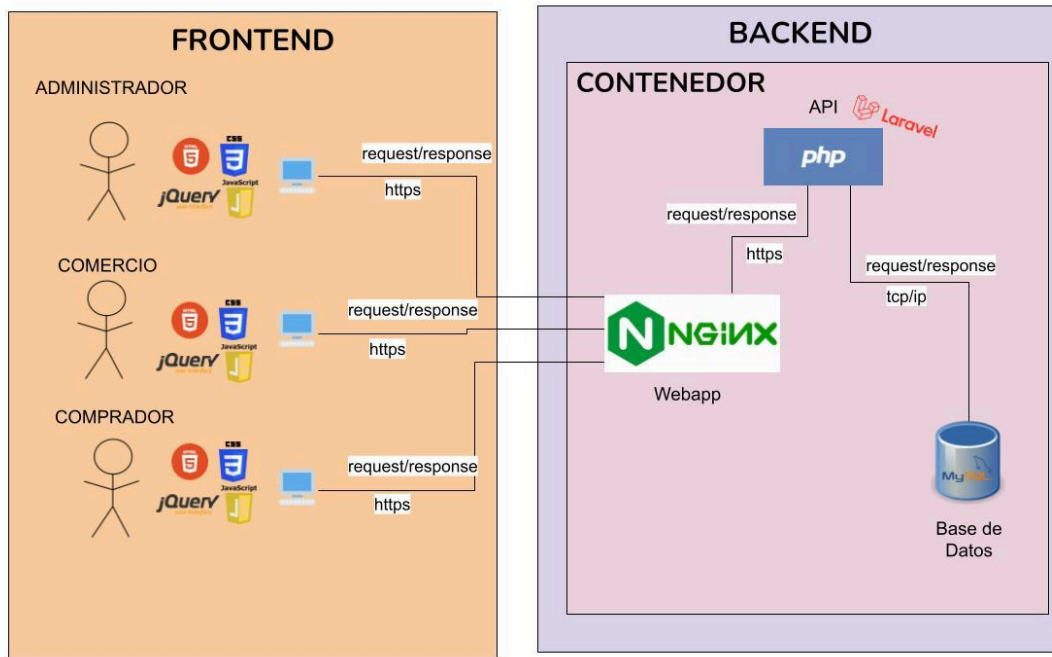


Figura 2: Esquema de la arquitectura del sistema detallado

3.5. Iteración 1

En esta primera iteración, que abarca desde el 1 al 15 de Octubre, vamos a desarrollar las siguientes historias de usuario:

HU0-Instalación y preparación del sistema

Las tareas relacionadas con esta historia son:

- Creación de un repositorio de GitHub para subir el proyecto y tener un control de las versiones del mismo - 4 de Octubre
 - El acceso al repositorio se hace a través de la siguiente url:
<https://github.com/angelamgr/ResurgeNet>
- Creación del contenedor local de Docker con todos los servicios que incluye (base de datos, api y webapp) - 4 Octubre
- Pruebas de aceptación - 5 Octubre
 - Funcionamiento y acceso correcto al repositorio
 - Despliegue correcto del contenedor con todos sus servicios

HU 1 - Creación de la landing page de la webapp

La *landing page* servirá como página de entrada a la plataforma, en ella se presentará la iniciativa, se visibilizan las funciones a las que se pueden acceder y podemos ver las opciones de navegación tanto para comercios como consumidores interesados en participar. Cuenta con varios botones destinados a llevarnos a contenidos concretos de la web como: el inicio de sesión, registro y contacto de los comercios con la plataforma, la visualización del listado de comercios, y la visualización de nuestros colaboradores.

Las tareas relacionadas son:

- Diseño de la interfaz - 11 de Octubre

- Conexión con las distintas páginas, en esta primera iteración con el inicio de sesión - 14 de Octubre
- Implementación estática - 14 de Octubre
- Implementación dinámica- 15 de Octubre
- Pruebas de aceptación - 15 de Octubre
 - Comprobación del correcto funcionamiento de los botones de la página
 - Visualización correcta de la página siguiendo el boceto ideado

HU2 - Como administrador quiero un inicio de sesión único para poder llevar a cabo mis funciones

Las tareas relacionadas con esta historia son las siguientes:

- Diseño de la interfaz - 5 de Octubre
- Diseño de la BD - 5 de Octubre
- Creación de la BD - 7 de Octubre
- Implementación estática - 11 de Octubre
- Implementación dinámica - 12 de Octubre
- Pruebas de aceptación - 15 de Octubre
 - Comprobación de que el usuario existe
 - Comprobación de que accede solo si está registrado

Se ha llevado a cabo la HU2 antes que la HU1 ya que durante la implementación de la historia 2 fue cuando pensamos en realizar la landing page para que quedará más coherente el primer vistazo a la web.

3.5.1. Diseño de la interfaz y de la BD

Para los diseños de la interfaz se va a utilizar la herramienta Figma ya que permite crear los bocetos de forma sencilla e interactiva, por lo que cuando tengamos todos realizados se podrá ver una demo del funcionamiento del sitio web.

Antes de empezar con el diseño de la interfaz he llevado a cabo la realización del logo y la elección de la paleta de colores:

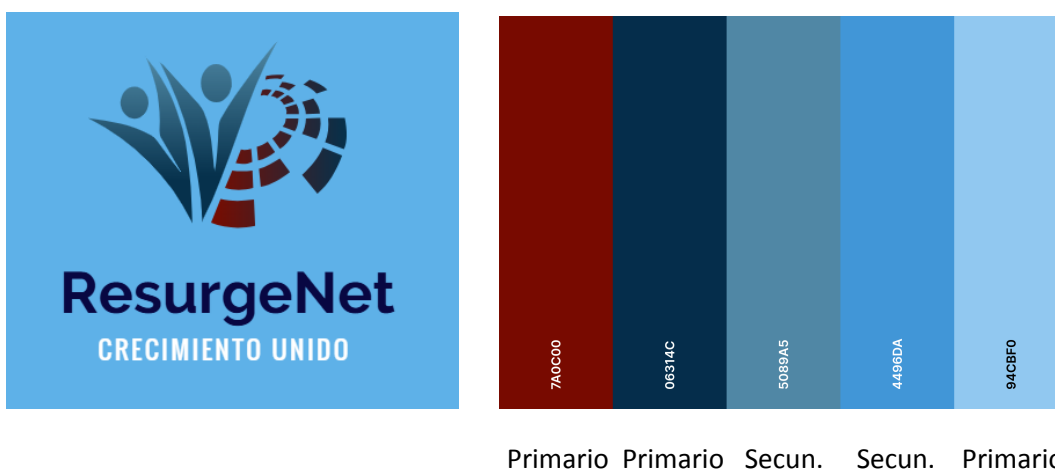


Figura 3: Logo y paleta de colores

Al tener ya definido estos aspectos así quedaría el primer boceto para el inicio de sesión del administrador:

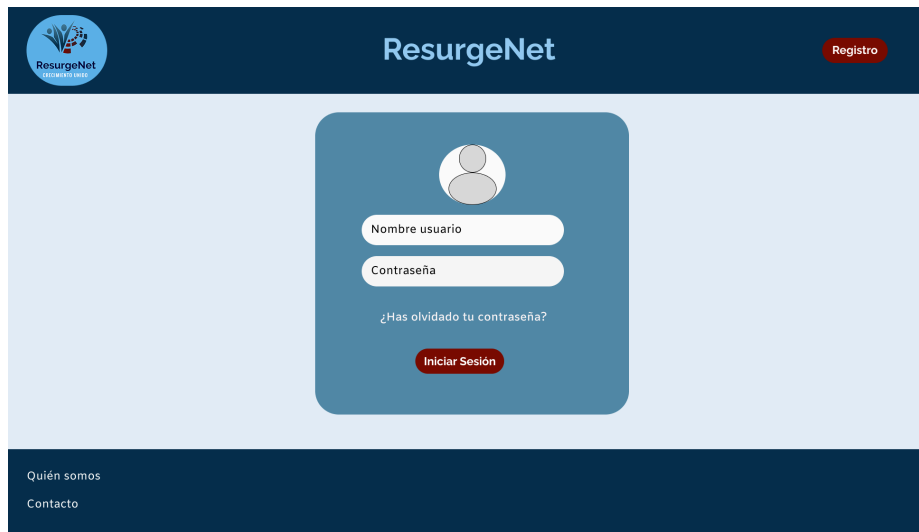


Figura 4: Boceto inicio de sesión administrador

La base de datos va a estar diseñada siguiendo el esquema Entidad-Relación, en esta primera HU quedará de la siguiente manera:

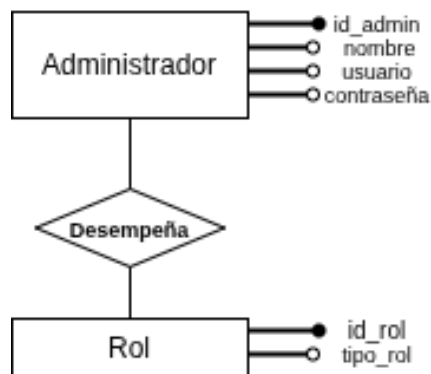


Figura 5: Esquema entidad relación HU2

Tenemos dos entidades principales y la relación que conecta a ambas entidades. La entidad Administrador tiene como clave primaria el identificador y tendrá una clave externa a la entidad Rol. La entidad Rol almacenará los distintos tipos de roles que hay dentro del sistema, será útil más adelante cuando creemos las historias de usuario enfocadas en consumidores y comercios, gracias al tipo de rol que desempeñe tendrá acceso a funcionalidades diferentes del sistema.

Al elaborar estos bocetos, se consideró también la primera visualización de la webapp para el usuario final, incluyendo la landing page desde la cual se podrá acceder a los distintos servicios, como el registro de nuevos consumidores y el inicio de sesión. El boceto base es el siguiente:



Figura 6: Boceto landing page

Como puede apreciarse en el boceto, nuestra página de inicio cuenta con la información necesaria para que un nuevo usuario sepa rápidamente a qué nos dedicamos y qué servicios ofrecemos. El diseño incluye los apartados de inicio de sesión y registro, así como acceso directo a nuestras principales funcionalidades “Trabaja con nosotros”, el “Listado de comercios” afectados y de los diferentes “Colaboradores” (como las ONGs que se mencionaron en el apartado 3.1)

3.5.2. Implementación estática y dinámica

Tras consensuar los bocetos presentados anteriormente con los tutores de este TFG , que actúan como cliente al que le presentan los avances, comienzo con la implementación de los mismos, diferenciando frontend y backend.

Parte estática - frontend

En esta parte hemos desarrollado los siguientes archivos:

- HTML: index.html e inicio_sesion.html
- CSS: estructura.css, index.css e inicio_sesion.css
- JS: login.js

Los archivos html tienen una estructura de encabezado (header) y pie de página (footer) común por lo que su aspecto se ha definido en el archivo estructura.css. Las particularidades de estilo de cada uno se han definido en archivos separados.

El archivo en JavaScript es donde tenemos la función que maneja la petición AJAX del inicio de sesión.

Parte dinámica - backend

Lo primero que tenemos que hacer es crear las tablas en la base de datos. Estamos usando MySQL por lo que trabajamos con la terminal. Como podemos ver en el esquema entidad relación definido en el apartado anterior (f.1) vamos a tener dos tablas: Rol y Administrador. Nos conectamos a la base de datos mediante el comando `docker exec -it db bash` y una vez conectados accedemos a la base de datos llamada ResurgeNet y procedemos a la creación de las tablas con las siguientes sentencias SQL:

```
CREATE TABLE rol(id_rol INT AUTO_INCREMENT PRIMARY KEY, tipo_rol VARCHAR(30) NOT NULL);
```

```
CREATE TABLE administrador(id_admin INT AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(30) NOT NULL, usuario VARCHAR(20) NOT NULL, password VARCHAR(10) NOT NULL, rol INT NOT NULL, FOREIGN KEY (rol) REFERENCES rol(id_rol));
```

Una vez que tenemos las tablas correctamente almacenadas en la base de datos, procedemos a insertar un administrador. Los pasos seguidos han sido:

- Inserción del rol 'administrador' en la tabla Rol usando la sentencia:
 - *INSERT INTO rol(tipo_rol) VALUES ('Administrador');*
- Creación de un trigger para encriptar la contraseña de los administradores cuando se inserta uno nuevo o se actualiza la contraseña de uno existente. Esto tiene como finalidad garantizar la seguridad de la información que estamos guardando en la base de datos. Al encriptar la contraseña, ésta no se almacena en texto plano evitando que sea accesible a usuarios no autorizados.
- Inserción de un administrador en la tabla Administrador asignándole el rol correspondiente. La sentencia usada ha sido:
 - *INSERT INTO administrador (nombre, usuario, password, rol) VALUES ('Angela Garrido', 'angelamgr', 'adminAG',1);*

Una vez que tenemos las tablas creadas tenemos que llevar a cabo la conexión del frontend con el backend, es decir, tenemos que hacer operativo el formulario de inicio de sesión del administrador. Para ello tenemos que configurar Laravel para que se conecte con la base de datos ResurgeNet que tenemos almacenada en MySQL , para eso tenemos que:

- Modificar el archivo .env, para que se enlace correctamente con la base de datos que tenemos en nuestro contenedor. Este archivo es proporcionado por Laravel y nos permite llevar a cabo la gestión de las variables de entorno. Las variables de entorno son valores externos al código fuente que permiten configurar el comportamiento de la aplicación según el entorno en el que se ejecute. En ellas se almacenan datos sensibles o específicos de la configuración, como el nombre de la base de datos, el usuario, la contraseña o el puerto de conexión.
- Comprobar que el *dockerfile* de nuestra API tiene la extensión .pdo_mysql, necesaria para que Laravel pueda conectarse a MySQL. Un *dockerfile* es un archivo de configuración que define cómo se construye la imagen de un contenedor, especificando el sistema operativo base, las dependencias y las configuraciones necesarias para que el sistema funcione en cualquier entorno. Este archivo ha sido creado automáticamente por la api, y solo hemos tenido que verificar que estuviera alineado con la configuración que buscamos.
- Limpiar y volver a cargar la configuración para asegurarnos que los cambios realizados en los dos puntos anteriores sean efectivos en el sistema, si no hacemos este paso podríamos estar utilizando los valores antiguos.
- Probar la conexión para confirmar que es exitosa

Una vez llevados a cabo los pasos anteriores nos quedaría conectar el formulario de inicio de sesión con Laravel, para ello:

- Creamos una ruta en Laravel para el inicio de sesión

- Creamos el controlador, que actúa como intermediario entre el frontend y la base de datos, es donde se va a llevar a cabo la validación de credenciales del usuario. Recogemos los datos del formulario que rellena el usuario y hacemos una serie de comprobaciones
 - Si el usuario existe
 - Si la contraseña introducida es correcta
 - Verificar si quiere iniciar la sesión un usuario con el rol administrador, esto será útil para cuando tengamos que controlar más usuarios ya que dependiendo de qué usuario sea, le será redirigido a una página u otra de la webapp
- Con jQuery conectamos el frontend, por tanto hemos generado un nuevo archivo, en este caso un script llamado login.js

Una vez tenemos todo lo anterior correctamente implementado podemos pasar a hacer las pruebas de aceptación que se describieron anteriormente:

- Entrar con el usuario que tenemos en la base de datos como administrador y que se permita iniciar sesión
- Entrar con un usuario que no está registrado y que no nos deje iniciar sesión, nos aparece un mensaje de “Credenciales incorrectas”

3.5.3. Revisión por parte de los tutores

Durante la revisión de los bocetos llevados a cabo en esta iteración, los clientes dijeron que el diseño les parecía adecuado y coherente. Como única modificación, solicitaron que el tamaño de la letra se aumentará para mejorar la legibilidad.

En general, la retroalimentación fue positiva y la petición indicada se tendrá en cuenta en las siguientes iteraciones.

3.5.4. Retrospectiva de la iteración

Para esta primera iteración la organización y planificación del trabajo ha sido adecuada, así como la distribución de tareas y la definición de los objetivos. Esto ha permitido que se lleven a cabo todos los objetivos.

La comunicación con los tutores ha sido fluida y ha contribuido a validar correctamente cada fase del desarrollo antes de su implementación.

En definitiva, la metodología seguida ha sido efectiva por lo que para las siguientes iteraciones se mantendrá la misma dinámica.

3.6. Iteración 2

En esta segunda iteración, que abarca desde el 16 de Octubre al 5 de Noviembre, vamos a desarrollar las siguientes historias de usuario:

HU3-Como administrador quiero un cierre de sesión seguro para que los datos no sean de fácil acceso

Las tareas relacionadas con esta historia son:

- Diseño de la interfaz - 20 de Octubre
- Diseño de la BD - 20 de Octubre

- Revisión BD - 20 de Octubre
- Implementación estática - 22 de Octubre
- Implementación dinámica - 24 de Octubre
- Pruebas de aceptación - 27 de Octubre
 - Cierre de sesión exitoso, el usuario es redirigido a la landing page y el token de la sesión se invalida
 - Si el usuario quiere ir hacia atrás en la web no puede ver el contenido

HU4- Como validador de comercios quiero contar con un inicio de sesión personalizado para poder llevar a cabo mis funciones

Las tareas relacionadas con esta historia son:

- Diseño de la interfaz - 28 de Octubre
- Diseño de la BD - 28 de Octubre
- Revisión BD - 29 de Octubre
- Implementación estática - 1 de Noviembre
- Implementación dinámica - 3 de Noviembre
- Pruebas de aceptación - 5 de Noviembre
 - Comprobación de que el usuario existe
 - Comprobación de que accede solo si está registrado

3.6.1. Diseño de la interfaz y de la BD

Primero vamos a llevar a cabo los diseños para la HU3, el cierre de sesión del administrador. En cuanto a la base de datos no tenemos que modificar nada, permanece igual que para las anteriores historias. Seguimos contando con dos tablas Administrador y Rol.

En cuanto al diseño, hemos tenido que diseñar un nuevo boceto que muestra el panel de administración, con sus distintas funciones, donde se ha incluido el botón para el cierre de sesión. Con este panel añadimos funcionalidad al botón de inicio de sesión creado en la iteración anterior para el login del administrador, que anteriormente solo nos decía si las credenciales eran correctas o no, pero no nos llevaba a una nueva página. El resultado es el siguiente:



Figura 7: Boceto dashboard administrador

Como podemos ver, cuando el administrador inicia sesión será redirigido a su área de trabajo donde podrá acceder a las distintas funciones que desempeña y tendrá su botón para llevar a cabo el cierre de sesión. Estos elementos estarán en un menú desplegable a la izquierda de la pantalla.

Para la HU4, correspondiente al inicio de sesión del validador de comercio, se mantendrá el mismo diseño que para el inicio de sesión del administrador, Figura 4. En esta historia de usuario, la base de datos es la que experimenta mayores modificaciones. Quedando así el esquema:

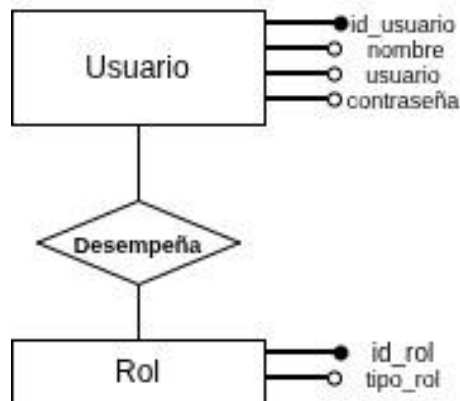


Figura 8: Esquema entidad relación HU3

Al diseñar la tabla correspondiente al validador de comercio, se observó que compartía los mismos atributos que la tabla de administradores. La única diferencia entre ambos tipos de usuario es el rol asociado. Por este motivo, se decidió unificar ambas entidades en una única tabla llamada Usuario, diferenciando los distintos tipos mediante el rol.

3.6.2. Implementación estática y dinámica

Parte estática - frontend HU3

En esta parte hemos desarrollado los siguientes archivos:

- HTML: admin_dashboard.html
- CSS: admin_dashboard.css
- JS: menu_dashboard.js

En estos archivos se ha implementado la estética de los bocetos, obteniendo como resultado lo siguiente:

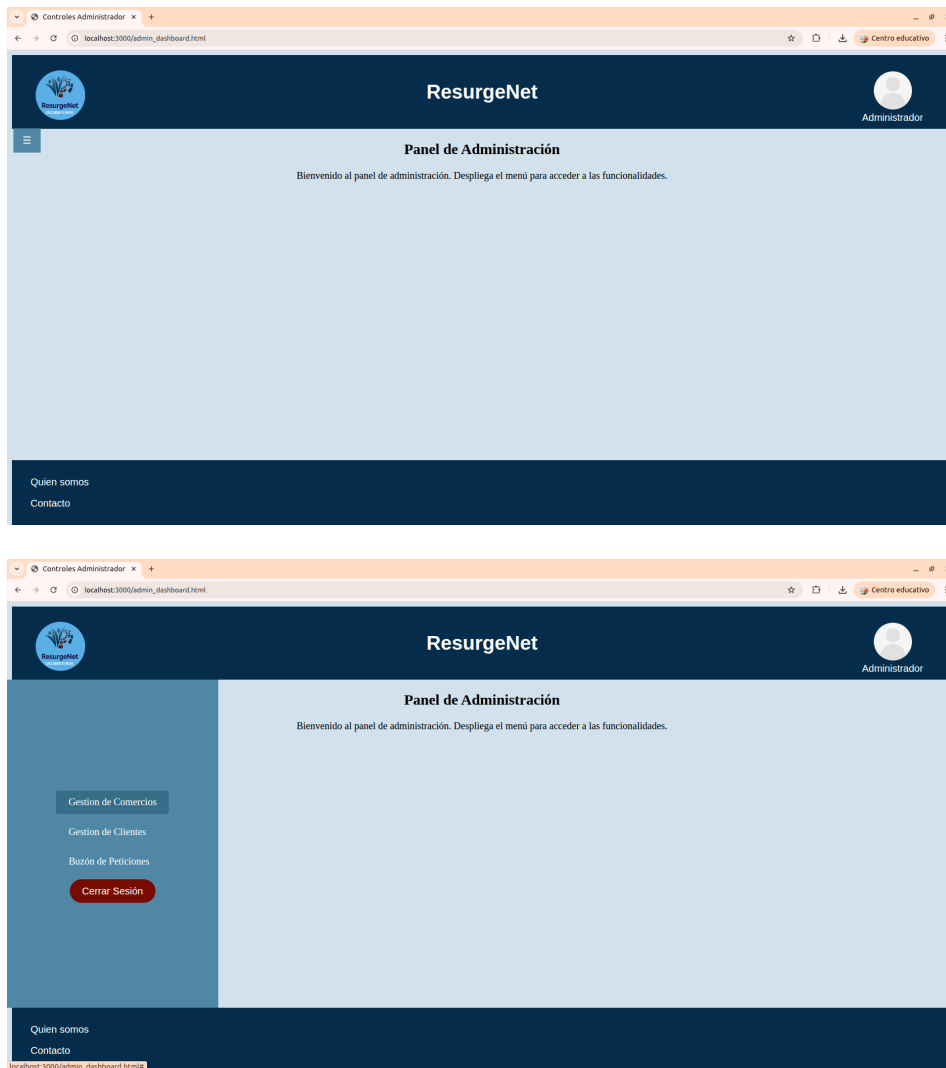


Figura 9: Resultados de la implementación estática de la HU3

Tenemos el botón para desplegar el menú, cuando este está abierto ese botón se oculta y nos aparecen las funcionalidades del administrador junto con el botón del cierre de sesión. Para plegar el menú solo hay que clicar fuera del mismo.

En el fichero js se ha implementado la funcionalidad de desplegar y ocultar el menú.

Parte dinámica - backend HU3

Antes de comenzar con el cierre de sesión se ha incluido la redirección desde el inicio de sesión, creado en la iteración anterior, a la página del dashboard del administrador que acabamos de crear.

Para llevar esto a cabo, se ha cambiado la respuesta del controlador de la api para que no solo nos aparezca el mensaje de "Sesión iniciada" sino que además nos haga un redirect a la página admin_dashboard.html. En el archivo login.js una vez que se recibe una respuesta exitosa, se utiliza window.location.href para redirigir automáticamente al usuario a la página del dashboard según la ruta indicada en el controlador.

Para el cierre de sesión del administrador hemos usado la gestión de sesiones de Laravel, almacenando el `id_admin` al iniciar sesión. Con esto podemos mantener la sesión activa sin tener que recurrir a tokens.

La forma en la que está montado el proyecto separa frontend de backend, ya que se encuentran en dominios distintos, y esto ha implicado que tengamos que modificar el archivo `VerifyCsrfToken`, un middleware cuya función principal es proteger la app de ataques del tipo Cross-Site Request Forgery. Este tipo de ataques consisten en engañar al usuario autenticado para que envíe una solicitud maliciosa a una app web, permitiendo que el atacante realice acciones no autorizadas como transferir datos. Por defecto se genera un token único para cada sesión de usuario, y al tener una arquitectura con front y backend separados este token se ve bloqueado. Al añadir las rutas en el fichero conseguimos que la comunicación en la api sea correcta entre ambos entornos. Con esto cubrimos la primera prueba de aceptación para esta historia de usuario: cerramos sesión y redirigimos a la landing page.

El archivo se encuentra en la siguiente ruta: `app/Http/Middleware/VerifyCsrfToken`.

<https://Laravel.com/docs/12.x/csrf>

<https://owasp.org/www-community/attacks/csrf>

Parte estática - frontend HU4

Dado que para permitir el inicio de sesión del validador de comercios se ha reutilizado la implementación del inicio de sesión del administrador (desarrollada para la HU2), no ha sido necesario realizar ninguna modificación o implementación adicional en el frontend para esta historia de usuario.

Parte dinámica - backend HU4

Antes de comenzar con la lógica del inicio de sesión del validador de comercio hemos tenido que modificar la base de datos para adaptarla al nuevo diagrama Entidad-Relación. Los pasos seguidos han sido:

- Borrado de la tabla `Administrador`
- Creación de la tabla `Usuario`, contiene los mismos atributos que la tabla `Administrador`
- Creación del trigger para encriptar la contraseña
- Inserción de los usuarios y del nuevo rol `Validador_comercio` en las tablas correspondientes

Una vez adaptada la base de datos hemos tenido que adaptar el inicio de sesión y el cierre de sesión del administrador, es decir, las historias de usuario dos y tres, para que apunten a la nueva tabla `Usuario` cuando se lleva a cabo la comprobación de las credenciales.

Tras esto hemos pasado a implementar el inicio de sesión del validador de comercios, para ello hemos definido la función `loginUser` en el archivo `AuthController.php` que integra el inicio de sesión de los diferentes usuarios del sistema. Esta modificación implicó también realizar cambios en las rutas, de manera que apunten correctamente a la nueva funcionalidad implementada.

La lógica de la función es la misma que la utilizada para el inicio de sesión del administrador, con la siguiente diferencia: la *redirección según el rol*, de manera que cada usuario accede al dashboard correspondiente según el valor del atributo `rol` en la tabla `Usuario`.

Una vez tenemos todo lo anterior correctamente implementado podemos pasar a hacer las pruebas de aceptación que se describieron anteriormente:

- Entrar con el usuario que tenemos en la base de datos como validador de comercios y que se permita iniciar sesión
- Entrar con un usuario que no está registrado y que no nos deje iniciar sesión, nos aparece un mensaje de “Credenciales incorrectas”

3.6.3. Revisión por parte de los tutores

Durante la revisión de los bocetos llevados a cabo en esta iteración, los clientes dijeron que el diseño les parecía adecuado y coherente. La modificación comentada ha sido la incorporación de la validación del formulario de inicio de sesión tanto para el validador de comercios como para el administrador.

En general, la retroalimentación fue positiva y la petición indicada se tendrá en cuenta en las siguientes iteraciones.

3.6.4. Retrospectiva de la iteración

Durante esta segunda iteración se han llevado a cabo casi en su totalidad las historias de usuario planificadas, solo ha quedado por hacer una de las validaciones del cierre de sesión del administrador: Si el usuario quiere ir hacia atrás en la web no puede ver el contenido. No se ha podido llevar a cabo esa validación por los siguientes motivos:

- Modificación del inicio de sesión del administrador, se tuvo que cambiar el original para que almacenará la sesión del mismo
- Familiarización con la gestión de sesiones con Laravel
- Modificación del esquema de base de datos para el inicio de sesión del validador de comercio

Como aspecto a mejorar, hemos visto la necesidad de profundizar el manejo de las sesiones en Laravel. Esto ha requerido más tiempo del que habíamos estimado, dejando la validación sobre esa historia pendiente, tal y como hemos explicado.

El resto de historias planificadas y objetivos han sido llevados a cabo satisfactoriamente.

Para la siguiente iteración se mantendrá la dinámica y planificación seguida. Se añadirá como objetivo extra terminar correctamente el cierre de sesión del administrador e implementar la validación del formulario de inicio de sesión, junto al resto de historias de usuario planificadas inicialmente.

3.7. Iteración 3

En esta tercera iteración, que abarca desde el 6 de Noviembre al 20 de Noviembre, vamos a desarrollar las siguientes historias de usuario:

HU3 - Finalización del cierre de sesión del administrador

De esta historia solo queda pendiente finalizar la tarea relacionada con la HU3, que es la siguiente validación: Si el usuario quiere ir hacia atrás en la web no puede ver el contenido. Esto se realizará antes de la finalización de la iteración.

HU5 - Como comercio quiero tener un inicio de sesión personalizado

Las tareas relacionadas con esta historia son:

- Diseño de la interfaz - 9 de Noviembre
- Diseño de la BD - 9 de Noviembre
- Revisión BD - 11 de Noviembre
- Implementación estática - 12 de Noviembre
- Implementación dinámica - 14 de Noviembre
- Pruebas de aceptación - 14 de Noviembre
 - Comprobación de que el usuario existe
 - Comprobación de que accede solo si está registrado

HU 6 - Como consumidor quiero contar con una plataforma de registro

Las tareas relacionadas con esta historia son:

- Diseño de la interfaz - 16 de Noviembre
- Diseño de la BD - 16 de Noviembre
- Revisión BD - 17 de Noviembre
- Implementación estática - 18 de Noviembre
- Implementación dinámica - 19 de Noviembre
- Pruebas de aceptación - 19 de Noviembre
 - Comprobación de que el usuario se registra con éxito
 - Comprobación de que los campos del formulario son obligatorios
 - Comprobación de que la información introducida en los campos es correcta
 - Comprobación de que se guarda correctamente en la base de datos toda su información relacionada
 - Comprobación de que no se registre si ya existe

Mejoras en el inicio de sesión del administrador y el validador de comercios

Vamos a añadir la validación de los campos del formulario de inicio de sesión. Esta tarea se llevará a cabo antes de que finalice la iteración.

3.7.1. Diseño de la interfaz y de la BD

Primero se llevarán a cabo los bocetos correspondientes a la HU5, el inicio de sesión del comercio. Para esta historia no tenemos que diseñar nada nuevo ya que reutilizaremos los bocetos del inicio de sesión del administrador. Se puede ver en la Figura 4.

Por otro lado, la base de datos sí que tendrá modificaciones. El Comercio va a tener unos atributos específicos, por lo que vamos a sacarlo como entidad aparte que hereda de la entidad Usuario para solo añadir esos atributos específicos s. Quedaría de la siguiente manera:

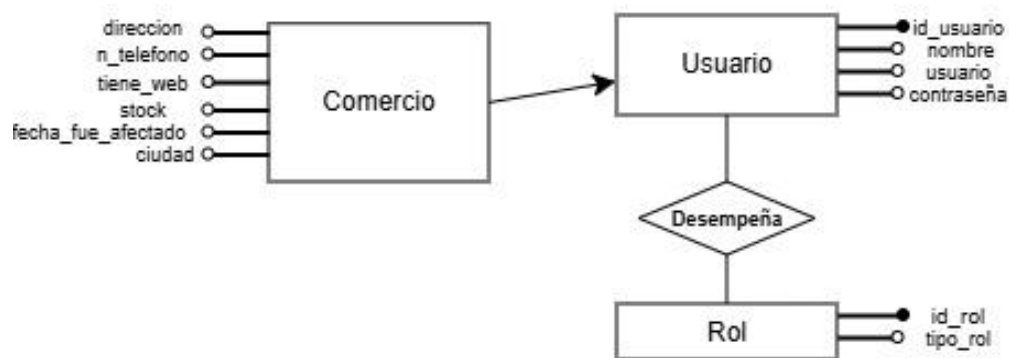


Figura 10: esquema de BD para la HU5

Para el registro de usuarios, HU6, tenemos que diseñar bocetos y realizar un nuevo diseño para la BD. La interfaz de usuario va a tener el siguiente aspecto:

Figura 11: boceto interfaz HU6, registro de consumidores

Como podemos ver, contamos con un formulario donde se van a requerir datos al usuario para que los rellene. El formulario mostrará todos los campos en una vista, y con esto nos aseguramos de mejorar su accesibilidad para personas con movilidad motora reducida. Cada uno de los campos será obligatorio y se indicará qué datos necesitan ser completados. Hemos incluido el campo fecha de nacimiento, lo que nos permite verificar que el usuario cumple la edad mínima legal necesaria para realizar compras online y aporta una capa adicional de identificación.

La BD quedará de la siguiente manera:

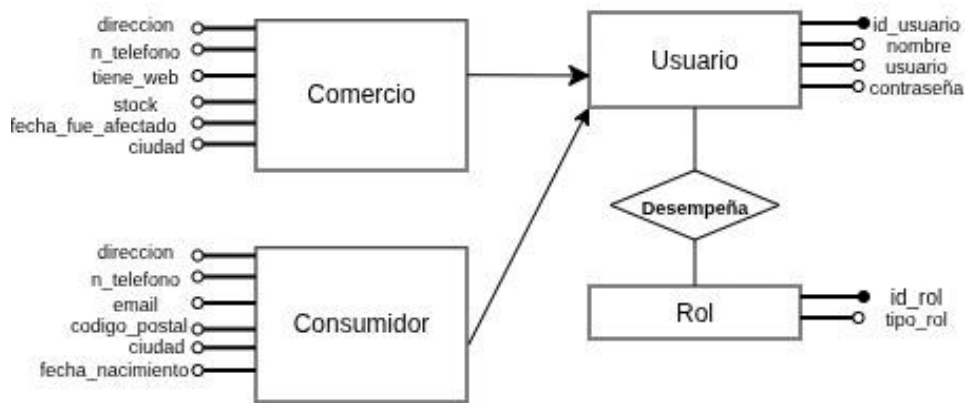


Figura 12: esquema entidad relación para el registro de consumidores

Al igual que para los comercios, los consumidores heredarán de usuario los atributos comunes y especificarán los propios en su entidad correspondiente.

3.7.2. Implementación estática y dinámica

Finalización HU3

Para la correcta finalización del cierre de sesión del administrador teníamos que asegurarnos de que una vez cerrada la sesión, al volver hacia atrás no se pudiera visualizar el dashboard del administrador ya que eso era una brecha de seguridad en nuestro sistema. Para implementar esto se ha llevado a cabo lo siguiente:

- Control de la sesión en el servidor: en el archivo `web.php` proporcionado por Laravel nos hemos asegurado de que antes de cargar el contenido, el servidor verifique si el usuario tiene la sesión válida, es decir si sigue existiendo la variable de sesión. Si ésta no existe, el servidor detiene la carga del dashboard del administrador y redirige a la página de inicio de sesión implementada en la primera iteración.
- Limpieza del historial al hacer el cierre de sesión: en el archivo `cierre_sesion_admin.js` hemos implementado un método que reemplaza la página actual por la landing page en el historial del navegador, en lugar de añadir una nueva entrada solamente. Con esto invalidamos la función del botón “atrás” evitando volver al dashboard
- Verificación: en el archivo `evitar_atras.js` hemos ideado una función que verifica si la página se ha cargado desde la caché, así, si se detecta que la página es una versión antigua, el script envía una nueva petición de verificación de la sesión. Si la respuesta del servidor indica que la sesión no está activa, se hace una redirección al inicio de sesión.

Tras las modificaciones llevadas a cabo tanto en el front como en el back conseguimos solucionar este problema de seguridad y finalizar esta historia de usuario. Toda esta funcionalidad desarrollada se aprovechará para llevar a cabo el cierre de sesión del resto de los usuarios de la plataforma.

Parte estática - frontend HU5

La implementación del inicio de sesión para comercios reutiliza la misma interfaz que se diseñó en la HU3 para el administrador, por lo que no ha sido necesario crear nuevos archivos: seguimos usando `inicio_sesion.html`, `inicio_sesion.css` y `login.js`.

La diferencia se encuentra en el backend: al iniciar sesión, el sistema verifica las credenciales y determina el tipo de usuario consultando la tabla usuario, pero tendremos que incluir en la base de datos tanto el nuevo rol como la lógica ideada en la Figura 10. De esta forma, aunque la interfaz sea idéntica, el comportamiento del sistema se adapta según el rol, redirigiendo posteriormente al usuario a la página o dashboard correspondiente.

Parte dinámica - backend HU5

Para realizar la implementación dinámica del inicio de sesión para los comercios tenemos que comenzar por implementar el esquema de base de datos mostrado en la Figura 10. Primero tenemos que definir el nuevo rol en la tabla Rol de la siguiente manera:

```
INSERT INTO rol(tipo_rol) VALUES ('Comercio');
```

Una vez tenemos el rol definido pasamos a implementar la tabla Comercio que como vemos en el esquema entidad relación hereda de Usuario e incorpora nuevos atributos propios del comercio.

```
CREATE TABLE comercio(id INT PRIMARY KEY, ciudad VARCHAR(30) NOT NULL, direccion VARCHAR(30) NOT NULL, n_telefono VARCHAR(10) NOT NULL, tiene_web BOOLEAN DEFAULT 0, FOREIGN KEY (id) REFERENCES usuario(id_usuario) ON DELETE CASCADE);
```

Simulamos herencia mediante una relación 1:1 entre Usuario y Comercio, usando id como clave compartida. También hemos implementado que si se borra un usuario de la tabla y es comercio entonces la información relacionada con el comercio en su tabla se elimina también. Esto se consigue con ON DELETE CASCADE.

Una vez tenemos la tabla comercio creada tenemos que hacer las siguientes inserciones:

- Inserción de un usuario de rol comercio en la tabla usuario
 - *INSERT INTO usuario(nombre, usuario, password, rol) VALUES ('Manuel Garrido', 'manuGr', 'comercio1', 4);*
- Inserción del resto de la información del comercio en su tabla correspondiente
 - *INSERT INTO comercio(id, ciudad, direccion, n_telefono, tiene_web) VALUES (3, 'Granada', 'Calle Don Quijote 5', 666899760, 1);*

Le asociamos el id del usuario que hemos insertado en el paso anterior para que queden unidas las tablas y se refleje la herencia.

Una vez terminada la implementación de la BD no tenemos que modificar nada más, la función de inicio de sesión de usuarios se ideó en la HU3 (inicio de sesión del administrador) y es válida para todos los usuarios de nuestro sistema. Cuando se inicia la sesión somos redirigidos a la *landing page*, esto es provisional hasta que se cree el *dashboard* de los comercios.

Nos queda llevar a cabo las validaciones:

- Entrar con el usuario que tenemos en la base de datos como administrador y que se permita iniciar sesión
- Entrar con un usuario que no está registrado y que no nos deje iniciar sesión, nos aparece un mensaje de "Credenciales incorrectas"

Parte estática - frontend HU6

En esta parte hemos desarrollado los siguientes archivos:

- HTML: registro_consumidores.html
- CSS: registro.css
- JS: validacion_registro.js

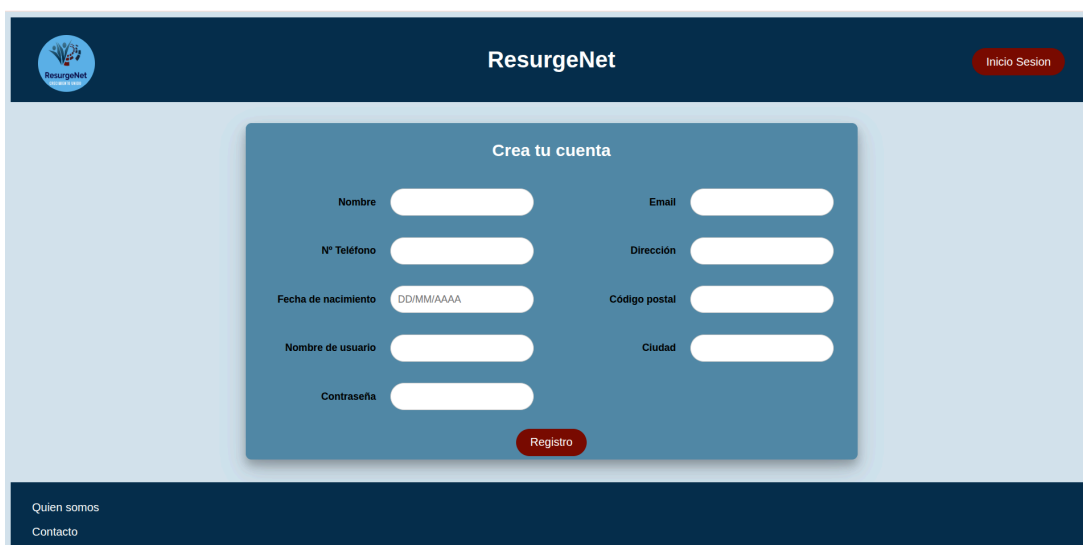
En estos archivos se ha implementado la estética de los bocetos mostrados en la Figura 11. Hemos implementado un formulario de registro que tiene una barra en el lateral izquierdo que nos permite hacer scroll para rellenar todos los campos. Estos campos son obligatorios, no podemos enviar un formulario vacío o semivacío.

Además hemos llevado a cabo la comprobación de que los campos se rellenan con el formato adecuado antes de que se envíe el formulario, por tanto antes de que se guarden los datos en nuestra base de datos. Estas validaciones las llevamos a cabo usando JS, por tanto se desarrollan en la parte del cliente. Al filtrar los datos incorrectos antes de que lleguen al servidor, reducimos el número de peticiones innecesarias, lo que evita sobrecargar el servidor y optimiza el uso de recursos. Las validaciones llevadas a cabo han sido:

- **Email** → comprobamos que contiene @, que antes de esta tenga contenido y que el dominio sea gmail.com o hotmail.com
- **Fecha** → nos aseguramos que tenga el formato DD/MM/YYYY y que no se introduzca una fecha futura
- **Código postal** → nos aseguramos de que tiene cinco dígitos
- **Número de teléfono** → comprobamos que es de nueve dígitos
- **Contraseña** → tiene que tener mínimo 6 caracteres y al menos un número

Las restricciones aplicadas al formulario son justificadas debido a la necesidad de garantizar la integridad, homogeneidad y seguridad de los datos que son introducidos en el sistema. Delimitar los dominios de email simplifica el control de usuarios legítimos en esta fase inicial del proyecto. Por otro lado, la longitud fija para el número de teléfono y el código postal adopta el sistema nacional previniendo errores de formato. En estas primeras fases se contempla que el alcance de esta web sea a nivel nacional.

Esto ha quedado de la siguiente manera:



The image shows a web application interface for 'ResurgeNet'. At the top, there is a dark blue header with the 'ResurgeNet' logo on the left, the text 'ResurgeNet' in the center, and a red button labeled 'Inicio Sesión' on the right. Below the header, the main content area has a light blue background. In the center, there is a white box with a blue border titled 'Crea tu cuenta'. Inside this box, there are several input fields for registration: 'Nombre', 'Email', 'Nº Teléfono', 'Dirección', 'Fecha de nacimiento' (with a placeholder 'DD/MM/AAAA'), 'Código postal', 'Nombre de usuario', 'Ciudad', and 'Contraseña'. A red button labeled 'Registro' is positioned at the bottom of the white box. At the bottom of the page, there is a dark blue footer with the text 'Quien somos' and 'Contacto' on the left.

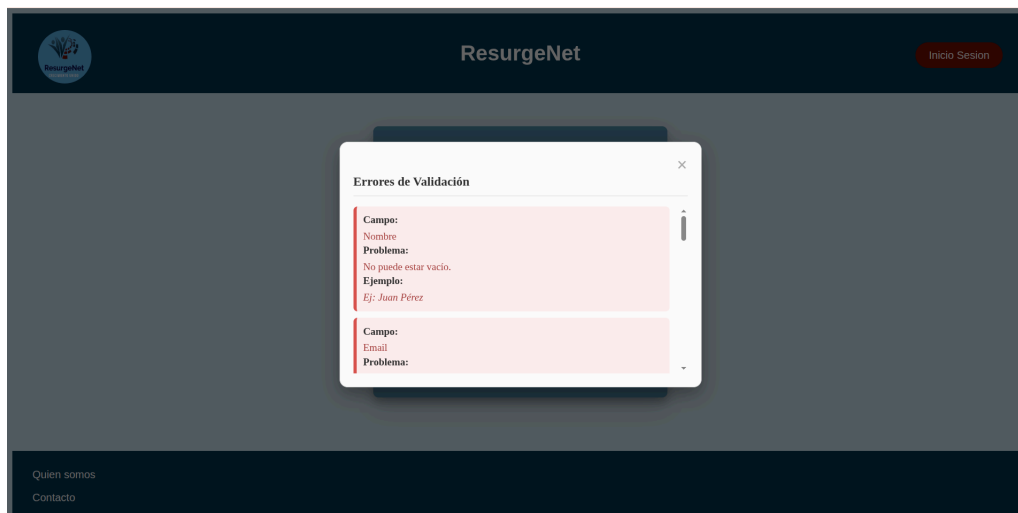


Figura 13: funcionamiento de la validación del formulario

Como se puede ver, cuando dejamos los campos vacíos o introducimos mal el contenido de los mismos nos aparece una pantalla que nos indica qué campo tiene el error, el tipo de error y un ejemplo de cómo se debe completar.

Parte dinámica - backend HU6

Para realizar la implementación dinámica del inicio de sesión para los comercios tenemos que comenzar por implementar el esquema de base de datos mostrado en la Figura 12. Primero tenemos que definir el nuevo rol en la tabla Rol de la siguiente manera:

```
INSERT INTO rol(tipo_rol) VALUES ('Consumidor');
```

Una vez tenemos el rol definido pasamos a implementar la tabla Consumidor que como vemos en el esquema entidad relación hereda de Usuario e incorpora nuevos atributos propios del consumidor.

```
CREATE TABLE consumidor(id INT PRIMARY KEY, direccion VARCHAR(30) NOT NULL,
ciudad VARCHAR(30) NOT NULL, cod_postal INT NOT NULL, n_telefono VARCHAR(10)
NOT NULL, email VARCHAR(35) NOT NULL, fecha_nac DATE NOT NULL, FOREIGN KEY
(id) REFERENCES usuario(id_usuario) ON DELETE CASCADE);
```

Simulamos herencia mediante una relación 1:1 entre Usuario y Consumidor, usando id como clave compartida. También hemos implementado que si se borra un usuario de la tabla y es consumidor entonces la información relacionada con el consumidor en su tabla se elimina también. Esto se consigue con ON DELETE CASCADE.

Una vez tenemos la tabla comercio creada tenemos que hacer las inserciones desde el formulario de registro. Para llevar a cabo esta lógica hemos implementado la función registerConsumer en el fichero AuthController, que es el que recoge todas las funciones que interactúan con la base de datos. Funciona de la siguiente manera:

- Inserción en la tabla usuario → tenemos que insertar los campos nombre, usuario y contraseña en la tabla padre, en nuestro caso usuario, para hacer la herencia de tablas
- Cambio de formato de la fecha → el consumidor al registrarse introduce la fecha con el formato DD/MM/YYYY y en sql se trabaja con las fechas en formato YYYY/MM/DD por lo que es necesario realizar una conversión de tipos

- Inserción en la tabla consumidor → insertamos el resto de campos del formulario en la tabla consumidor teniendo en cuenta la clave externa a la tabla usuario que es el id, este toma el valor que haya generado para la tabla usuario. De esa forma ambas tablas se enlazan y queda representada la herencia de clases.

Si la inserción ha sido correcta mostramos el mensaje por pantalla y redirigimos al consumidor a la página de inicio de sesión. Esta lógica se ha llevado a cabo en el fichero `validacion_registro.js`.

Una vez llevado tenemos en funcionamiento lo anterior pasamos a realizar las validaciones:

- La comprobación de que los campos son rellenados correctamente se hace en la parte del cliente, es decir, en el front end. Se ha ilustrado el funcionamiento en la figura 13.
- Una vez que se muestra el mensaje de que el registro ha sido correcto, comprobamos que se haya actualizado la base de datos, para ello ejecutamos el comando *SELECT * FROM consumidor*; y podemos ver todos los que tenemos añadidos hasta la fecha.

```
mysql> SELECT * FROM consumidor;
```

id	direccion	ciudad	cod_postal	n_telefono	email	fecha_nac
9	Gran Vía 5	Madrid	15002	666085950	sofiP_22@gmail.com	1996-05-12
10	Calle Cuevas Almanzora 8	Sevilla	12008	679896335	pedromartin03@gmail.com	1960-09-07
11	Calle Mediterraneo 45	Granada	18184	647963210	lauriG@gmail.com	2000-02-14

```
3 rows in set (0.00 sec)

mysql>
```

Figura 14: Tabla consumidores

Mejoras en el inicio de sesión del administrador y el validador de comercios

Hemos modificado el archivo `login.js` creado para el inicio de sesión del administrador en la HU2 para que, antes de que los datos sean comprobados por la base de datos y seamos redirigidos a la página del administrador, nos aseguremos de que los campos están todos rellenos y la contraseña contiene mínimo 6 caracteres.

Al igual que para la validación del formulario de registro de la HU6, hemos creado una ventana emergente que nos va diciendo qué errores tenemos y un ejemplo de cómo introducir los campos correctamente.

3.7.3. Revisión por parte de los tutores

Durante la revisión de los bocetos llevados a cabo en esta iteración, los clientes sugirieron modificaciones sobre el boceto del formulario de registro para favorecer la accesibilidad del mismo. Estos cambios han sido llevados a cabo obteniendo el boceto final que podemos ver en la figura 11.

En general, la retroalimentación fue positiva y la petición indicada se realizó con éxito.

3.7.4. Retrospectiva de la iteración

Para esta tercera iteración la organización y planificación del trabajo ha sido adecuada, así como la distribución de tareas y la definición de los objetivos. Esto ha permitido que se lleven a cabo todos ellos.

La comunicación con los tutores ha sido fluida y ha contribuido a validar correctamente cada fase del desarrollo antes de su implementación. Para la siguiente iteración vamos a añadir un estudio de la LPDGDD para asegurarnos de que estamos llevando a cabo de forma correcta el tratamiento de datos de nuestros usuarios del sistema.

En definitiva, la metodología seguida ha sido efectiva por lo que para las siguientes iteraciones se mantendrá la misma dinámica.

3.8. Iteración 4

En esta cuarta iteración, que abarca desde el 21 de Noviembre al 5 de Diciembre, vamos a desarrollar las siguientes historias de usuario:

HU7 - Inicio de sesión del consumidor

Las tareas relacionadas con esta historia son:

- Diseño de la interfaz - 24 de Noviembre
- Diseño de la BD - 26 de Noviembre
- Revisión BD - 29 de Noviembre
- Implementación estática - 2 de Diciembre
- Implementación dinámica - 4 de Diciembre
- Pruebas de aceptación - 5 de Diciembre
 - Comprobación de que el usuario existe
 - Comprobación de que accede solo si está registrado
 - Comprobación de que se accede a su cuenta y no a la de otro usuario

Mejoras

Vamos a modificar la forma en la que se ven los mensajes de ‘Sesión iniciada’ para que sigan la misma estética que las ventanas emergentes del registro de consumidores. También vamos a estudiar la LPDGDD para ver si estamos llevando a cabo un tratamiento correcto de los datos de los usuarios y realizar las modificaciones que sean necesarias.

3.8.1. Diseño de la interfaz y de la BD

Esta historia de usuario sigue la misma línea que las historias de inicio de sesión del administrador y del validador de comercio. Por tanto, no es necesario hacer ninguna modificación de la base de datos, al igual que no es necesario idear un nuevo boceto de la interfaz. Nos quedamos con el que se representa en la figura 4.

3.8.2. Implementación estática y dinámica

Para el *inicio de sesión del consumidor* solo es necesario hacer una pequeña modificación en el backend. Esta modificación es la redirección según el rol del usuario, por lo que añadimos al consumidor y establecemos que cuando inicie sesión sea redirigido al index.html. Esto se hará de forma provisional hasta que se construya su dashboard. El resto de funciones son comunes a lo que desarrollamos para el resto de inicios de sesión.

En cuanto a las *mejoras del inicio de sesión de los usuarios*, se ha llevado a cabo una mejora en el aspecto de las notificaciones. Por tanto, sólo hemos realizado cambios a nivel de frontend. Hemos añadido el modal que hemos creado en la validación de los formularios en el script de inicio de sesión y cierre. Además hemos añadido la estética deseada en los css

correspondientes. Con esto hemos unificado la estética ideada en la validación de formularios, quedando de la siguiente manera:

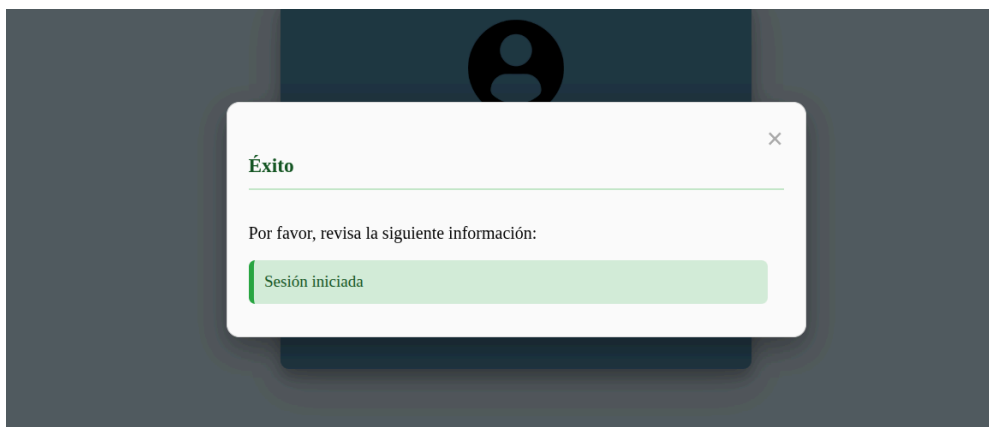


Figura 15: Mensajes de inicio y cierre de sesión con nueva estética

Tras 3 segundos el usuario será redirigido a su apartado dentro del sistema. En caso del administrador se le mostrará su dashboard.

Como última parte de esta iteración, hemos revisado la LOPDGDD para asegurarnos de que hacemos un tratamiento de los datos de los usuarios de forma correcta. Hemos consultado el BOE y, en concreto, se han tenido en cuenta el *artículo 4*, relativo a la exactitud de los datos; el *artículo 5*, sobre el deber de confidencialidad; el *artículo 15*, que regula el derecho de supresión; y el *artículo 32*, referente al bloqueo de datos. Estos artículos establecen la necesidad de garantizar que los datos personales sean correctos, estén protegidos frente a accesos no autorizados y puedan ser eliminados o bloqueados cuando proceda.

Durante la implementación llevada a cabo en estas cuatro iteraciones se han implementado medidas como el cifrado de contraseñas, acceso controlado al sistema mediante roles, y control de sesiones de usuario. En las siguientes iteraciones se seguirá mejorando el tratamiento de los datos permitiendo la actualización de los mismos, así como la edición y borrado.

3.8.3. Revisión por parte de los tutores

Durante la revisión de los bocetos llevados a cabo en esta iteración, los clientes dijeron que el diseño les parecía adecuado y coherente. Se solicitó la revisión de la LOPDGDD para corroborar que el tratamiento de los datos que se ha ido llevando a cabo era correcto.

En general, la retroalimentación fue positiva y la petición indicada se tendrá en cuenta en las siguientes iteraciones.

3.8.3. Retrospectiva de la iteración

Para esta cuarta iteración la organización y planificación del trabajo ha sido adecuada, así como la distribución de tareas y la definición de los objetivos. Esto ha permitido que se lleven a cabo todos ellos.

La comunicación con los tutores ha sido fluida y ha contribuido a validar correctamente cada fase del desarrollo antes de su implementación. Para la siguiente iteración vamos a añadir un

estudio de la LPDGDD para asegurarnos de que estamos llevando a cabo de forma correcta el tratamiento de datos de nuestros usuarios del sistema.

En definitiva, la metodología seguida ha sido efectiva por lo que para las siguientes iteraciones se mantendrá la misma dinámica.

3.9. Iteración 5

En esta quinta iteración, que abarca desde el 22 de Diciembre al 5 de Enero, vamos a desarrollar las siguientes historias de usuario:

HU8 - Como comercio quiero tener un espacio en el que subir los productos para empezar con su venta

Las tareas relacionadas con esta historia son:

- Diseño de la interfaz - 23 de Diciembre
- Diseño de la BD - 26 de Diciembre
- Revisión BD - 28 de Diciembre
- Implementación estática - 1 de Enero
- Implementación dinámica - 4 de Enero
- Pruebas de aceptación - 5 de Enero
 - Comprobación de acceso correcto al espacio del comercio
 - Comprobación de la validación del formulario
 - Comprobación de que aparece el producto insertado en el dashboard

3.9.1. Diseño de la interfaz y de la BD

En cuanto al diseño de la interfaz para esta historia de usuario, vamos a seguir con la estética llevada a cabo para el desarrollo del dashboard del administrador, que podemos ver en la figura 9, pero introduciremos una serie de cambios.



Figura 16: Interfaz para el alta de productos (HU8)

Cuando el comercio acceda a su dashboard tras iniciar sesión, se mostrará una primera pantalla que contará con un menú desplegable desde el cual tendrá acceso a sus funciones. Cuando acceda a “Alta de productos” será redirigido a un espacio que tendrá un formulario de registro de los mismos. En futuras historias de usuario se implementará un espacio en el que pueda ver los productos que tiene dados de alta.

En cuanto a la base de datos, necesitaremos introducir una nueva entidad, Productos, así como una nueva relación, Tiene, con la entidad Comercio. Con esto representamos que los comercios tienen productos que serán a los que tendrá acceso el consumidor. El diagrama entidad relación quedaría de la siguiente forma tras introducir el cambio expuesto:

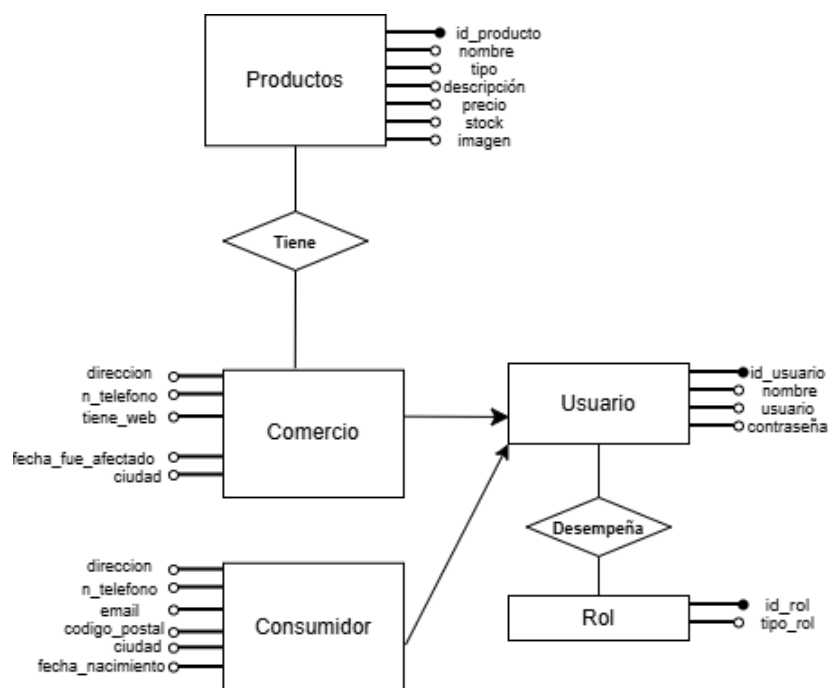


Figura 17: Diagrama E-R actualizado para el dashboard del comercio

3.9.2. Implementación estática y dinámica

Para la **implementación estática** del dashboard del comercio y el alta de productos vamos a reutilizar lo implementado para el dashboard del administrador. Vamos a tomar la estructura de esta página como base e introduciremos los cambios expuestos en la Figura 16. Por tanto crearemos los siguientes archivos:

- HTML: comercio_dashboard.html y alta_productos.html
- CSS: alta_productos.css
- JS: validar_producto.js

Una vez terminada la implementación de los bocetos el resultado ha sido el siguiente:

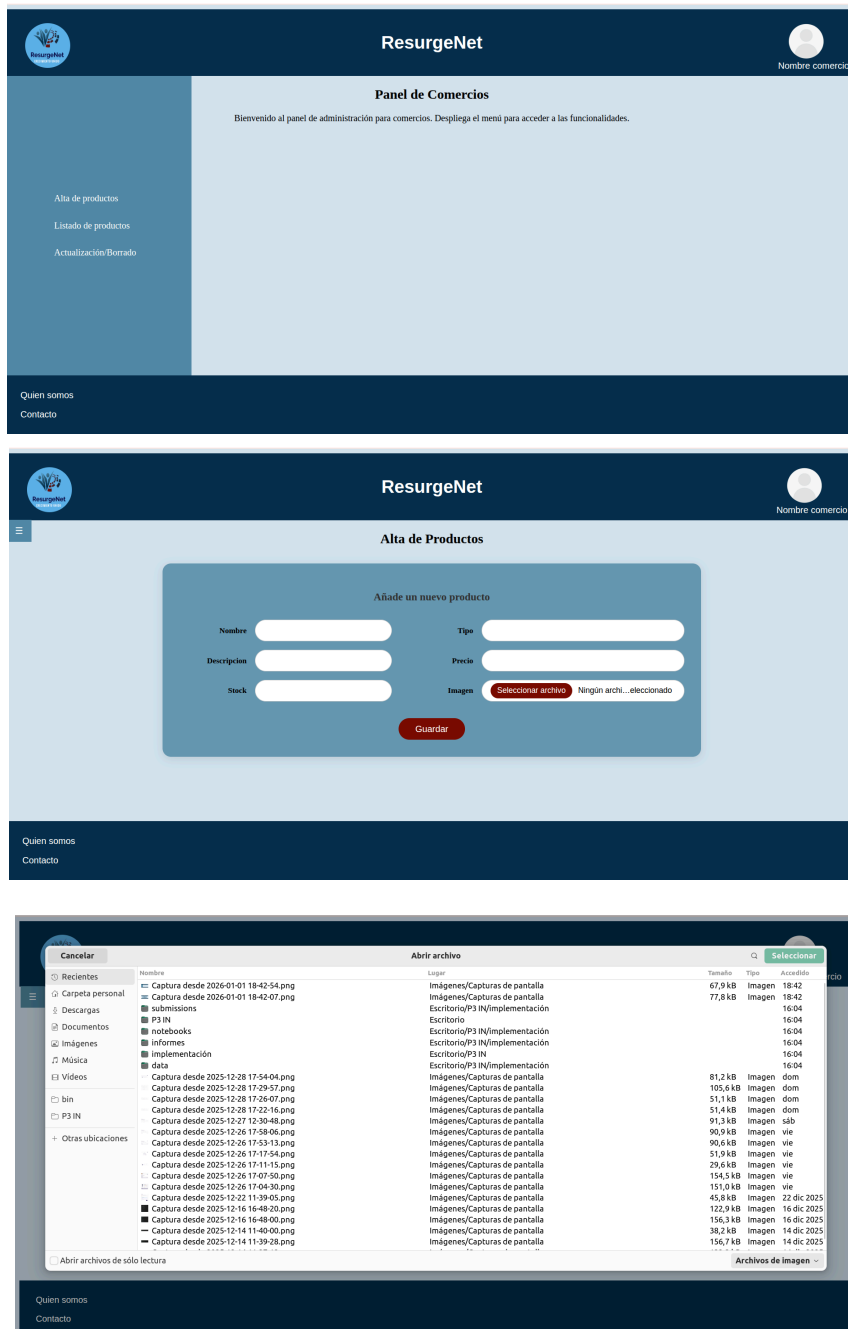


Figura 18: resultado de la implementación estática

Una vez que iniciamos sesión, si somos un comercio, nos reconduce a nuestro panel de control donde tenemos un menú desplegable con las distintas funciones. Una vez que entramos en “Alta de Productos” nos vamos a la segunda imagen que se ve en la figura 18, donde tenemos el formulario de alta. El funcionamiento es el mismo que el resto de formularios desarrollados cambiando el campo de la imagen, cuando pulsamos el botón “Seleccionar archivo” nos aparece nuestra carpeta local para seleccionar la que deseemos.

En el fichero .js tenemos la validación del formulario de subida, donde se comprueba que los campos no están vacíos, el formato decimal del número del precio y el formato de la imagen. Si algún campo tiene errores o está vacío nos mostrará los mensajes de error.

Si la validación del formulario es correcta, con este archivo mandamos la información recogida al backend para que se pueda hacer la inserción correspondiente en la tabla productos.

Así se visualiza la validación del formulario:

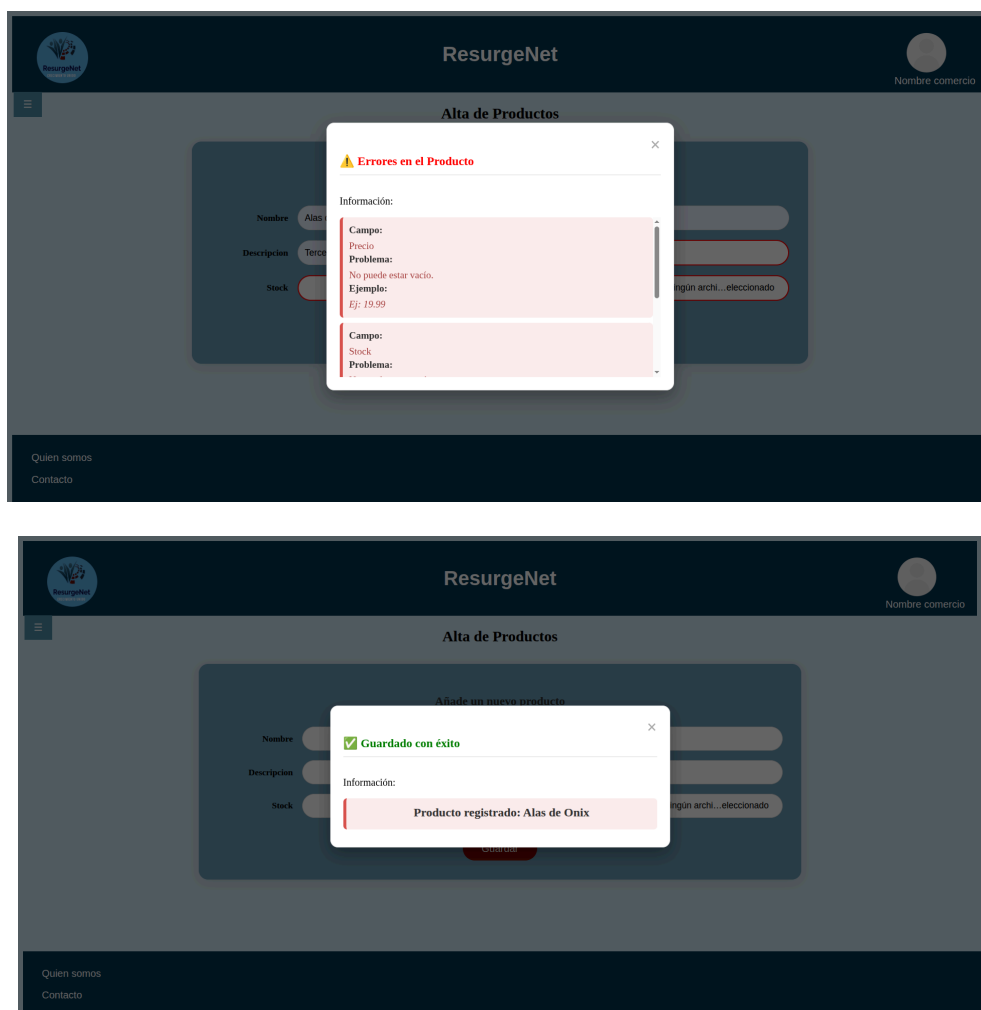


Figura 19: Salida de mensajes en la validación del formulario

Para la **implementación dinámica**, lo primero que debemos hacer es crear nuestra nueva entidad, Producto, en la base de datos con todos los atributos que podemos ver en la Figura 17. Para la creación de la tabla se ha utilizado el siguiente comando:

```
CREATE TABLE productos(id_producto INT AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(20) NOT NULL, tipo VARCHAR(30) NOT NULL, descripcion TEXT NOT NULL,
```

```
precio DECIMAL(10,2) NOT NULL, stock INT NOT NULL, imagen VARCHAR(255),
id_comercio INT NOT NULL, FOREIGN KEY (id_comercio) REFERENCES
usuario(id_usuario));
```

La imagen será guardada como la ruta de la misma, tenemos una referencia al id del comercio para saber qué productos pertenecen a cada comercio. Esto será útil para mostrar la lista de productos que tiene disponibles cada comercio en su área dentro de la web.

Una vez tenemos la tabla creada tenemos que hacer las inserciones desde el formulario de alta. Los pasos a seguir son:

- Definimos la ruta para el registro de nuevos productos en el archivo api.php
- Definimos la función registerProduct en el archivo AuthController.php. Esta función tendrá dos partes
 - Obtenemos el id del comercio para guardarlo en nuestra base de datos, esto nos permitirá obtener los productos propios de cada comercio. Para llevar esto a cabo hemos modificado la lógica del inicio de sesión para que nos devuelva el id del usuario y se guarde en el navegador hasta el cierre de sesión.
 - Gestionamos la imagen mediante una lógica de tratamiento de archivos mediante el objeto Request de Laravel. Con esto el sistema genera un nombre único para cada imagen, con esto evitamos colisiones de archivos, y los almacena en el directorio public/uploads/productos, devolviendo la ruta para la inserción en la base de datos.
 - <https://Laravel.com/docs/10.x/filesystem>
 - Obtenemos la información del formulario y la insertamos en la tabla productos creada previamente

Una vez que lo tenemos todo implementado pasamos a llevar a cabo las pruebas de aceptación.

- Comprobación del acceso al dashboard, esto se lleva a cabo correctamente ya que solo los usuarios que son comercios pueden acceder. El resto de usuarios son correctamente redirigidos a su área.
- Comprobación de la validación del formulario, como se ve en la Figura 19 se muestran correctamente los mensajes de error cuando el usuario deja los campos vacíos o introduce un formato incorrecto en los datos, al igual que se visualiza de forma correcta el mensaje de éxito tras una inserción.
- Comprobación de la inserción en la base de datos. Como podemos ver en la siguiente imagen, la tabla de productos se completa correctamente tras mostrar en el front el mensaje de éxito.

```
mysql> SELECT * FROM productos;
```

id_producto	nombre	tipo	descripcion	precio	stock	imagen	id_comercio
1	Alas de Onix	Libro Fantasia	Tercera entrega de la saga Empirio escrito por Rebeca Yarros	23.90	5	uploads/productos/1767438955_alas_onix.jpg	3
2	La Asistente	Thriller	Primera entrega del thriller superventas escrito por Freida McFadden	20.99	3	uploads/productos/1767439069_LaAsistente.jpg	3

```
2 rows in set (0.01 sec)

mysql>
```

Figura 20: Estado de la tabla productos tras la inserción

3.9.3. Revisión por parte de los tutores

Durante la revisión de los bocetos llevados a cabo en esta iteración, los clientes dijeron que el diseño les parecía adecuado y coherente. En general, la retroalimentación fue positiva y útil para saber cómo seguir enfocando el trabajo en las siguientes iteraciones.

3.9.3. Retrospectiva de la iteración

Para esta iteración la organización y planificación del trabajo ha sido adecuada, así como la distribución de tareas y la definición de los objetivos. Esto ha permitido que se lleve a cabo todo lo planificado.

La comunicación con los tutores ha sido fluida y ha contribuido a validar correctamente cada fase del desarrollo antes de su implementación.

En definitiva, la metodología seguida ha sido efectiva por lo que para las siguientes iteraciones se mantendrá la misma dinámica.

3.10. Iteración 6

En esta sexta iteración que abarca desde el 29 de enero hasta el 12 de febrero se van a abordar las siguientes historias de usuario:

HU9 - Como administrador quiero tener acceso a la lista de comercios que han pasado la validación para poder proceder al alta de los mismos

Las tareas relacionadas con esta historia son:

- Diseño de la interfaz - 29 de Enero
- Diseño de la BD - 30 de Enero
- Revisión BD - 1 de Febrero
- Implementación estática - 3 de Febrero
- Implementación dinámica - 4 de Febrero
- Pruebas de aceptación - 4 de Febrero
 - Comprobación de acceso correcto al listado de comercios
 - Comprobación de que se da de alta correctamente al comercio en la base de datos
 - Comprobación de que si ya esta dado de alta el comercio no se puede volver a dar de alta

HU10 - Como administrador del sistema quiero poder dar de baja a los comercios

Las tareas relacionadas con esta historia son:

- Diseño de la interfaz - 5 de Febrero
- Diseño de la BD - 6 de Enero
- Revisión BD - 7 de Febrero
- Implementación estática - 7 de Febrero
- Implementación dinámica - 8 de Febrero
- Pruebas de aceptación - 8 de Febrero
 - Comprobación de acceso correcto al listado de comercios
 - Comprobación de que se elimina correctamente al comercio en la base de datos

- Comprobación de que el listado se actualiza correctamente tras eliminar uno de los comercios, es decir, que el eliminado no siga apareciendo

HU11 - Como administrador del sistema quiero poder dar de baja a los consumidores que realizarán compras en los comercios promocionados

Las tareas relacionadas con esta historia son:

- Diseño de la interfaz -9 de Febrero
- Diseño de la BD - 9 de Enero
- Revisión BD - 11 de Febrero
- Implementación estática - 11 de Febrero
- Implementación dinámica - 12 de Febrero
- Pruebas de aceptación - 12 de Febrero
 - Comprobación de acceso correcto al listado de consumidores
 - Comprobación de que se elimina correctamente al consumidor en la base de datos
 - Comprobación de que el listado se actualiza correctamente tras eliminar uno de los consumidores, es decir, que el eliminado no siga apareciendo

3.10.1. Diseño de la interfaz y de la BD

Primero se diseñarán los bocetos relacionados con la HU9, dar de alta a los comercios. Esta funcionalidad pertenece al administrador, por lo tanto, desde el panel de administración, que podemos ver en la figura 21, se le da acceso a un listado con los nombres de los comercios, donde cada uno tiene un botón para proceder con su alta. El boceto queda de la siguiente manera:



Figura 21: Boceto HU9, alta de los comercios tras validación.

La estructura de tablas que componen la base de datos quedará como está actualmente. El único cambio que se incorporará es la aparición de un nuevo campo en la tabla *Comercio* que es el *estado*. Esto tomará dos valores: “validación” o “alta”. Cuando un comercio mande la solicitud al validador, el estado del mismo será “validación”. Una vez que el validador dé el visto bueno al comercio, éste procederá a dar de alta el nuevo comercio validado en su panel de

administración, donde aparecerá, y el estado del comercio cambiará a “alta”. Por tanto la base de datos quedaría de la siguiente manera:

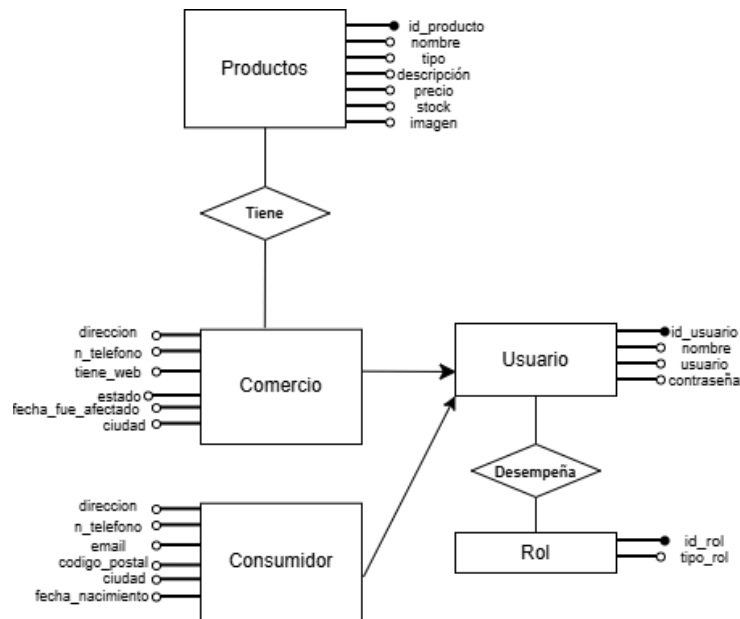


Figura 22: Diagrama E-R actualizado a la HU9

Para la HU10, eliminar un comercio, se sigue con el mismo boceto que para dar de alta a los mismos, solo que incluyendo un nuevo botón para eliminar (Fig. 22).



Figura 23: Boceto HU10, eliminación de comercios.

La base de datos para esta historia de usuario permanecerá de la misma manera que para la historia anterior.

Por último, diseñamos los bocetos correspondientes de la HU11, la eliminación de consumidores. Al igual que para la HU9, desde el panel de administración se accederá a esta funcionalidad donde veremos una lista de consumidores y un botón de eliminar para cada uno de ellos.



Figura 24: Boceto para la HU11, eliminar consumidores.

Con respecto a la base de datos, no necesitamos hacer modificaciones para esta historia de usuario.

3.10.2. Implementación estática y dinámica

Parte estática - frontend HU9

En esta parte hemos desarrollado los siguientes archivos:

- HTML: gestion_comercios_admin.html
- CSS: gestion_comercios_admin.css

En estos archivos se ha implementado el boceto que podemos ver en la figura 21 de los bocetos, obteniendo como resultado lo siguiente:

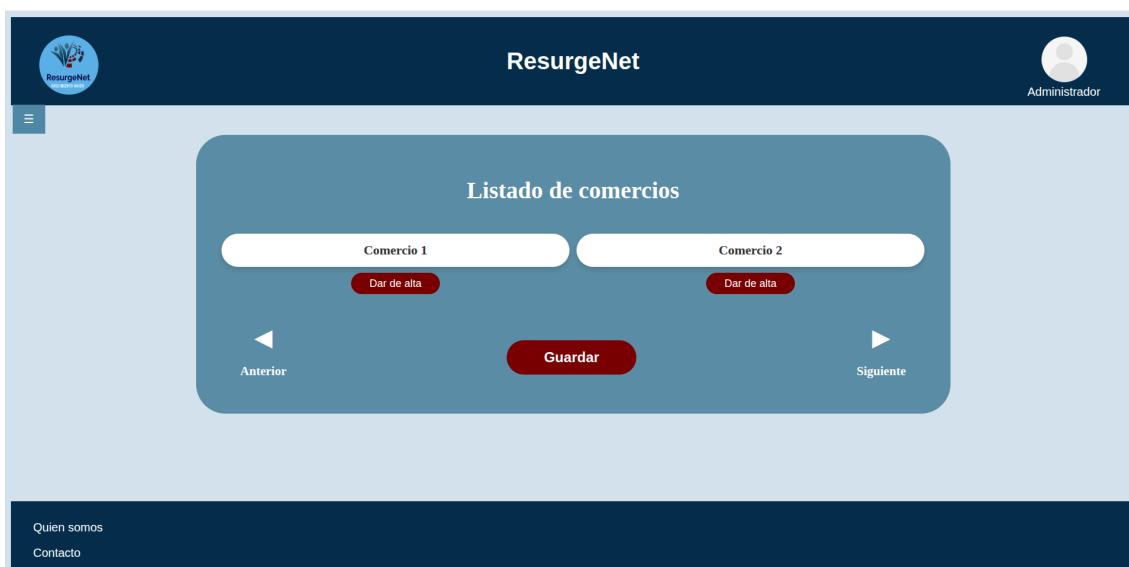


Figura 25: Implementación estática de la HU9

A este panel se accede desde el dashboard principal del administrador, mediante el menú lateral, puede verse en la figura 9. Contamos con un listado de los comercios que se encuentran en la base de datos, con el backend nos encargamos de volcar esa información, y con el botón para dar de alta a cada uno de ellos. Además tenemos en la parte inferior dos botones para la navegación por el listado (“Anterior” y “Siguiente”), contamos también con el botón “Guardar” para asegurar la constancia de los cambios realizados en cada parte del listado.

Parte dinámica - backend HU9

Para llevar a cabo la implementación dinámica de esta historia, primero tenemos que modificar la base de datos y añadirle el nuevo campo a la tabla de comercio. Este campo nos indica el estado en que se encuentra el mismo, si en validación o ya dado de alta cuando lleve a cabo la acción el administrador. Por tanto, la actualización del mismo está ligada al botón ‘Dar de alta’ que se encuentra en el panel del administrador.

Para incluir ese nuevo campo, estado, se realiza el siguiente comando sobre la base de datos:

```
ALTER TABLE comercio ADD estado ENUM('validacion','alta') DEFAULT 'validacion';
```

Lo hemos definido con un enumerado para que solo pueda tomar esos valores, de forma que aseguramos la integridad de los datos que se almacenan.

El resto de implementación dinámica correspondiente a esta historia se llevará a cabo en la siguiente iteración.

Parte estática - frontend HU10

Para esta historia trabajamos con lo creado en la HU9, ya que vamos a añadir un nuevo botón para cada comercio que le permita al administrador del sistema eliminar el mismo. Siguiendo el boceto, figura 23, la implementación quedaría de la siguiente manera:



Figura 26: Implementación estática de la HU10

Parte estática - frontend HU11

En esta parte se han desarrollado los siguientes archivos:

- HTML: gestion_consumidores_admin.html

- CSS: gestion_cons_comer_admin.css
- JS: gestion_consumidores.js

En estos archivos se han implementado la estética de los bocetos obteniendo el siguiente resultado:

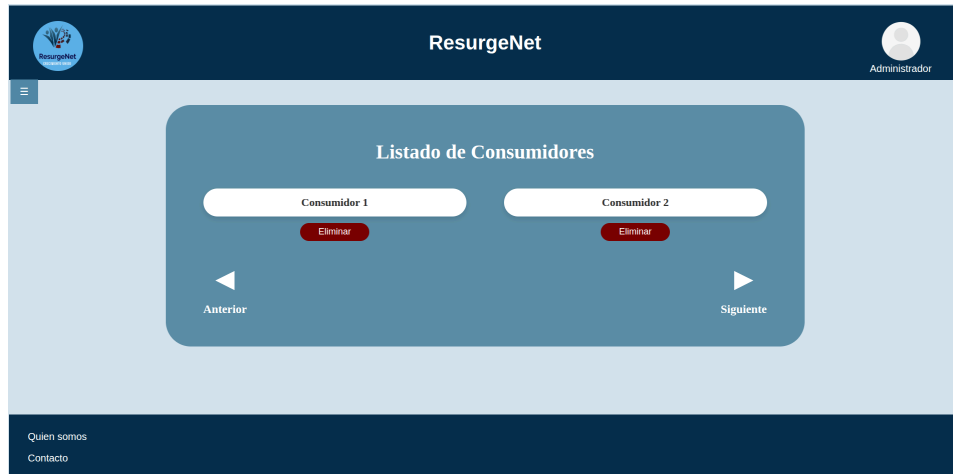


Figura 27: Implementación estática de la HU11

Como para la gestión de comercios accedemos a este panel a través del dashboard principal de administración. El funcionamiento es el mismo que para los comercios, vemos un listado de los consumidores y cada uno tiene asociado su botón para ser eliminado.

El fichero js se encarga de recoger la respuesta del backend y, si esta respuesta no está vacía, extrae la información para mostrarla en el formato que deseamos. Si está vacía nos muestra un mensaje que indica que no hay consumidores registrados.

Parte dinámica - backend HU11

Lo primero que he llevado a cabo ha sido una función, `getConsumer`, para obtener todos los consumidores que tenemos almacenados en la base de datos. Para ello se ha hecho uso de la tabla `usuario` y se han seleccionado los consumidores por el rol que les corresponde, en este caso el 2. Esta función nos devuelve la información en un formato json que es procesada por el script `gestion_consumidores.js`. El resultado es el siguiente:

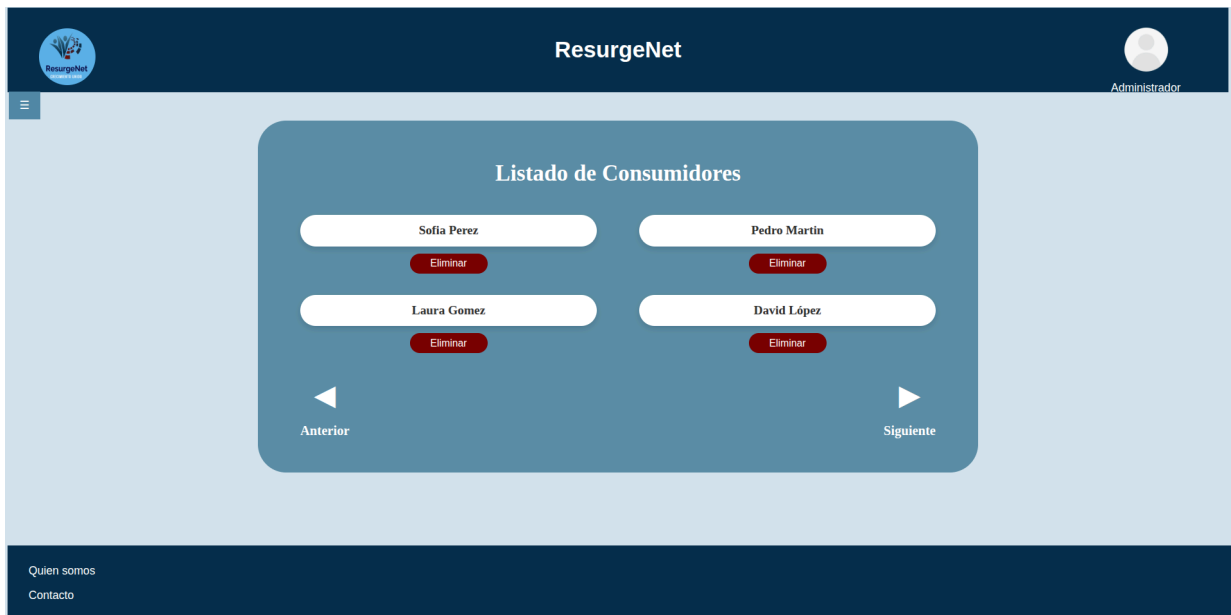


Figura 28: Resultado de la obtención de los consumidores desde la BD

El siguiente paso es la implementación de una función para eliminar los consumidores que se deseen. De esto se encarga `deleteConsumer` que recibe el id del mismo, obtenido con `getConsumer`, y elimina al usuario de la tabla `usuario`. La implementación de la BD se ha llevado a cabo para que si eliminamos en `usuario` esto tenga un efecto en cascada al resto de tablas, por tanto eliminando en `usuario` eliminamos también en `consumidor`. Si llevamos a cabo el borrado de David López cuyo id es el 12 la BD queda de la siguiente manera:

```
mysql> SELECT * FROM usuario;
```

id_usuario	nombre	usuario	password	rol
1	Angela Garrido	angelamgr	25f43b1486ad95a1398e3eeb3d83bc4010015fcc9bedb35b432e00298d5021f7	1
2	David Lopez	david03	3b9b06829b5fb7d3affad0f2b9226c247f4b20d9032ed978363f2e8b6a336367	3
3	Manuel Garrido	manuGr	460a1446ca3e5c2707a62fb405cb643a7bf95dc0752a1505a40f638af4ced341	4
9	Sofia Perez	sofi22	1018bdb757f02e266e25810e9dcb87c861c864bbf08d71d992c4483f9fdcbc82	2
10	Pedro Martin	pedroM	aefae09a1e6d0558cef1e4517643ec0c34f64ce8fe783f51fc2e382d47ccadbe	2
11	Laura Gomez	lauri00	71d84d6e497ba1c8c88258293b64cb1d7e0ec4d3ca87a553285637eefa673939	2
12	David Lopez	davidCa	81de37c5764676d3c09584f7b07769d53f939ae5fcef7c477cbe9d4eebb7849b	2

```
7 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM consumidor;
```

id	direccion	ciudad	cod_postal	n_telefono	email	fecha_nac
9	Gran Vía 5	Madrid	15002	666085950	sofiP_22@gmail.com	1996-05-12
10	Calle Cuevas Almanzora 8	Sevilla	12008	679896335	pedromartin03@gmail.com	1960-09-07
11	Calle Mediterraneo 45	Granada	18184	647963210	lauriG@gmail.com	2000-02-14
12	Calle del Mediterraneo 34	Granada	15078	666203541	davidcala003@gmail.com	2003-09-28

```
4 rows in set (0.00 sec)
```

Figura 29: Estado de la BD antes de eliminar al consumidor

```
mysql> SELECT * FROM usuario;
```

id_usuario	nombre	usuario	password	rol
1	Angela Garrido	angelangr	25f43b1486ad95a1398e3eeb3d83bc4010015fcc9bedb35b432e00298d5021f7	1
2	David Lopez	david03	3b9b06829b5fb7d3affad0f2b9226c247f4b20d9032ed978363f2e8b6a336367	3
3	Manuel Garrido	manuGr	460a1446ca3e5c2707a62fb405cb643a7bf95dc0752a1505a40f638af4ced341	4
9	Sofia Perez	sofi22	1018bdb757f02e266e25810e9dc87c861c864bbf08d71d992c4483f9fdcbc82	2
10	Pedro Martin	pedroM	aefae09a1e6d0558cef1e4517643ec0c34f64ce8fe783f51fc2e382d47ccadbe	2
11	Laura Gomez	lauri00	71d84d6e497ba1c8c88258293b64cb1d7e0ec4d3ca87a553285637eefa673939	2

```
6 rows in set (0.00 sec)
```



```
mysql> SELECT * FROM consumidor;
```

id	direccion	ciudad	cod_postal	n_telefono	email	fecha_nac
9	Gran Vía 5	Madrid	15002	666085950	sofiP_22@gmail.com	1996-05-12
10	Calle Cuevas Almazora 8	Sevilla	12008	679896335	pedromartin03@gmail.com	1960-09-07
11	Calle Mediterraneo 45	Granada	18184	647963210	lauriG@gmail.com	2000-02-14

```
3 rows in set (0.00 sec)
```

```
mysql>
```

Figura 30: Estado de la BD tras eliminar al consumidor

Las pruebas de aceptación de las historias de usuario realizadas se han ido llevando a cabo en cada comprobación del estado de la base de datos al igual que de acceso, todo ello puede verse en las capturas de pantalla proporcionadas para esta iteración.

3.10.3. Revisión por parte de los tutores

Durante la revisión de los bocetos llevados a cabo en esta iteración, los clientes dijeron que el diseño les parecía adecuado y coherente. Sin embargo, tras una reunión al final de esta iteración se solicitó lo siguiente:

- Modificación de los bocetos y la lógica en la gestión de comercios de forma que se diferencie dar de alta a los mismos con poder eliminarlos.
- Incorporación de un listado de accesibilidad que sirva como 'check list' para las distintas funcionalidades incluidas.

En general, la retroalimentación fue positiva y las peticiones indicadas se tendrán en cuenta en las siguientes iteraciones.

3.10.3. Retrospectiva de la iteración

Durante esta sexta iteración se han llevado a cabo casi en su totalidad las historias de usuario planificadas, solo ha quedado por hacer la parte dinámica de las historias 10 y 11 que son las correspondientes a la gestión de los comercios. Al estar en el periodo de exámenes el trabajo se ha visto ralentizado así como las reuniones con los tutores.

Para la siguiente iteración se mantendrá la dinámica y planificación seguida. Atendiendo ahora a la planificación estimada en el periodo de Febrero a Junio, que puede verse en el punto 3.6.2 de este documento. En la siguiente iteración, la séptima, se priorizará la correcta finalización de las historias correspondientes a la gestión de los comercios así como la incorporación del listado de accesibilidad.

3.11. Iteración 7

En esta séptima iteración que abarca desde el 25 de febrero hasta el 11 de marzo se van a llevar a cabo las siguientes historias de usuario:

Finalización de las HU 9 Y 10 - 5 de Marzo

HU12 - Como validador de comercios quiero contar con un formulario a través del cual los comercios nos envíen su solicitud, en formato de ticket digital.

Las tareas relacionadas con esta historia son:

- Diseño de la interfaz - 2 de Marzo
- Diseño de la BD - 2 de Marzo
- Revisión BD - 4 Marzo
- Implementación estática - 5 de Marzo
- Implementación dinámica - 7 de Marzo
- Pruebas de aceptación - 7 de Marzo
 - Comprobación de que se accede correctamente al panel de control del validador
 - Comprobación de que puede desplegarse el menú de sus funciones
 - Comprobación de que accede correctamente a su bandeja de entrada donde aparecerán las solicitudes de los comercios

HU13 - Como validador de comercios quiero contar con un buzón a través del que pueda ponerme en contacto con los comercios para hacerles saber el estado de su solicitud

Las tareas relacionadas con esta historia son:

- Diseño de la interfaz - 7 de Marzo
- Diseño de la BD - 7 de Marzo
- Revisión BD - 9 de Marzo
- Implementación estática - 8 de Marzo
- Implementación dinámica - 11 de Marzo
- Pruebas de aceptación - 11 Marzo
 - Comprobación de acceso correcto a su página de administración
 - Comprobación de que nos aparece el listado de comercios a los que podemos mandar la notificación
 - Comprobación de que los estados de la solicitud son correctos

3.11.1. Diseño de la interfaz y de la BD

Tras llevar a cabo una revisión de la iteración anterior con los tutores se ha decidido llevar a cabo la siguiente modificación en la HU9 y HU10, el alta y baja de los comercios en el sistema: se separarán ambas funcionalidades en vistas diferentes:

- **HU9 (alta)** - estará en un listado aparte como “listado de comercios en espera”, aquí se mostrarán solo los que están en la fase final de validación a la espera de ser dados de alta por el administrador.
- **HU10 (eliminación)** - se encontrará en otro listado, en el que se puede activar o desactivar el comercio en caso de que la eliminación sea algo temporal y también se podrá eliminar de forma permanente.

Por tanto los nuevos bocetos para esta parte del sistema son los siguientes:



Figura 31: Boceto de la nueva versión de la HU9



Figura 32: Boceto de la nueva versión de la HU10

Con respecto a la base de datos, la única modificación que se realizó en la iteración anterior fue la aparición del campo estado en la tabla comercio. Necesitamos cambiar este enumerado para que conste con tres estados no solo los dos que hay actualmente: activo, desactivado temporal y validación.

Con respecto a la HU12, el formulario de contacto a través del que los comercios hacen la solicitud al validador, contamos con el siguiente diseño para la parte estática:

Figura 33: boceto de la HU12, formulario de contacto comercio-validador

Se enlazará a este formulario a través de la landing page (Figura 6) cuando se pulse el botón “Contáctenos” de la sección “Trabaja con nosotros”.

Para gestionar la información del formulario de contacto en nuestra base de datos, se ha diseñado una nueva entidad denominada 'Solicitud-Comercio'. Dado que esta solicitud puede ser enviada por comercios aún no registrados, la entidad almacenará directamente la información de cada campo que compone el formulario. Esta nueva tabla se relaciona directamente con la entidad 'Usuario', estando disponible solo para los que tienen el rol de validador. Para asegurar eso se implementará una restricción cuando pasemos a la fase de desarrollo del nuevo esquema.

Nuestro esquema de BD quedaría de la siguiente manera:

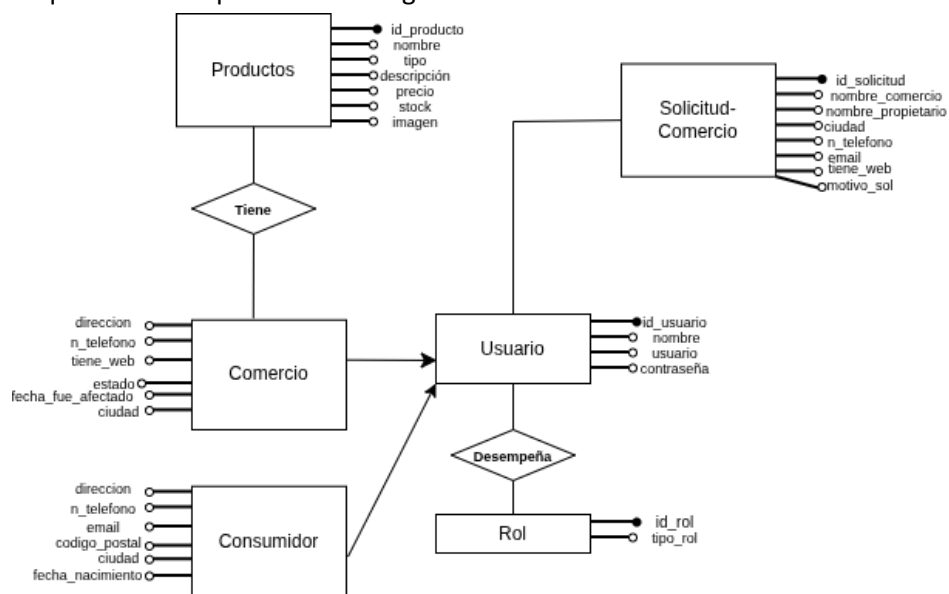


Figura 34: esquema E-R de la base de datos para la HU12

Por último pasamos a hacer los diseños relativos a la HU13, buzón de solicitudes para el validador. El diseño de la interfaz consta de un listado donde vemos el nombre del comercio y el motivo de su solicitud, además tiene dos botones uno para aceptar la solicitud y otro para denegar dicha solicitud. El boceto ha quedado de la siguiente manera:



Figura 35: Boceto para la HU13, gestión de las solicitudes por parte del validador

Con respecto a la base de datos mantenemos el diseño que teníamos para la HU12 que puede verse en la Figura 34.

3.11.2. Implementación estática y dinámica

Parte estática - frontend HU9

En esta parte se han desarrollado los siguientes archivos:

- **HTML:** gestion_comercios_admin_espera.html
- **CSS:** gestion_cons_comer_admin.css
- **JS:** gestion_comercios_espera.js

En estos archivos se ha implementado la estética de los bocetos obteniendo el siguiente resultado:



Figura 36: Implementación estática de la nueva versión de la HU10

Se ha utilizado el mismo fichero que para la realización de la parte estática en la iteración 6, la modificación ha sido el título del apartado dentro del dashboard del administrador. El archivo en JavaScript es donde tenemos la función que maneja la petición AJAX del listado de comercios así como el cambio en el estado del comercio seleccionado.

Parte dinámica - backend HU9

Lo primero que vamos a hacer es modificar el enumerado que se creó en la iteración anterior para la tabla Comercio que nos controla el estado del mismo. La necesidad de esta modificación se explica al inicio del apartado 3.11.1, y es el separar la interfaz de los comercios en espera de los comercios de alta. Se lleva a cabo con el siguiente comando:

```
ALTER TABLE comercio MODIFY COLUMN estado ENUM('validacion','activo', 'desactivado tmp')
DEFAULT 'validacion';
```

Una vez tenemos esto realizado vamos a pasar a realizar una nueva función en nuestro archivo AuthController.php que se encargará de obtener todos los comercios que tienen en el campo estado "validación". La visualización de ese listado queda de la siguiente manera:



Figura 37: Implementación dinámica del listado de comercios en espera

Una vez que obtengamos el listado de los comercios vamos a darle funcionalidad al botón ‘Dar de alta’. Cuando el administrador pulsa dicho botón lo que se realizará es una actualización del campo estado de la base de datos, pasamos de validación a activo y ese comercio desaparecerá de la lista. Esta lógica está implementada en la función `activarComercio` en el archivo `AuthController.php`. El estado de la base de datos tras dar de alta al comercio que tenemos es el siguiente:

```
mysql> SELECT * FROM comercio;
+-----+-----+-----+-----+-----+-----+
| id | ciudad | direccion | n_telefono | tiene_web | estado |
+-----+-----+-----+-----+-----+-----+
| 3 | Granada | Calle Don Quijote 5 | 666899760 | 1 | validacion |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM comercio;
+-----+-----+-----+-----+-----+-----+
| id | ciudad | direccion | n_telefono | tiene_web | estado |
+-----+-----+-----+-----+-----+-----+
| 3 | Granada | Calle Don Quijote 5 | 666899760 | 1 | activo |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
```

Figura 38: Estado de la base de datos tras dar de alta al comercio

Al realizar estas historias de usuario, se ha detectado la necesidad de contar con un nuevo campo en la tabla `comercio`, el del nombre del comercio, ya que este será el que debería aparecer en el listado, no el nombre del usuario que es propietario. Esto se ha realizado a través del siguiente comando:

```
ALTER TABLE comercio ADD nombreComercio VARCHAR(100);
```

Incorporando este cambio la base de datos queda de la siguiente manera:

```
mysql> SELECT * FROM comercio;
+-----+-----+-----+-----+-----+-----+-----+
| id | ciudad | direccion | n_telefono | tiene_web | estado | nombreComercio |
+-----+-----+-----+-----+-----+-----+-----+
| 3 | Granada | Calle Don Quijote 5 | 666899760 | 1 | validacion | NULL |
+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> UPDATE comercio SET nombreComercio = 'cultivoSaludable' WHERE id=3;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> SELECT * FROM comercio;
+-----+-----+-----+-----+-----+-----+-----+
| id | ciudad | direccion | n_telefono | tiene_web | estado | nombreComercio |
+-----+-----+-----+-----+-----+-----+-----+
| 3 | Granada | Calle Don Quijote 5 | 666899760 | 1 | validacion | cultivoSaludable |
+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
```

Figura 39: Estado de la base de datos tras la incorporación del nuevo campo

Y nuestra visualización en la interfaz es la siguiente:



Figura 40: Visualización del nombre asociado al comercio en nuestra interfaz

Parte estática - frontend HU10

En esta parte se han desarrollado los siguientes archivos:

- HTML: gestion_comercios_alta_admin.html
- CSS: gestion_comercios_alta_admin.css
- JS: gestion_comercios_activos.js

Estos archivos se han implementado siguiendo la estética de los bocetos, obteniendo el siguiente resultado:



Figura 41: Resultado de la implementación estática de la HU10

Como podemos ver el listado cuenta con una serie de botones a través de los que se activa, desactiva o elimina el comercio. Se ha implementado mediante iconos “estándar” para facilitar el uso de la plataforma a cualquier usuario.

Parte dinámica - backend HU10

Para comenzar con la parte estática se ha desarrollado la función `getComerciosActivos` en el archivo `AuthController.php`. El funcionamiento es el mismo que para obtener los comercios en estado 'validacion, simplemente filtrar por estado 'activo' para obtener los recursos.

Tras esto se ha implementado la conexión entre el back y el front end con el archivo `gestion_comercios_activos.js`. La visualización del listado queda de la siguiente manera:



Figura 42: Implementación dinámica del listado de comercios activos

Una vez obtenido el listado vamos a darle funcionalidad a los botones. Tenemos el siguiente flujo:

- Si el comercio está 'activo', el botón asociado a la funcionalidad de 'Activar' quedará opaca indicando que no se puede utilizar. Solo estará disponible 'Desactivar' y 'Eliminar'
- Si el comercio está en estado 'desactivado tmp' el botón asociado a la funcionalidad de 'Desactivar' quedará opaca indicando que no se puede utilizar. Solo estará disponible 'Activar' y 'Eliminar'

Hemos desarrollado dos funciones en `AuthController.php` para cambiar de 'activo' a 'desactivado tmp' (`estadoDesactivarComercio`) y otra con la funcionalidad inversa (`estadoActivoComercio`). Estas funciones se encargan de cambiar el valor del atributo 'estado' de nuestra base de datos. Se puede ver cómo se ha implementado esto en las siguientes imágenes:

```
Database changed
mysql> SELECT * FROM comercio;
+----+-----+-----+-----+-----+-----+-----+
| id | ciudad | direccion | n_telefono | tiene_web | estado | nombreComercio |
+----+-----+-----+-----+-----+-----+-----+
| 3  | Granada | Calle Don Quijote 5 | 666899760 | 1 | validacion | cultivoSaludable |
| 13 | Valencia | Calle Mar 13 | 678904530 | 0 | activo | Javier&Hijos |
+----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>
```

Figura 43: Estado inicial de la BD antes de modificar el comercio que tenemos activo



Figura 44: Visualización de la implementación dinámica cuando el comercio está 'activo'

Vamos a modificar el comercio y vamos a desactivarlo temporalmente, comprobando así cómo queda la base de datos y cómo se visualiza en nuestro front:

```
mysql> SELECT * FROM comercio;
+-----+-----+-----+-----+-----+-----+-----+
| id | ciudad | direccion | n_telefono | tiene_web | estado | nombreComercio |
+-----+-----+-----+-----+-----+-----+-----+
| 3 | Granada | Calle Don Quijote 5 | 666899760 | 1 | validacion | cultivoSaludable |
| 13 | Valencia | Calle Mar 13 | 678904530 | 0 | desactivado tmp | Javier&Hijos |
+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>
```

Figura 45: Nuevo estado de la BD tras desactivar el comercio temporalmente



Figura 46: Visualización de la desactivación en el frontend

Por último, hemos implementado la funcionalidad para el botón de eliminar el comercio, haremos una nueva función `deleteComercio` que recibe el id del comercio y lo elimina de

nuestra base de datos. Lo aplicamos, como ejemplo, sobre el comercio que tenemos actualmente, Javier&Hijos, y obtenemos el siguiente estado en la base de datos:



Figura 47: Flujo de visualización en el front

```
mysql> SELECT * FROM comercio;
+----+-----+-----+-----+-----+-----+-----+
| id | ciudad | direccion | n_telefono | tiene_web | estado | nombreComercio |
+----+-----+-----+-----+-----+-----+-----+
| 3 | Granada | Calle Don Quijote 5 | 666899760 | 1 | validacion | cultivoSaludable |
+----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
```

Figura 48: Estado de la base de datos tras eliminar el comercio

Parte estática - frontend HU12

En esta parte se han implementado los siguientes archivos:

- HTML: formulario_contacto_comercios.html
- CSS: formulario_contacto_comercios.css
- JS: formulario_contecto_comercio.js

Estos archivos se han implementado siguiendo la estética de los bocetos, obteniendo el siguiente resultado:

The image shows a web interface for 'ResurgeNet'. At the top, there is a dark blue header with the 'ResurgeNet' logo on the left and an 'Inicio Sesión' button on the right. Below the header is a light blue section containing a white rounded rectangle with the title 'Trabaja con nosotros'. Inside this rectangle is a contact form with the following fields: 'Nombre personal' (text input), 'Nombre comercio' (text input), 'Email' (text input), 'Ciudad' (text input), 'Nº Teléfono' (text input), 'Motivo de solicitud' (text area), and '¿Web operativa?' (dropdown menu with 'Selecciona una opción'). A red button labeled 'Enviar formulario' is positioned below the form fields. At the bottom of the page, a dark blue footer contains the links 'Quien somos' and 'Contacto'.

Figura 49: Implementación estática de la HU13, formulario de contacto comercio-validador

Como se comentó anteriormente a este formulario se accede desde la landing page en el área de 'Trabaja con nosotros' al pulsar el botón 'Contáctenos'. (Figura 6)

El archivo `formulario_contacto_comercios.js` cumple dos funciones principales: por un lado, realiza las validaciones del formulario, de manera similar a otros formularios como el de registro; y por otro, establece la comunicación entre el front-end y el back-end.

Parte dinámica - backend HU11

Como primer paso en la implementación de esta historia de usuario sería añadir la nueva tabla 'solicitudComercio' a nuestra base de datos. Se ha llevado a cabo con el siguiente comando:

```
CREATE TABLE solicitudComercio(id_solicitud INT AUTO_INCREMENT PRIMARY KEY,
nombrePropietario VARCHAR(50) NOT NULL, nombreComercio VARCHAR(50) NOT NULL,
email VARCHAR(40) NOT NULL, ciudad VARCHAR(40) NOT NULL, n_telefono
VARCHAR(10) NOT NULL,motivoSolicitud VARCHAR(300) NOT NULL, tiene_web
BOOLEAN NOT NULL);
```

```
mysql> CREATE TABLE solicitudComercio(id_solicitud INT AUTO_INCREMENT PRIMARY KEY,
Y, nombrePropietario VARCHAR(50) NOT NULL, nombreComercio VARCHAR(50) NOT NULL,
email VARCHAR(40) NOT NULL, ciudad VARCHAR(40) NOT NULL, n_telefono VARCHAR(10)
NOT NULL, motivoSolicitud VARCHAR(300) NOT NULL, tiene_web BOOLEAN NOT NULL);
Query OK, 0 rows affected (0.03 sec)

mysql> SELECT * FROM solicitudComercio;
Empty set (0.01 sec)

mysql> SHOW TABLES;
+-----+
| Tables_in_ResurgeNet |
+-----+
| comercio              |
| consumidor            |
| productos              |
| rol                    |
| solicitudComercio      |
| usuario                |
+-----+
6 rows in set (0.00 sec)
```

Figura 50: Estado de la BD tras incorporar la nueva tabla

Una vez tenemos incluida esta tabla en la base de datos vamos a realizar una función, solicitudComercio, en nuestro archivo AuthController para guardar correctamente en nuestra nueva tabla la información propia que el comercio escriba en el formulario.

El flujo de ejecución sería el siguiente:

The image shows a web form titled 'Trabaja con nosotros' on the ResurgeNet website. The form fields are: Nombre personal (Miguel Heredia), Nombre comercio (RecambiosTodoCoche), Email (miguelH@gmail.com), Ciudad (Valencia), N° Teléfono (666089550), Motivo de solicitud (Tras las inundaciones sufridas en la DANA mi comercio se vio afectado pero he conseguido salvar piezas que me gustaría vender), and ¿Web operativa? (No). A red 'Enviar formulario' button is at the bottom.

Below the form, a terminal window shows the SQL query results for the 'solicitudComercio' table:

```
mysql> SELECT * FROM solicitudComercio;
+-----+-----+-----+-----+-----+-----+-----+
| id_solicitud | nombrePropietario | nombreComercio | email | ciudad | n_telefono | motivoSolicitud |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | Miguel Heredia | RecambiosTodoCoche | miguelH@gmail.com | Valencia | 666089550 | Tras las inundaciones sufridas en la DANA mi comercio se vio afectado p |
| ero he conseguido salvar piezas que me gustara vender | 0 |
+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Figura 51: Flujo de ejecución de la HU12, solicitud de los comercios

Parte estática - frontend HU13

En esta parte se han desarrollado los siguientes archivos:

- HTML: dashboard_validador.html
- CSS: dashboard_validador.css

La implementación del boceto que podemos ver en la Figura 35 ha quedado de la siguiente manera:

The image shows a dashboard titled 'Listado de Comercios esperando validacion'. It displays a table with columns: Comercio 1, Motivo solicitud, and buttons for 'Aceptar' and 'Denegar'. The first row shows 'Comercio 1' with the reason 'Tras las afectaciones sufridas por el temporal...' and a green checkmark for 'Aceptar' and a red 'X' for 'Denegar'. Navigation buttons 'Anterior' and 'Siguiente' are at the bottom.

Figura 52: Implementación estática del dashboard del validador de comercios

Las pruebas de aceptación de las historias de usuario realizadas se han ido llevando a cabo en cada comprobación del estado de la base de datos, al igual que se ha comprobado el correcto acceso por los usuarios correspondientes a las diferentes interfaces creadas. Todo ello puede verse en las capturas de pantalla proporcionadas para esta iteración.

3.11.3. Accesibilidad

Tras una reunión con los tutores antes del inicio de esta iteración, se acordó que la accesibilidad de la web se medirá como una prueba de aceptación adicional para cada historia de usuario. Esta prueba consistirá en la validación de una serie de puntos basados en buenas prácticas y estándares de accesibilidad web. Los estándares utilizados se recogerán de la WCAG 2.1, un conjunto de normas internacionales desarrolladas por el W3C para garantizar que sitios web, aplicaciones y contenido digital sean accesibles para personas con todo tipo de capacidades. Vamos a basarnos en los siguientes puntos para comprobar la accesibilidad de la web:

- Contar con **alternativas textuales**, es decir, proporcionar un texto alternativo para imágenes sobre todo si son informativas o realizan una acción
- **Evitar elementos distractores** como carruseles en movimiento automático o parpadeos que causan distracciones
- Debemos ofrecer un **buen contraste** en los elementos de la página para facilitar su visualización
- La estructura de nuestra web tiene que **mantener un orden lógico coherente** para que se puedan usar correctamente los lectores de pantalla
- Debemos tener **ayudas de navegación** como menús visibles.
- Los **enlaces y los botones** deben tener nombres internos que sean visibles y los identifiquen sin problema
- El **diseño de las pantallas debe ser simple**, evitando ventanas emergentes o un uso excesivo del scroll

Para evaluar su cumplimiento, además de llevarlo a cabo manualmente, se utilizará la herramienta WAVE, que identifica automáticamente problemas comunes de accesibilidad (como imágenes sin texto alternativo, contraste insuficiente, etiquetas de formulario ausentes o enlaces poco descriptivos) mostrando los errores visualmente sobre la página y generando un informe que sirve como evidencia del cumplimiento de los criterios WCAG 2.1 nivel AA. Un ejemplo de cómo funciona esta herramienta es el siguiente:



Figura 54: Funcionamiento de la herramienta WAVE

A continuación vemos lo que ocurre con cada historia de usuario realizada:

HU	Salida WAVE	Solución
HU 1 - landing page	1 error - el html no tiene definido el idioma 1 warning - jerarquía de encabezados no consecutiva	Definir el idioma del html a español. Modificar la jerarquía de encabezados
HU 2 - inicio sesión administrador HU 4 - inicio de sesión del validador HU 5 - inicio de sesión del comercio HU 7 - inicio de sesión del consumidor	3 errores - el html no tiene definido el idioma y faltan los labels del formulario de inicio de sesión	Definir el idioma del html a español. Añadir el label al los campos del formulario.
HU 3 - cierre de sesión del administrador	1 error - Definir el idioma del html a español. 4 errores de contraste - en el menú desplegable no hay suficiente contraste entre el fondo y los títulos de las funcionalidades. Tampoco hay suficiente contraste con el botón del menú desplegable y el fondo	Definir el idioma del html a español. Cambiar el color de las funcionalidades de blanco a negro. Cambiar el color del menú desplegable a al mismo que el encabezado
HU 6 - registro de consumidores	1 error - el html no tiene definido el idioma	Definir el idioma del html a español.
HU 8 - subida de productos por parte del comercio	1 error - el html no tiene definido el idioma	Definir el idioma del html a español.
HU 9 - gestión de comercios en espera	1 error - el html no tiene definido el idioma	Definir el idioma del html a español.

HU 10 - gestión de comercios dados de alta	7 errores de contraste - en la cabecera de la tabla y en los botones inferiores de “Anterior” y “Siguiente”	Cambiar el texto a negro para esos títulos.
HU 12 - formulario de contacto comercio-validador	7 errores de etiquetado - tenía las etiquetas de cada campo creadas pero no con un for asociado	Añadir el for dentro de la etiqueta para cada campo del formulario

Tras esta primera validación automática realizada con la herramienta hemos profundizado la misma con una comprobación manual simulando diferentes formas de interacción con la web.

Comprobación	Resultado	Cambios realizados
Navegación mediante el teclado	Todos los elementos interactivos son accesibles mediante la tecla TAB	No han sido necesarios
Texto alternativo	Todas las imágenes informativas incluyen el atributo alt con el texto alternativo	No han sido necesarios
Estructura semántica	Se usan correctamente los encabezados y las etiquetas estructurales	No han sido necesarios
Diseño simple	La interfaz no cuenta con elementos que distraigan al usuario	No han sido necesarios

3.11.4. Revisión por parte de los tutores

Durante la revisión de los bocetos llevados a cabo en esta iteración, los clientes dijeron que el diseño les parecía adecuado y coherente. En general, la retroalimentación fue positiva y útil para saber cómo seguir enfocando el trabajo en las siguientes iteraciones.

3.11.5. Retrospectiva de la iteración

Durante esta séptima iteración se han llevado a cabo casi en su totalidad las historias de usuario planificadas, además de la finalización de las historias 10 y 11 que quedaron pendientes en la sexta iteración. Ha quedado por realizar la parte dinámica de la última historia de usuario, el dashboard del validador de comercios, correspondiente con la HU13. Ya que modificar la lógica de la gestión de comercios por parte del administrador ha llevado más tiempo que el planteado inicialmente, hemos tenido que reestructurar tanto a nivel de diseño como de implementación.

Para la siguiente iteración se mantendrá la dinámica y planificación seguida. Siguiendo con la planificación estimada para este periodo. En la siguiente iteración, la octava, se priorizará la correcta finalización de la historia correspondiente al dashboard del administrador. Es posible mantener la planificación ya que solo tendremos que implementar desde cero una única HU, la 14, y que lo que queda de la HU13 es la parte dinámica.

3.12. Iteración 8

En esta octava iteración que abarca desde el 16 de Marzo hasta el 4 de Abril se van a abordar las siguientes historias de usuario:

Finalización de la HU13 - 20 de marzo

HU14 - Como comercio quiero tener acceso a la web de mi tienda de forma que pueda actualizar sus productos

Las tareas relacionadas con esta historia son:

- Diseño de la interfaz - 22 de Marzo
- Diseño de la BD - 23 de Marzo
- Revisión BD - 25 Marzo
- Implementación estática - 28 de Marzo
- Implementación dinámica - 30 de Marzo
- Pruebas de aceptación - 3 de Abril
 - Comprobación de que se accede correctamente al listado de productos a través del dashboard del comercio
 - Comprobación de que los productos listados son los correspondientes al comercio que inicia la sesión
 - Comprobación de que se puede acceder al formulario donde se actualizan los productos
 - Comprobación de que se actualiza la información correctamente en la base de datos

Originalmente estaba pensado realizar las historias 14 (listado de los productos para el consumidor) y la 15 (plataforma de pago para el consumidor), sin embargo para seguir un orden lógico adecuado se ha decidido realizar estas historias y dejar la HU15 para la siguiente iteración.

3.12.1. Diseño de la interfaz y de la BD

Para esta iteración se diseñará la interfaz correspondiente para la HU14, la actualización de los productos por parte del comercio. Ya contamos con el dashboard del comercio, realizado en la HU8 (Figura 16), por lo que vamos a habilitar un nuevo espacio donde tengamos una lista de los productos y pulsando en cada uno de ellos contemos con un formulario donde modificar cada campo del mismo. Por tanto se elaborarán dos bocetos: uno relativo al listado de productos y otro al formulario de actualización.

Los bocetos quedarían de la siguiente manera:



Figura 55: Boceto del listado de productos

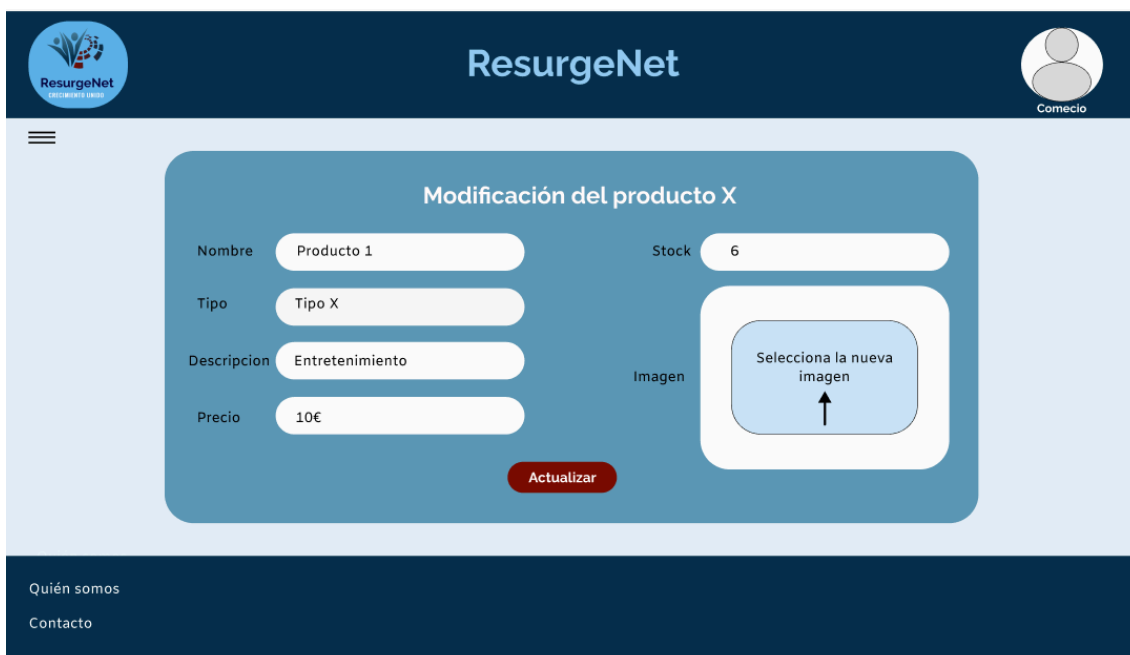


Figura 56: Boceto del formulario de actualización de cada producto

En el formulario estará la información actual del producto seleccionado cargada el comercio, de tal forma que solo se tiene que modificar aquel campo que sea necesario y al pulsar el botón actualizar se cambiará la información en la BD.

En cuanto a la BD no habría que realizar ninguna modificación del esquema entidad relación ya que actualmente contamos con la tabla producto y la tabla comercio, que son las dos implicadas en esta historia de usuario. Se puede ver el último esquema de la base de datos en la Figura 34.

3.12.2. Implementación estática y dinámica

Parte dinámica - HU13

La implementación de esta historia de usuario, relativa a la validación de las peticiones de los comercios, implica lo siguiente cuando el validador de comercios acepta la petición:

- El estado de la solicitud pasa a 'aceptada'
- Se lleva a cabo la creación de un usuario con el rol 'comercio'
- Se creará un nuevo comercio usando los datos que proporciona en el formulario con el estado 'validado'
- El administrador será quién da de alta a ese nuevo comercio y le proporcionará unas credenciales temporales a través del email que dejó en el formulario. Esas credenciales serán un usuario y contraseña iniciales que posteriormente, cuando acceda al sistema podrá modificar o mantener, a preferencia del comercio. Con esto se pretende que el comercio tenga acceso a su dashboard desde el momento en que es aceptado.

Si la solicitud es denegada, se enviará un mensaje estándar vía email al comercio que realizó la solicitud indicando que fue denegada.

Por tanto, esta historia nos implica incorporar la lógica de respuesta del administrador al comercio que ha sido dado de alta, aparte de todo lo relativo al validador. Vamos a comenzar por hacer la parte del validador de comercios.

Antes de comenzar con el desarrollo se ha visto que necesitamos incorporar un nuevo atributo en nuestra tabla solicitudComercio, el estado de la solicitud, que será un enumerado que cuenta con cuatro posibles estados: pendiente (será el estado por defecto), aceptada, denegada o alta por administrador. Se ha llevado a cabo su incorporación en la base de datos alterando la tabla correspondiente.

Tenemos que realizar el volcado de las solicitudes pendientes de la BD al dashboard, esto se ha desarrollado elaborando una función, getSolicitudesComercio, en el archivo AuthController. Esta función se encarga de devolvernos un json con el id de la solicitud, el nombre del comercio y el motivo de la misma. Mediante un archivo js, llamado solicitudesComercios.js conectamos el front con el backend consiguiendo mostrar las solicitudes que tenemos actualmente en la base de datos.

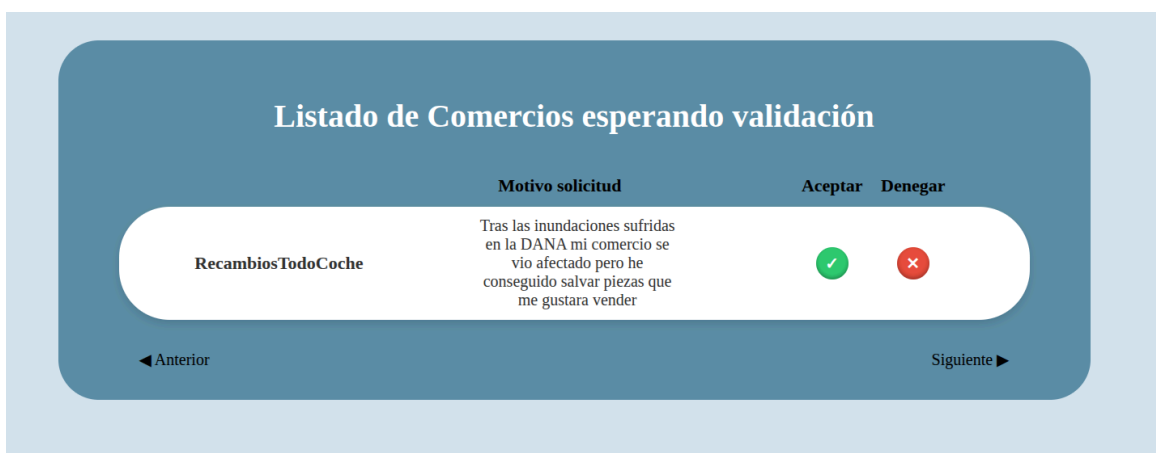


Figura 57: Volcado del listado de solicitudes en el dashboard del validador

Actualmente contamos con una única solicitud, como puede verse por el recuento de filas que tiene nuestra base de datos:

```
mysql> SELECT COUNT(*) AS total_filas
-> FROM solicitudComercio;
+-----+
| total_filas |
+-----+
|          1 |
+-----+
1 row in set (0.02 sec)

mysql>
```

Figura 58: Recuento de filas de la base de datos solicitudComercio

Ahora pasamos a darle funcionalidad a los botones que nos permiten aceptar o denegar la solicitud. Vamos a comenzar por la denegación de dicha solicitud. Una vez que se pulse el botón correspondiente, el estado pasará a ser 'denegado' en la base de datos por lo que dejará de verse en el listado y se enviará el email al comercio. Esto se recoge en la función `denegarSolicitudComercio` que se encuentra en el archivo `AuthController`.

Inicialmente se ha intentado hacer el envío efectivo de mensajes usando un proveedor de correo comercial con dominio de Gmail. Sin embargo, éste bloquea el acceso si no se activa la verificación en dos pasos y se usa la contraseña de aplicación. Para no depender del SMTP (que es el protocolo que usan los servidores de correo para enviar emails) hemos optado por la opción de registro en log durante el desarrollo.

Hemos modificado el archivo `.env` que nos proporciona Laravel dando valor al campo `MAIL_MAILER=log`, lo que indica que no se realizan envíos efectivos a destinatarios externos a través de la red, sino que todos los mensajes se registran de forma local en el archivo `laravel.log`. Esto nos evita errores de autenticación y hace que podamos comprobar que nuestra función `denegarSolicitudComercio` funciona correctamente.

Como trabajo futuro podría incorporarse un servicio de envío de correo que ya nos proporciona un SMTP listo para usar. Para ello solo tenemos que modificar el archivo `.env` y cambiar el `MAIL_MAILER` a SMTP.

Si denegamos la solicitud del único comercio que tenemos listado vemos que desaparece el mismo, dejando el listado vacío. Observamos también que cambia el estado de la BD y que el comercio (en este caso nosotros mismos) recibe el email correspondiente. Podemos ver esto en las siguientes capturas de pantalla:



```
mysql> SELECT estado FROM solicitudComercio WHERE id_solicitud=1;
+-----+
| estado |
+-----+
| denegada |
+-----+
1 row in set (0.00 sec)

mysql>
```

Figura 59: Estado de la interfaz y de la BD tras denegar la solicitud

```
[2026-03-24 11:25:39] local.INFO: Password recibida: validador1
[2026-03-24 11:25:39] local.INFO: Hash generado: e52473318e65c0bd9ecc1c94141e9b34688906b7776f2f72824dbb0c9e366cd8
[2026-03-24 11:25:39] local.INFO: Hash en BD: e52473318e65c0bd9ecc1c94141e9b34688906b7776f2f72824dbb0c9e366cd8
[2026-03-24 11:36:39] local.DEBUG: From: ResurgeNet <infoResurgeNet@gmail.com>
To: angelamgr@correo.ugr.es
Subject: Solicitud denegada
MIME-Version: 1.0
Date: Tue, 24 Mar 2026 11:36:39 +0000
Message-ID: <f70d2d84e3e87d69459736688da9a0fe@gmail.com>
Content-Type: text/plain; charset=utf-8
Content-Transfer-Encoding: quoted-printable

Hola RecambiosTodoCoche,

Lamentablemente su solicitud ha sido denegada.

Saludos,
ResurgeNet
```

Figura 60: Email recibido por el comercio visualizado en los logs de Laravel

Por último, pasamos a implementar la funcionalidad de la aceptación de la solicitud. Por parte del validador de comercio tenemos que incorporar una función, `aceptarSolicitudComercio`, que realice dos cosas:

- Cambie el estado de la solicitud a 'aceptada'
- Cree un nuevo usuario con rol de comercio, con usuario y contraseña como el nombre del comercio que se encuentra almacenado en la tabla `solicitudComercio`.

Como se puede ver en la siguiente imagen, cuando una solicitud es aceptada se cambia el estado de la misma en nuestra tabla `solicitudComercio` y además podemos ver que se ha creado un nuevo usuario con el rol 3 que es el correspondiente a los comercios:

```
mysql> SELECT estado FROM solicitudComercio WHERE id_solicitud=1;
+-----+
| estado |
+-----+
| aceptada |
+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM usuario WHERE rol=3;
+-----+-----+-----+-----+-----+
| id_usuario | nombre | usuario | password | rol |
+-----+-----+-----+-----+-----+
| 2 | David Lopez | david03 | e52473318e65c0bd9ecc1c94141e9b34688906b7776f2f72824dbb0c9e366cd8 | 3 |
| 14 | Miguel Heredia | RecambiosTodoCoche | 112b31df65d811e50e5771525a5f4f81fd1f1ba9fbbe8e6d28209d18ae2b8c33 | 3 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>
```

Figura 61: Estado de la base de datos tras la aceptación de la solicitud por parte del validador

Podemos ver cómo el estado de la solicitud del comercio RecambiosTodoCoche pasa a estar ‘aceptada’ y cómo contamos ahora con ese nuevo usuario con el rol de comercio en la tabla correspondiente.

En la parte del administrador tenemos que modificar la función para dar de alta a los comercios, cuando se dé de alta el flujo será:

- Creación de un comercio volcando los datos de la tabla solicitudComercio, esto se implementa en la función activarComercio que se creó para realizar la HU9.
- Notificación de sus credenciales al comercio para que acceda a su dashboard

```
mysql> SELECT * FROM comercio;
+-----+-----+-----+-----+-----+-----+-----+
| id | ciudad | direccion | n_telefono | tiene_web | estado | nombreComercio |
+-----+-----+-----+-----+-----+-----+-----+
| 3 | Granada | Calle Don Quijote 5 | 666899760 | 1 | activo | cultivoSaludable |
| 14 | Valencia | Valencia | 666089550 | 0 | activo | RecambiosTodoCoche |
+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT estado FROM solicitudComercio WHERE nombreComercio='RecambiosTodoCoche';
+-----+
| estado |
+-----+
| alta admin |
+-----+
1 row in set (0.00 sec)

mysql>
```

Figura 62: Estado de la base de datos tras la modificación del alta de comercios

Como podemos observar en la imagen anterior, cuando el administrador da de alta al comercio, el estado de la solicitud cambia a ‘alta admin’ y se crea dicho comercio en la tabla correspondiente.

```

[2026-03-26 15:27:59] local.DEBUG: From: ResurgeNet <infoResurgeNet@gmail.com>
To: laura.martin@gmail.com
Subject: Solicitud aceptada
MIME-Version: 1.0
Date: Thu, 26 Mar 2026 15:27:59 +0000
Message-ID: <ae765f2a0e874483684b4b98c146ed0@gmail.com>
Content-Type: text/plain; charset=utf-8
Content-Transfer-Encoding: quoted-printable

Hola EntreLineas,
Su solicitud ha sido aceptada y se ha dado de alta en nuestra plataforma.
Sus credenciales de acceso son:
Usuario: EntreLineas
Contraseña: EntreLineas

Saludos,
ResurgeNet

```

Figura 63: Email visualizado en el log de Laravel

Las pruebas de aceptación correspondientes a esta historia de usuario se han ido llevando a cabo en cada comprobación del estado de la base de datos, al igual que se ha comprobado el correcto acceso del validador a su dashboard. Todo ello puede verse en las capturas de pantalla proporcionadas para esta iteración.

Parte estática - HU14

En esta parte se han desarrollado los siguientes archivos para el listado de productos:

- HTML: listado_productos.html
- JS: listado_productos.js

La implementación del boceto que podemos ver en la Figura 55 ha quedado de la siguiente manera:



Figura 64: Implementación estática del listado de productos

A través de este listado podemos acceder a la modificación específica de cada producto. A través del fichero en js conectamos el back con el frontend haciendo posible el volcado de la base de datos al listado que ve el comercio.

Para la edición de los productos hemos aprovechado el formato que teníamos del alta de los productos, usando el mismo modelo de frontend pero con alguna modificación en los títulos, quedando de la siguiente manera:

The screenshot shows a web application interface for updating a product. The header is dark blue with the 'ResurgeNet' logo on the left and a user profile 'N. Comercio' on the right. The main content area is light blue and titled 'Actualiza tu producto'. It contains a form with the following fields: 'Nombre' (Alas de Onix), 'Tipo' (Libro Fantasia), 'Descripción' (Tercera entrega de la saga Empirio), 'Precio' (23.90), 'Stock' (5), and 'Imagen' (Seleccionar archivo). A red 'Guardar' button is at the bottom of the form. The footer is dark blue with links for 'Quien somos' and 'Contacto'.

Figura 65: Implementación estática del formulario de actualización de productos

Hemos creado un nuevo archivo js, `preinfo_producto.js`, que traiga del back al frontend la información de cada producto y esté previamente rellena dicha información. También se ha implementado el archivo `actualizar_prod.js`

Parte dinámica - HU14

Primero se ha implementado una nueva función en nuestro archivo `AuthController`, `getProductosComercio(id_comercio)`, que se encarga de extraer el nombre de los productos asociados al comercio que identificamos con el id y devolverlos en un formato json. Accediendo como el comercio que tiene el id 3 podemos ver que su listado corresponde con lo que tenemos en la base de datos. Dicho listado se ve en la Figura 64. Si accedemos a la tabla producto de nuestra base de datos podemos ver que el listado se corresponde a lo que nos muestra el front:

```
mysql> SELECT nombre FROM productos WHERE id_comercio = 3;
+-----+
| nombre |
+-----+
| Alas de Onix |
| La Asistente |
+-----+
2 rows in set (0.00 sec)

mysql>
```

Figura 66: Datos de la base de datos para el comercio 3

Por último, se ha implementado la funcionalidad de precargar la información que ya tenemos guardada de cada producto para su posterior actualización. Se han implementado dos funciones:

- getInfoProducto, se encarga de recoger toda la información del producto y devolverla en un json para que se pueda cargar con el archivo js previamente creado.
- actualizarInfoProducto, se encarga de recoger la nueva información del formulario y actualizar la base de datos

Una vez que se actualiza el producto volvemos al listado. Hemos cambiado el precio del artículo 'Alas de Onix' de 23 a 30 euros. Podemos ver que se realiza de forma correcta consultando el estado de la base de datos:

```
mysql> SELECT precio FROM productos WHERE nombre = 'Alas de Onix';
+-----+
| precio |
+-----+
| 23.00 |
+-----+
1 row in set (0.01 sec)

mysql> SELECT precio FROM productos WHERE nombre = 'Alas de Onix';
+-----+
| precio |
+-----+
| 30.00 |
+-----+
1 row in set (0.00 sec)

mysql>
```

Figura 77: Datos de la base de datos tras modificar un producto

Se han realizado las pruebas de aceptación de esta historia de usuario durante cada verificación del estado de la base de datos, así como la comprobación del acceso correcto del validador a su dashboard. Todo esto se refleja en las capturas de pantalla incluidas en esta iteración.

Comprobación de la accesibilidad de las HU desarrolladas

Lo primero que hemos hecho ha sido pasarle la herramienta WAVE a cada una de las implementaciones realizadas obteniendo los siguientes resultados:

HU	Salida WAVE	Solución
HU 13	0 errores	No hay que modificar nada
HU 14 - listado	0 errores	No hay que modificar nada
HU 14 - formulario actualización	1 error - no está bien asociada la etiqueta al campo del formulario para el nombre del producto	Poner el mismo nombre para el campo for del label que para el id del input

Para las comprobaciones manuales se ha navegado por la página con el TAB y se ha comprobado su correcto funcionamiento. El resto de las comprobaciones (texto alternativo, estructura semántica y diseño simple) se ha comprobado mediante se realizaba la implementación, por lo que su estado actual es el adecuado.

3.12.4. Revisión por parte de los tutores

Durante la revisión de los bocetos llevados a cabo en esta iteración, los clientes dijeron que el diseño les parecía adecuado y coherente. En general, la retroalimentación fue positiva y útil para saber cómo seguir enfocando el trabajo en las siguientes iteraciones.

3.12.5. Retrospectiva de la iteración

Durante el desarrollo de esta iteración se han completado en su totalidad las HU planificadas. Además, se han adquirido conocimientos de la gestión de emails usando Laravel y se ha pensado en cómo agilizar este proceso cuando este proyecto pase de local a una solución real y utilizable.

Por tanto, la planificación temporal ha sido la adecuada y se mantendrá para la siguiente iteración.

3.13. Iteración 9

En esta novena iteración que abarca desde el 8 de Abril hasta el 22 de Abril se van a abordar las siguientes historias de usuario:

HU15 - Como validador de comercios quiero un cierre de sesión seguro para que los datos no sean de fácil acceso

Las tareas relacionadas con esta historia son:

- Diseño de la interfaz - 8 de Abril
- Diseño de la BD - 8 Abril de Marzo
- Revisión BD - 9 Abril
- Implementación estática - 10 Abril
- Implementación dinámica - 11 Abril
- Pruebas de aceptación - 11 de Abril
 - Cierre de sesión exitoso, el usuario es redirigido a la landing page y el token de la sesión se invalida
 - Si el usuario quiere ir hacia atrás en la web no puede ver el contenido, es decir, que si un usuario pulsa el botón de retroceso en el navegador web (la flecha de ir a la página anterior) no podrá ver el espacio del validador de comercios. Esto es una medida de seguridad para garantizar la privacidad de los datos.
 - Comprobación de la accesibilidad

HU16 - Como comercio quiero un cierre de sesión seguro para que los datos no sean de fácil acceso

Las tareas relacionadas son las mismas que para la HU15, compartiendo igualmente las pruebas de aceptación. Esta tarea se realizará entre los días 12 y 15 de Abril.

HU17 - Como consumidor quiero un cierre de sesión seguro para que los datos no sean de fácil acceso

Para esta historia sucede lo mismo que para la anterior, se realizará entre los días 16 y 18 de Abril.

Mejoras en la visualización de las notificaciones de las acciones realizadas para las historias anteriores.

Con esto lo que se quiere es unificar la estética de las notificaciones de la web. Se realizará desde el 19 al 22 de Abril.

3.13.1. Diseño de la interfaz y de la BD

Para las historias de usuario relativas al comercio y al validador no ha sido necesario diseñar ningún boceto nuevo. El cierre de sesión sigue la misma estética que para el administrador: un botón en el menú desplegable lateral. Se puede ver en la figura 7.

El cierre de sesión para el consumidor nos implica realizar su dashboard, que va a seguir la línea de los bocetos realizados para las anteriores, se pueden ver en las figuras 7, y 16. Lo que cambia respecto a las demás son las funcionalidades a las que tendrá acceso.

Con respecto a la base de datos, no tenemos que realizar ninguna modificación, mantenemos el diseño que tenemos y que puede consultarse en la figura 34.

3.13.2. Implementación estática y dinámica

Implementación HU 15

La implementación del cierre de sesión del validador ha sido sencilla, ya que solo hemos tenido que modificar el menú desplegable lateral. En él se ha incluido el botón de cierre de sesión .

La lógica del botón ya se encontraba implementada para el administrador por lo que hemos enlazado el documento js, cierre_sesion_admin.js, a las páginas del validador. La funcionalidad sería que cuando pulsamos el botón se mira el token del usuario que tiene la sesión iniciada, independientemente de su rol, y con ello comprobamos que la sesión está activa y se produce el cierre de la misma.

El resultado ha sido el siguiente:

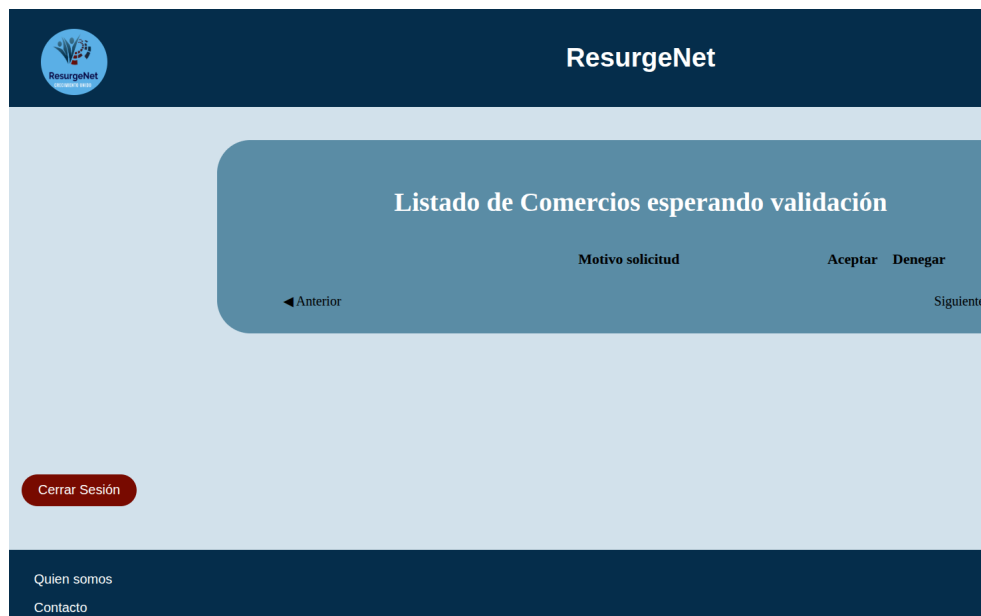


Figura 77: Implementación del cierre de sesión para los comercios

Lo que cambia respecto al del administrador es que el validador no tiene menú lateral desplegable por ahora, ya que su funcionalidad actual es solo aceptar/denegar las peticiones. Por ese motivo, el botón para el cierre de sesión se ha colocado en la esquina lateral izquierda antes del footer. Si en un futuro se incluyen otras funciones como un histórico de solicitudes se añadirá un menú desplegable que incluya todas ellas.

Implementación HU16

La implementación del cierre de sesión para los comercios es igual que para el validador. Hemos aprovechado la funcionalidad que ya teníamos para el administrador quedando de la siguiente manera:



Figura 78: Implementación del cierre de sesión para los comercios

Una vez que pulsamos el botón de cierre de sesión se redirige a la landing page y nos aparece un mensaje que nos informa que seremos redirigidos en 3 segundos.

Implementación HU17

Para esta historia de usuario hemos reutilizado los dashboard con los que ya contábamos, es decir, hemos realizado una copia del existente para el del comercio y hemos realizado modificaciones sobre el mismo. Esto se ha podido hacer así ya que es el panel de inicio, cuando se desarrollen las funcionalidades del consumidor tendremos que hacer su propio código. La implementación ha quedado de la siguiente manera:

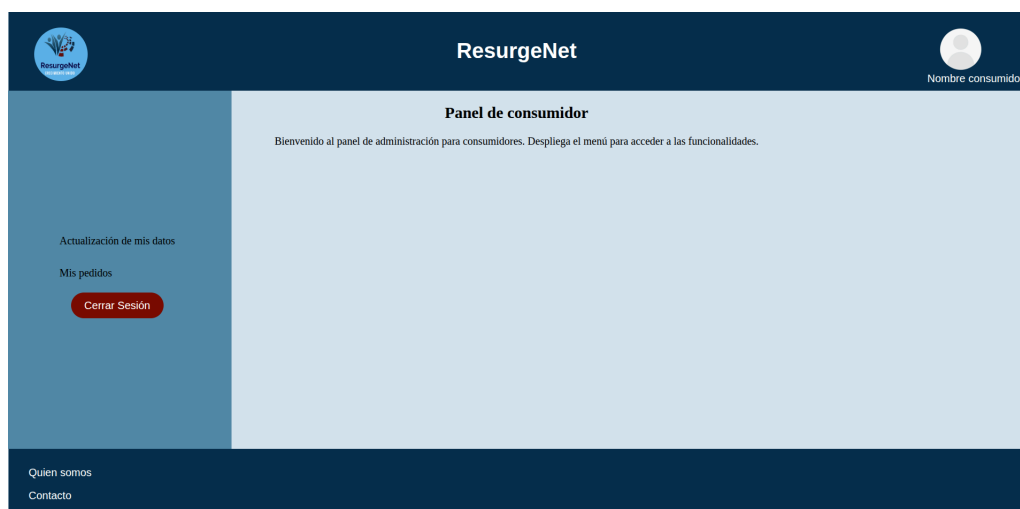


Figura 79: Dashboard para los consumidores incluyendo el cierre de sesión

La lógica para el botón de cierre de sesión se ha implementado de la misma forma que para los comercios.

Implementación de las mejoras de las notificaciones de las anteriores HUs

Para las mejoras, lo que se ha llevado a cabo ha sido la sustitución de los mensajes de tipo alert a unos personalizados en los siguientes archivos js:

- gestion_consumidores.js
- carga_comercios_activos.js
- actualizar_prod.js
- gestion_comercios_espera.js
- gestion_consumidores.js
- solicitudesComercios.js

Para realizar el cambios hemos puesto la función showModal que utilizábamos en otros archivos, como el login.js, y hemos ajustado los alert previos que teníamos. Esto posteriormente en una etapa de refinamiento se implementará como un archivo aparte común para varios js.

Comprobación de la accesibilidad de las HU desarrolladas

Lo primero que hemos hecho ha sido pasarle la herramienta WAVE a cada una de las implementaciones realizadas obteniendo los siguientes resultados:

HU	Salida WAVE	Solución
HU 15 - cierre del validador	0 errores	No hay que modificar nada
HU 16 - cierre del comercio	0 errores	No hay que modificar nada
HU 17 - dashboard + cierre del consumidor	0 errores	No hay que modificar nada

Para las comprobaciones manuales se ha navegado por la página con el TAB y se ha comprobado su correcto funcionamiento. El resto de las comprobaciones (texto alternativo, estructura semántica y diseño simple) se ha comprobado mediante se realizaba la implementación, por lo que su estado actual es el adecuado.

3.13.4. Revisión por parte de los tutores

Durante la revisión no se han llevado a cabo bocetos por lo que no se ha podido obtener una opinión de los clientes de los mismos. Sin embargo, al ser el mismo formato que para el resto de iteraciones la retroalimentación fue positiva.

3.13.5. Retrospectiva de la iteración

Durante el desarrollo de esta iteración se han completado en su totalidad las HU planificadas. Por tanto, la planificación temporal ha sido la adecuada y se mantendrá para la siguiente iteración.

3.14. Iteración 10

Durante esta iteración, que abarca desde el 27 de Abril hasta el 13 de Mayo, vamos a desarrollar las historias de usuario relacionadas con el dashboard del consumidor. Estas historias no formaban parte originalmente del listado y se han tenido que planificar para esta iteración. El motivo es que, al llevar a cabo el cierre de sesión para este tipo de usuario, nos hemos dado cuenta de la necesidad de incluirlas para que cuenten con un dashboard completo. Tener este espacio completo para los consumidores es necesario ya que proporciona un punto centralizado desde el que gestionar la información y funcionalidades asociadas a este tipo de usuario, garantiza una experiencia de uso consistente dentro del sistema.

Por tanto las HU que de implementarán son:

HU18 - Como consumidor quiero modificar mis datos personales

Las tareas relacionadas con esta historia son:

- Diseño de la interfaz
- Diseño de la BD
- Revisión BD
- Implementación estática
- Implementación dinámica
- Pruebas de aceptación
 - Visualización correcta de los datos actuales
 - Cambios correctos de los datos tanto para el consumidor como en la base de datos
 - Comprobación de la accesibilidad

HU19 - Como consumidor quiero tener una zona donde ver los pedidos que he realizado y consultar su estado

Las tareas relacionadas con esta historia son:

- Diseño de la interfaz
- Diseño de la BD
- Revisión BD
- Implementación estática
- Implementación dinámica
- Pruebas de aceptación
 - Visualización correcta de los pedidos
 - Acceso a los datos específicos de cada pedido
 - Comprobación de la accesibilidad

Es un hecho que la inteligencia artificial está cada día más incorporada en el entorno tecnológico por lo que nos hemos propuesto lo siguiente: en esta iteración vamos a usar la IA desde el primer momento para que implemente estas historias de usuario y comprobar realmente su nivel de utilidad. Con esta decisión pretendemos que la IA nos ayude a resolver problemas puntuales, que es el uso que hasta ahora se le ha dado a la IA durante el desarrollo de este proyecto, pero también queremos que nos ayude a realizar un desarrollo supervisado como si fuera un miembro más del equipo de trabajo del proyecto.

3.14.1. Preparación del entorno

Durante el desarrollo del proyecto el editor de código utilizado ha sido Visual Studio Code, sin asistencia de IA. Para esta última iteración se tomó la decisión de incorporar un asistente de inteligencia artificial con el objetivo explicado anteriormente.

La primera herramienta que se consideró para llevarlo a cabo fue usar GitHub Copilot Chat que está integrado con Visual Studio Code. Sin embargo presenta dos limitaciones que nos hicieron descartarlo:

- **Límite de tokens** → el plan gratuito impone un límite mensual de uso del chat basado en tokens. Para una fase de experimentación intensiva donde se pretende realizar al completo dos historias de usuario este límite es demasiado restrictivo y habría interrumpido el flujo de trabajo antes de completarlo. [35][36]
- **Carga computacional en el entorno local** → las funcionalidades avanzadas del modelo agente de esta herramienta requieren un gran procesamiento en la máquina local. El hardware disponible no cuenta con los recursos suficientes para soportar la carga de forma fluida.

Frente a estas limitaciones, siendo la más problemática la limitación hardware, se buscó una alternativa en la nube.

La primera fue utilizar GitHub Code Spaces que es un entorno de desarrollo completo alojado en la nube que proporciona una instancia de Visual Studio Code accesible desde el navegador. También tenemos una opción más ligera llamada github.dev que se abre pulsando la tecla . (punto) desde cualquier repositorio. Es gratuita e instantánea, pero no tiene terminal ni nos permite ejecutar código. El problema con esta alternativa es que seguimos teniendo el plan gratuito, por lo que seguimos con las limitaciones de tokens comentadas, para paliar esto se intentó cambiar el plan, pero el 20 de Abril GitHub pausó los nuevos registros para los planes de pago por el gran uso que tiene su modelo agente. [31][32][36][37]

Ante la imposibilidad de usar GitHub Copilot se buscó otra opción que nos permitiera cumplir los mismos objetivos: un asistente de inteligencia artificial en la nube capaz de integrarse en nuestro repositorio y operar sobre él de forma autónoma bajo supervisión. El elegido ha sido Claude [29] por los siguientes motivos:

- No tiene bloqueo para mejorar el plan ni límites semanales restrictivos [30]
- Funcionamiento completamente en la nube ya que todo el procesamiento se hace en los servidores de Anthropic.[29] El equipo local solo tiene que ejecutar el navegador y contar con conexión a Internet, por lo que reducimos los requisitos hardware necesarios para usar la herramienta.
- Integración real y sencilla con GitHub mediante Git MCP [28], esto nos permite trabajar en la nube sin usar nuestro sistema local que hemos comprobado que no da las prestaciones que se necesitan.
- Capacidad de razonamiento sobre el proyecto al completo, porque Claude es capaz de analizar la arquitectura global del proyecto, detectar los problemas de configuración y explicar cada decisión razonadamente[29][30], esto facilita la supervisión y el aprendizaje
- Accesible desde el navegador sin instalaciones adicionales [29].

En el siguiente apartado se detalla cómo se ha realizado la integración de Claude con nuestro repositorio de GitHub.

3.14.2. Integración de Claude con GitHub

Nuestro objetivo es que Claude pueda leer y escribir directamente sobre el repositorio de GitHub, y se llevaron a cabo tres intentos para obtener la solución final. Antes de ver estos intentos vamos a explicar una serie de conceptos:

- **¿Qué es un conector?** → Para que Claude pueda interactuar con servicios externos necesita conectarse a ellos, usar un puente que autoriza a la herramienta a comunicarse con esos servicios en nombre del usuario. Ese puente es conocido como conector. [30]
- **¿Qué es MCP?** → Es un protocolo estándar creado por Anthropic que define cómo los asistentes de IA pueden conectarse e interactuar con herramientas y servicios externos de forma segura y estructurada. [28], [39], [40] En nuestro proyecto, los servicios externos corresponden a las funcionalidades proporcionadas por la web app desarrollada.
- **¿Qué es OAuth y por qué GitHub solicita autorización?** → Es un mecanismo de autorización estándar que permite que una app acceda a los recursos de otra en nombre del usuario sin que este tenga que compartir su contraseña [38]. En este caso, GitHub utiliza este mecanismo para que el usuario conceda explícitamente permisos de acceso al repositorio del proyecto desde el que Claude va a trabajar e interactuar. Este paso es obligatorio por motivos de seguridad, ya que sin dicha autorización Claude no puede acceder ni realizar operaciones sobre el repositorio.

Primer intento

Se quiso hacer una integración nativa de GitHub en Claude, para ello tuvimos que iniciar sesión en claude.ai entrar en ajustes y conectores. Una vez ahí darle a “Conectar” la integración con GitHub que es un conector nativo de Anthropic.

El resultado fue que Claude podía leer ficheros que contiene el repositorio de GitHub donde subimos este proyecto y listar los repositorios asociados a nuestra cuenta de usuario pero no tenía permisos de escritura. No tener permiso de escritura supone una limitación, ya que para aprovechar la integración es necesario permitir que el asistente pueda modificar archivos y aplicar cambios sobre el código del proyecto.

Segundo intento

Se probó el conector de GitHub Copilot, disponible en el listado de integraciones de Claude. El resultado fue el mismo que el anterior, acceso de solo lectura.

Tercer intento - solución final

Se optó por utilizar Claude GitHub MCP Connector. Tuvimos que revocar todos los accesos previos desde GitHub, volver a los conectores de Claude y conectar Git MCP. GitHub nos muestra una pantalla de autorización que debemos permitir. Una vez concedida la autorización verificamos si aparece en Installed GitHub Apps y le damos acceso al repositorio del proyecto, llamado ResurgeNet, que es donde se encuentra alojado el código fuente que vamos desarrollando.

En la siguiente tabla vemos un resumen de los conectores probados:

Conector	Lectura	Escritura	Cómo activarlo
Integración nativa con GitHub	Sí	No	Ajustes → Conectores
GitHub Copilot MCP	Sí	No	Ajustes → Conectores
Claude GitHub MCP Connector	Sí	Sí	Ajustes → Conectores → Git MCP + instalación en GitHub

3.14.3. Para qué vamos a usar Claude

Una vez completada la integración se decidió usar Claude como asistente de desarrollo de software con el objetivo de analizar cómo se comporta la IA cuando dispone de acceso contextualizado a un proyecto completo y puede participar activamente en el mismo.

No queremos delegar completamente el desarrollo sino evaluar el flujo de trabajo que genera esta herramienta como agente capaz de: proponer, generar o modificar implementaciones, mientras que el programador toma un rol de supervisor y validador de toma de decisiones.

Por tanto se va a usar Claude para:

- Generación de bocetos y diseños
- Modificación y revisión del diseño de la base de datos
- Implementación de parte estática y dinámica de las HU correspondientes a esta iteración
- Revisión de la accesibilidad del sistema

Nuestro rol ha sido, como se comentó anteriormente, el de analizar los resultados propuestos, validar los mismos y corregir los posibles errores decidiendo que integramos finalmente en el proyecto.

3.14.4. Realización de diseños Claude + Figma

Para poder utilizar Figma solo tenemos que activar el conector (Ajustes → Conectores) y darle el enlace del fichero donde están los bocetos. Una vez que cuenta con esa información solo tenemos que especificar lo que queremos que haga.

El primer boceto que vamos a realizar es el correspondiente a la HU18, el formulario para modificar los datos que tendrá accesible el consumidor desde su dashboard. El mensaje enviado a Claude ha sido el siguiente:

Petición 1: “Necesito que me hagas un diseño siguiendo la estética de los que ya hay para el dashboard de los consumidores. Es un formulario para modificar sus datos.”

El resultado obtenido ha sido el siguiente:

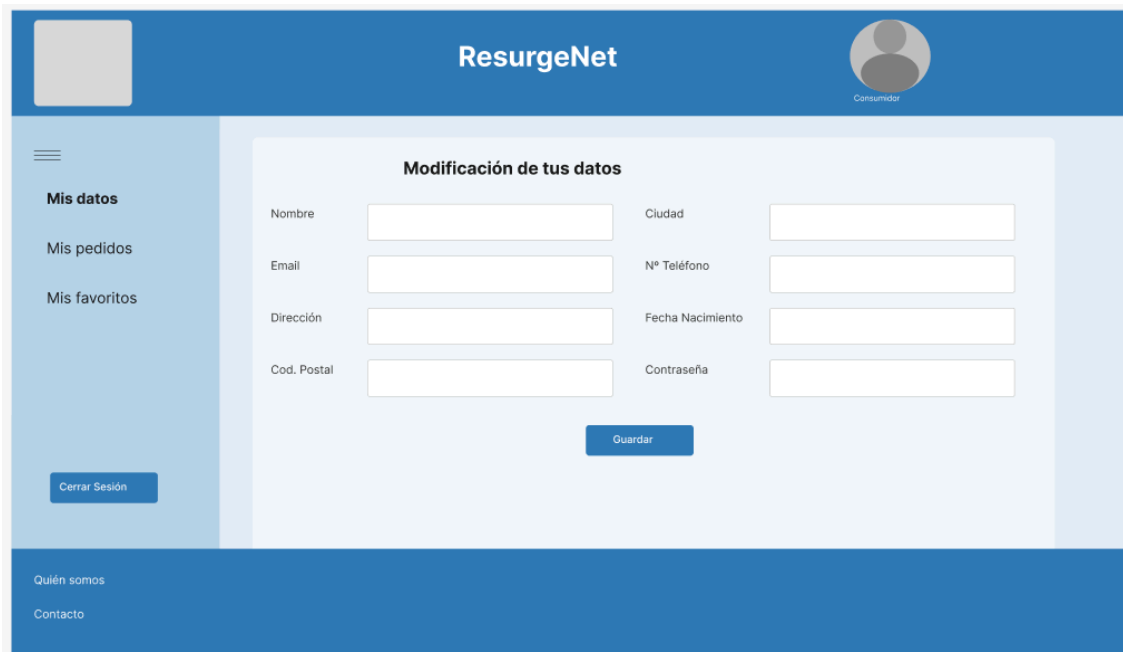


Figura 80: Resultado de la primera petición de diseño con Claude

Como se puede apreciar, comparando este con el resto de diseños, ha captado la idea del formulario pero no los colores ni el formato de los botones. Por tanto, vamos a hacer otra petición más específica.

Al hacer una nueva petición Claude nos dice que no puede realizarla, esto se debe a que el plan gratuito de Figma tiene limitaciones con el uso del protocolo MCP. Tendríamos que pasarnos a una versión de pago para poder seguir diseñando.

Como alternativa se intentó utilizar Figma Make, que es una herramienta de Figma que usa la inteligencia artificial para generar interfaces y prototipos. Se probó a refinar el diseño que se puede ver en la Figura 80, sin embargo el tiempo invertido no nos llevó a un diseño satisfactorio. Hay que hacer muchas indicaciones para que quede justo como se desea, sería más rápido hacerlo manualmente que usando esta herramienta.

Por tanto, en vista de que quedan por realizar dos diseños de interfaz, se ha decidido llevarlo a cabo manualmente. De cara a futuros proyectos, sería interesante contar con el plan Profesional de Figma para aprovechar la integración con Claude, ya que es una herramienta fácil de usar que obtiene buenos resultados con pocos comandos.

Finalmente el diseño para la HU18 ha quedado de la siguiente manera:



ResurgeNet



Consumidor

Inicio

Mis pedidos

Cerrar Sesión

Quién somos

Contacto

Modificación de tus datos

Nombre

Email

Dirección

Cod. Postal

Ciudad

N° Teléfono

Fecha Nacimiento

Contraseña

Guardar

Figura 81: Diseño refinado manualmente para la modificación de los datos del consumidor

Como podemos ver, desde el dashboard del consumidor tendremos acceso, a través del menú desplegable, al formulario de modificación de los datos. Éste nos muestra los campos que podemos modificar, que son los mismos que rellenamos al crear nuestra cuenta. Cuando pulsemos el botón de guardar, esos cambios se harán efectivos en la base de datos.

El siguiente boceto que tenemos que realizar es el asociado a la HU19, la visualización del estado del pedido por parte del consumidor. Podremos acceder al área “Mis pedidos” tanto desde el dashboard como desde el apartado “Mis datos”. Se mostrará una tabla donde veremos el número de referencia del pedido, el comercio que lo ha enviado y el estado. Los diferentes estados pueden ser: en preparación, enviado, en reparto y entregado.

El boceto queda de la siguiente manera:



ResurgeNet



Consumidor

Inicio

Mis pedidos

Quién somos

Contacto

Estado de mis pedidos

N° de pedido	Nombre Comercio	Estado del pedido
XXXXXX	Comercio1	En reparto
XXXXXX	Comercio1	En preparación

Anterior
Siguiente

Figura 91: Boceto para la interfaz de la HU19, estado de los pedidos del consumidor

3.14.5. Diseño de la base de datos con Claude

El último esquema de base de datos que tenemos desarrollado es el que podemos ver en la Figura 34.

Para llevar a cabo la HU18, actualización de datos personales por parte de los consumidores, no tendríamos que hacer modificaciones en nuestra base de datos. Ya contamos con la información necesaria al tener la tabla Usuario y Consumidor conectadas. Cuando el consumidor actualice los datos se hará una modificación sobre la tabla correspondiente y se volcará la nueva información en la visualización de sus datos.

Ahora vamos con la HU19, consultar el estado de los pedidos siendo consumidor. Para crear el nuevo esquema le hemos pedido a Claude que estudie el antiguo y nos indique qué cambios se requieren. Nos ha dicho que necesitamos incorporar una nueva tabla: pedido, que se relacione con el consumidor y con el comercio. Necesitamos que haya relación con dichas tablas para saber a qué consumidor pertenece cada pedido, qué comercio ha sido implicado y qué productos se han solicitado por parte del consumidor.

Por tanto nuestro esquema de base de datos queda de la siguiente manera:

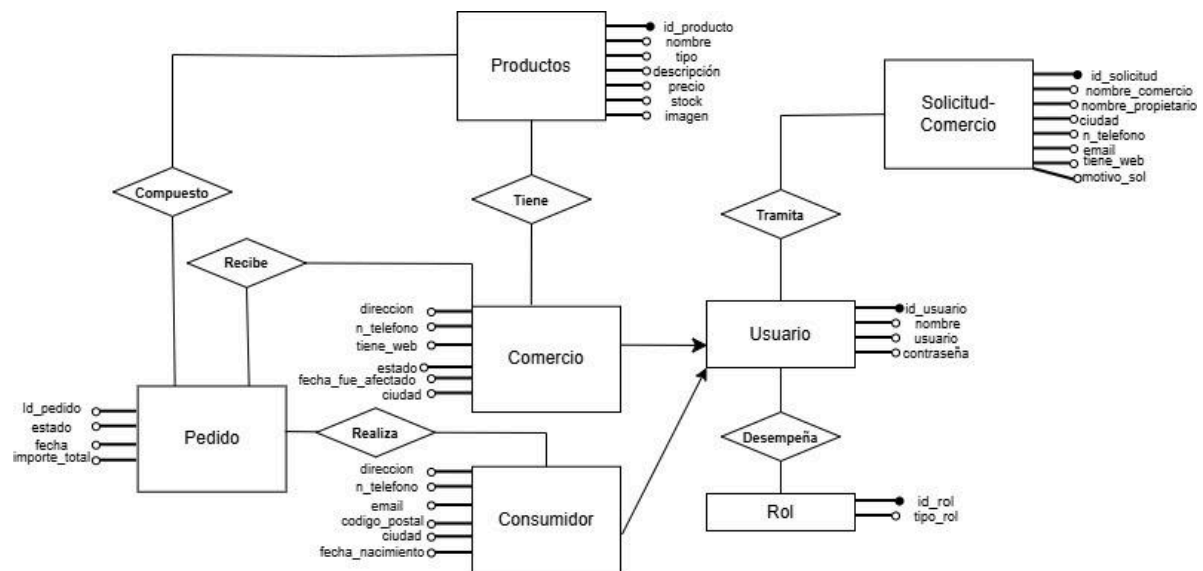


Figura 92: Esquema E-R actualizado para la HU19, consulta del estado del pedido por parte del consumidor

3.14.6. Implementación estática y dinámica con Claude

Le vamos a pedir a Claude que nos ayude en la implementación de la HU18 e HU19.

Implementación HU18

Ya contamos con Claude integrado en nuestro GitHub y podemos hacerle peticiones para que nos genere código funcional. Vamos a aprovechar esto para comenzar con la implementación de la actualización de los datos de los consumidores.

La primera petición que vamos a hacer es la realización estática de esta historia de usuario, vamos a pedirle que genere nuevos archivos con el código necesario para:

- Acceder desde el dashboard del consumidor al apartado 'Mis datos'

- Una vez dentro, puede ver un formulario con los campos que cuenta el boceto mostrado en la Figura 81, así como el correspondiente botón para guardar los cambios.

La petición a Claude ha sido lo más exhaustiva posible para que nos de el mejor resultado, siendo la siguiente:

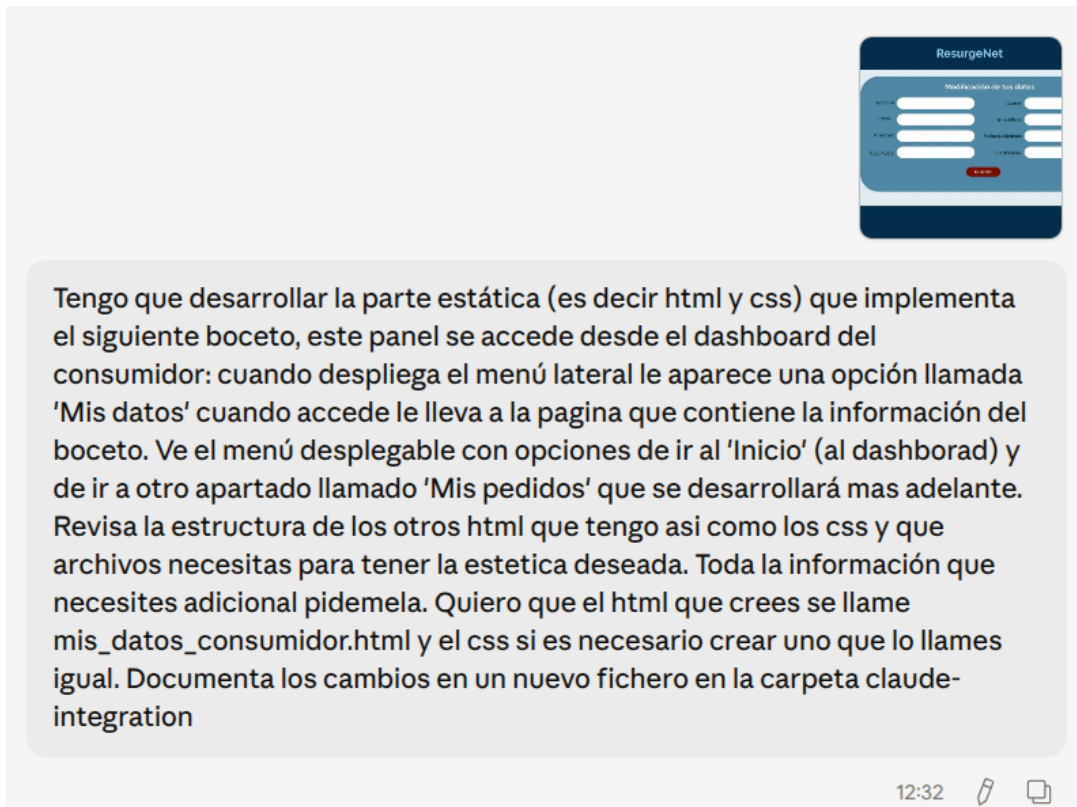


Figura 82: Petición a Claude para que genere la parte estática de la HU18

Una vez que ha finalizado de desarrollarlo, volcamos lo que tenemos nuevo en el repositorio, a nuestro entorno local, para revisar qué archivos ha generado y cómo.

Lo primero que hemos revisado ha sido la generación en las carpetas correspondientes de los archivos solicitados, nos ha creado un archivo HTML llamado 'mis_datos_consumidor.html' y uno CSS llamado 'mis_datos_consumidor.css' cada uno como se solicitó y en su carpeta de ruta correspondiente.

```
angela@angela-HP-Laptop-14s-dq4xxx: ~/Escritorio/ResurgeNet
angela@angela-HP-Laptop-14s-dq4xxx:~/Escritorio/ResurgeNet$ ls frontend/
actualizar_producto.html      gestion_consumidores_admin.html
admin_dashboard.html         imagenes
alta_productos.html          index.html
comercio_dashboard.html      inicio_sesion.html
consumidor_dashboard.html    js
dashboard_validador.html    listado_productos_comercio.html
formulario_contacto_comercios.html mis_datos_consumidor.html
gestion_comercios_admin_espera.html registro_consumidores.html
gestion_comercios_alta_admin.html style
angela@angela-HP-Laptop-14s-dq4xxx:~/Escritorio/ResurgeNet$ ls frontend/style/
admin_dashboard.css      gestion_cons_comer_admin.css
alta_productos.css       inicio_sesion.css
dashboard_validador.css  landing.css
estructura.css           listado_productos.css
formulario_contacto_comercios.css mis_datos_consumidor.css
gestion_comercio_alta_admin.css registro.css
```

Figura 83: Comprobación de la correcta creación de los archivos solicitados a Claude

El siguiente paso es comprobar el contenido de esos archivos. Tras revisar el html se puede comprobar que sigue la línea del resto de htmls creados manualmente, así como también ocurre con los css. Vamos a ver los pasos seguidos cuando queremos acceder a la nueva funcionalidad creada:

- Iniciamos la sesión como un consumidor y nos lleva al dashboard
- Cuando desplegamos el menú lateral y queremos acceder a 'Mis datos' no podemos, no se ha modificado el enlace desde el archivo 'consumidor_dashboard.html' con el archivo 'mis_datos_consumidor.html'. Tenemos que hacerle una nueva petición a Claude para que solucione ese problema. Esta petición es **"Desde el archivo consumidor_dashboard.html no tengo acceso desde el menú lateral al nuevo apartado 'mis_datos_consumidor.html'. Solúcnalo"**

Una vez que tenemos ambas páginas enlazadas, podemos ver que el formulario de datos sigue la estética indicada y cumple con los requerimientos.

The screenshot shows the ResurgeNet user interface. At the top, there's a dark blue header with the ResurgeNet logo on the left, the text 'ResurgeNet' in the center, and a user profile icon labeled 'Consumidor' on the right. Below the header, a light blue sidebar contains a hamburger menu icon. The main content area is titled 'Modificación de tus datos' and features a form titled 'Actualiza tu información personal'. The form has two columns of input fields: 'Nombre', 'Email', 'Dirección', and 'Cod. Postal' on the left; 'Ciudad', 'Nº Teléfono', 'Fecha Nacimiento' (with a DD/MM/AAAA placeholder), and 'Contraseña' on the right. A red 'Guardar' button is centered at the bottom of the form. The footer is dark blue with links for 'Quien somos' and 'Contacto'.

Figura 84: Resultado final de la implementación estática de la HU18

Ahora mismo tanto el formulario como el botón no tienen funcionalidad, vamos a pedirle a Claude que nos implemente lo siguiente:

- **Tarea 1** → Una función, en el archivo AuthController.php, para obtener los datos del consumidor que tiene la sesión iniciada, para que el formulario pueda contar con los campos rellenos con la información actual. El campo de la contraseña no se mostrará por motivos de seguridad,
- **Tarea 2** → Otra función en el mismo archivo para actualizar los nuevos datos que se ingresen en el formulario. Vamos a comprobar que no se quieran actualizar los datos que ya teníamos guardados, especialmente para el campo de la contraseña, ya que no tendrá visualización previa.
- **Tarea 3** → Un fichero JavaScript que nos une el front con el backend y que residirá en la ruta frontend/js.

Vamos ir pidiéndole cada una de las tareas poco a poco. Para la primera tarea le hemos indicado lo siguiente:

“Ya tenemos la parte estática, ahora vamos a ir haciendo la dinámica. Quiero que el formulario esté previamente relleno con los datos del consumidor que ha iniciado sesión. Toda la parte de la base de datos se hace en el fichero AuthControllers.php y las rutas se definen en el archivo api.php. Estudia el contenido de esos archivos para que sepas como se hacen las cosas de la parte del back”

La respuesta de Claude fue la siguiente:

Ya tengo todo lo que necesito. Ahora lo entiendo perfectamente:

Lo que existe actualmente:

- Cuando el consumidor hace login, el backend devuelve `id_usuario` y el JS de login lo guarda en `localStorage` con la clave `id_usuario_comercio` (misma clave para todos los roles).
- La sesión en el backend usa `session(['id_usuario'])`, pero el front no la usa — **todo se gestiona vía `localStorage`**.
- En la BD, los datos del consumidor están repartidos en dos tablas: `usuario` (nombre) y `consumidor` (dirección, ciudad, cod_postal, n_telefono, email, fecha_nac), enlazadas por `id`.

Lo que hay que hacer:

1. Crear en el backend un nuevo endpoint `GET /perfil_consumidor/{id}` que haga un JOIN entre `usuario` y `consumidor` y devuelva todos los campos del formulario.
2. Crear el JS `mis_datos_consumidor.js` que al cargar la página lea el `id_usuario_comercio` del `localStorage`, llame al endpoint y rellene los inputs.

¿Te parece bien ese planteamiento? Si me das el visto bueno lo implemento todo de una vez: el método en `AuthController.php`, la ruta en `api.php` y el fichero JS.

Figura 85: Razonamiento de Claude para llevar a cabo la petición

Como se puede ver, ha comprendido la tarea que hay que llevar a cabo y nos ofrece una solución que se alinea con nuestro planteamiento inicial y a lo que se ha llevado a cabo para el resto de historias de usuario. Le decimos que puede hacerlo y obtenemos lo siguiente:

- En el archivo `AuthController.php` tenemos una nueva función llamada `getPerfilConsumidor` que une los datos de la tabla `usuario` y la tabla `consumidor` usando el identificador del mismo para obtener el perfil completo del consumidor que tiene la sesión iniciada. Esos datos se devuelven en un json.
- En el archivo `api.php` encontramos la nueva ruta llamada `perfil_consumidor`.
- En la carpeta `frontend/js` contamos con un nuevo archivo llamado `mis_datos_consumidor.js` que precarga el formulario con los datos del consumidor que tiene la sesión iniciada. Esto sería el contenido de la tarea 3 mencionada anteriormente, como podemos ver, aunque no se le haya especificado explícitamente en la petición a Claude la creación de este archivo, lo ha llevado a cabo siguiendo la lógica del resto del proyecto.

Una vez que hemos comprobado el código que contienen los archivos, pasamos a comprobar el funcionamiento. Cuando accedemos al apartado ‘Mis datos’ del consumidor con id 9 obtenemos lo siguiente:

Figura 86: Resultado de la implementación de la Tarea 1

Comprobándolo con la base de datos, vemos que la información volcada es correcta:

```
Database changed
mysql> SELECT * FROM usuario;
+----+-----+-----+-----+-----+
| id_usuario | nombre | usuario | password | rol |
+----+-----+-----+-----+-----+
| 1 | Angela Garrido | angelamgr | 25f43b1486ad95a1398e3eeb3d83bc4010015fcc9bedb35b432e00298d5021f7 | 1 |
| 2 | David Lopez | david03 | e52473318e65c0b49ecc1c94141e9b34688906b7776f2f72824dbb0c9e366cd8 | 3 |
| 3 | Manuel Garrido | manuGr | 460a1446ca3e5c2707a62fb405cb643a7bf95dc0752a1505a40f639af4ced341 | 4 |
| 9 | Sofia Perez | sofi22 | 1018bdb757f02e266e25810e9dcb87c861c864bbf08d71d992c4483f9fdbc82 | 2 |
| 13 | Javier Garcia | javigar | 64d7de35447e6943b365232c6bdf560f86873ade1233f07e5d238dda88ab149 | 4 |
| 14 | Miguel Heredia | RecambiosTodoCoche | 112b31df65d811e50e5771525a5f4f81fd1f1ba9fbbe8e6d28209d18ae2b8c33 | 3 |
| 15 | Laura Martinez | EntreLineas | 88459fe78d7b07408fc021547c1d267b8cc128973ed16e2208674db3639c6659 | 3 |
+----+-----+-----+-----+-----+
7 rows in set (0.00 sec)

mysql> SELECT * FROM consumidor;
+----+-----+-----+-----+-----+-----+-----+
| id | direccion | ciudad | cod_postal | n_telefono | email | fecha_nac |
+----+-----+-----+-----+-----+-----+-----+
| 9 | Gran Vía 5 | Madrid | 15002 | 666085950 | sofiP_22@gmail.com | 1996-05-12 |
+----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Figura 87: Contenido de la base de datos para el usuario consumidor con id 9

La Tarea 1 está completamente operativa, por lo que pasamos a hacer la petición para la implementación de la Tarea 2, que el consumidor modifique los datos que desee y el cambio se haga efectivo en nuestra base de datos. La petición realizada ha sido la siguiente:

“Vamos al siguiente paso. Tienes que hacer una función en AuthControllers.php para que cuando se modifique algún cambio del formulario y se pulse el botón de guardar se actualice la base de datos. Recuerda crear la ruta en el api.php y crear/modificar los archivos que consideres aunque antes de hacerlo preguntame y explicame porque lo haces. Ten en cuenta también que cuando se guarden los cambios debería de salir un mensaje indicándolo con la estructura showModal del resto de js, también implementa que si quiere cambiar la fecha de nacimiento no sea a futuro (es decir una fecha adelantada a la actual)”

El procedimiento a seguir ha sido el mismo que el anterior: comprobación de creación de archivos → comprobación del contenido → visualización en la web app.

El fichero AutControllers.php tiene una nueva función llamada actualizarPerfilConsumidor que lo que hace es:

- Validar la fecha de nacimiento que se quiere modificar para que no pueda ser la actual ni una fecha futura.
- Actualizar el nombre en la tabla usuario
- Actualizar el resto de datos en la tabla consumidor
- Actualizar la contraseña solo si el campo se ha rellenado

Al ver el flujo de trabajo que ha seguido, nos hemos dado cuenta de que no necesitamos actualizar toda la tabla si no solo los campos que sufren modificaciones. Con el enfoque actual que nos ha dado Claude, aunque el consumidor no modifique nada si pulsa el botón “Guardar” el backend lanza dos actualizaciones que modifican columnas tanto en las tablas del consumidor como en la de usuario. Esto tiene un coste innecesario sin que haya cambios reales. Por ello se hizo una nueva petición para que modificara la lógica de esta función y quedará optimizada. Ahora hace lo siguiente:

- Carga el perfil actual
- Compara cada campo y construye dinámicamente el array de datos a actualizar, solo con los que son diferentes
- Si el array está vacío no ejecuta la actualización de la tabla

El fichero `mis_datos_consumidor.js` también ha sido modificado incluyendo:

- La función `showModal` para darle estructura a las notificaciones, y se visualiza lo siguiente cuando se han actualizado los datos:

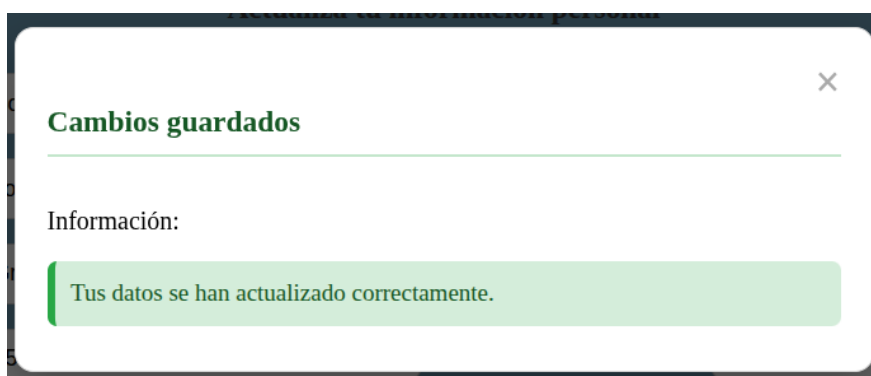


Figura 88: Visualización de las notificaciones siguiendo la estética del resto.

- Funcionalidad para guardar los cambios permitiendo que se envíen los datos modificados al backend

Para probar el funcionamiento en la interfaz, hemos modificado los datos como si fuéramos la consumidora Sofía Perez, hemos cambiado el código postal y la ciudad. Inicialmente nuestra base de datos tiene almacenada la siguiente información para estos campos:

```
Database changed
mysql> SELECT ciudad, cod_postal FROM consumidor;
+-----+-----+
| ciudad | cod_postal |
+-----+-----+
| Madrid |      15002 |
+-----+-----+
1 row in set (0.00 sec)
```

Figura 89: Valores de los campos ciudad y código postal antes del cambio

Tras rellenar el formulario con la nueva información, el código postal pasa a ser 15034 y la ciudad Alicante, y si guardáramos los cambios obtenemos lo siguiente:

```
mysql> SELECT ciudad, cod_postal FROM consumidor;
+-----+-----+
| ciudad | cod_postal |
+-----+-----+
| Alicante | 15034 |
+-----+-----+
1 row in set (0.00 sec)

mysql>
```

Figura 89: Valores de los campos ciudad y código postal tras el cambio

Si volvemos a iniciar sesión como Sofía Perez y vamos al apartado “Mis datos” vemos que se visualiza correctamente la nueva información:

The screenshot shows a web form titled "Modificación de tus datos" (Modification of your data). Inside, there's a section "Actualiza tu información personal" (Update your personal information). The form contains several input fields with labels and values:

Field Label	Value
Nombre	Sofia Perez
Ciudad	Alicante
Email	sofiP_22@gmail.com
Nº Teléfono	666085950
Dirección	Gran Vía 5
Fecha Nacimiento	12/05/1996
Cod. Postal	15034
Contraseña	Dejar en blanco para no cambiar

At the bottom of the form is a red button labeled "Guardar" (Save).

Figura 90: Comprobación de la correcta visualización de los datos actualizados

Como podemos comprobar, los cambios se ven reflejados correctamente tanto en la base de datos como en la webapp.

Implementación HU19

Al igual que para el desarrollo de la anterior historia de usuario hemos hecho la petición a Claude para que lo implemente. Esta vez ha sido una petición más extensa que engloba tanto el front como el backend. Queríamos poner a prueba el comportamiento con instrucciones que demanda más trabajo en distintos archivos. La petición ha sido la siguiente:

“Tenemos que hacer otra historia de usuario, que el consumidor desde el dashboard de inicio pueda acceder al apartado 'Mis pedidos' y que allí se vea lo que te paso en el boceto. Además de ese boceto hay que revisar la base de datos, para que admita la lógica de mirar el estado de los pedidos. Quiero que implementes todo lo necesario para el frontend y las funciones del backend para que se consiga volcar lo que se guarde en las tablas en la interfaz.

Además, dime qué tengo que modificar de la base de datos en base al esquema y lo que queremos hacer. Antes de hacer nada infórmame de qué vas a crear”

Su respuesta fue la siguiente:

- **Esquema de BD** → Añadir la tabla pedidos a la base de datos que tenga relación con el consumidor y el comercio
- **Frontend** → creación de un nuevo archivo HTML (mis_pedidos_consumidor.html), un nuevo archivo CSS para los estilos propios de la página y la creación de un nuevo archivo JS (mis_pedidos_consumidor.js) que conecte el back con el front y obtengamos la visualización de los pedidos actuales que se guarden en la base de datos. Conectar el fichero consumidor_dashboard.html con mis_pedidos_consumidor.html
- **Backend** → un nuevo método en el archivo AuthController.php llamado getPedidosConsumidor que hace un JOIN entre pedido, usuario y comercio para devolver: el identificador del pedido, el nombre del comercio y el estado del mismo de cada pedido que haya realizado el consumidor que tiene la sesión iniciada. Además una nueva ruta en el archivo api.php

Una vez analizados los cambios que quiere hacer, vemos que se alinean con lo que hemos solicitado, por lo que le decimos que puede llevarlos a cabo. El único cambio que no puede hacer directamente es la creación de la tabla pedido ya que Claude no tiene acceso al entorno de MySQL que utilizamos, sin embargo sí que nos da la sentencia que tenemos que ejecutar para la correcta creación de la tabla:

```
CREATE TABLE pedido(id_pedido INT AUTO_INCREMENT PRIMARY KEY, id_consumidor INT NOT NULL, id_comercio INT NOT NULL, estado ENUM('En preparación', 'En reparto', 'Entregado', 'Cancelado') NOT NULL DEFAULT 'En preparación', fecha DATETIME DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY (id_consumidor) REFERENCES usuario(id_usuario) ON DELETE CASCADE, FOREIGN KEY (id_comercio) REFERENCES comercio(id) ON DELETE CASCADE );
```

Una vez analizado el contenido de los ficheros que ha creado, vamos a comprobar la visualización en la interfaz. Desde el dashboard del consumidor accedemos al apartado ‘Mis pedidos’ visualizando lo siguiente:



Figura 93: Visualización de la interfaz para ver el estado de los pedidos del consumidor

Como podemos ver, sigue perfectamente la estética del resto de nuestras páginas creadas. Así es como se vería si no tenemos ningún pedido realizado, cuando el consumidor tiene pedidos hechos vemos lo siguiente:



Figura 94: Visualización del listado de pedidos

Podemos ver cinco pedidos por sección, cuando tenemos esos cinco se activan las flechas de “Anterior” y “Siguiente” para poder navegar por las páginas de los pedidos.

Si visualizamos el contenido de la base de datos del sistema podemos ver que concuerda con los datos que se visualizan:

```
mysql> SELECT * FROM pedido;
+-----+-----+-----+-----+-----+
| id_pedido | id_consumidor | id_comercio | estado | fecha |
+-----+-----+-----+-----+-----+
| 4 | 9 | 3 | Entregado | 2026-05-15 10:59:46 |
| 6 | 9 | 14 | En reparto | 2026-05-15 11:01:02 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT nombreComercio FROM comercio WHERE id=3 OR id=14;
+-----+
| nombreComercio |
+-----+
| cultivoSaludable |
| RecambiosTodoCoche |
+-----+
2 rows in set (0.01 sec)
```

Figura 95: Datos de la base de datos para la tabla pedidos

Como se puede comprobar, Claude trabaja muy bien con instrucciones que implican modificaciones en múltiples archivos de forma simultánea y enlazada. Obteniendo una resolución satisfactoria para esta historia de usuario.

Las pruebas de aceptación de las historias de usuario realizadas se han ido llevando a cabo en cada comprobación del estado de la base de datos al igual que de acceso y visualización de

contenido, todo ello puede verse en las capturas de pantalla proporcionadas para esta iteración.

3.14.7. Comprobación de la accesibilidad de las HUs desarrolladas

Lo primero que hemos hecho ha sido pasarle la herramienta WAVE a cada una de las implementaciones realizadas obteniendo los siguientes resultados:

HU	Salida WAVE	Solución
HU 18 - modificación de datos	0 errores	No hay que modificar nada
HU 19 - estado de los pedidos	5 errores de contraste	Cambiar los colores de la cabecera de la tabla y resaltar los botones de navegación del formulario

Además le pedimos a Claude que analizara el resto de aspectos como la navegación usando el tabulador, los textos alternativos etc. y nos generó el siguiente informe:

Resumen de prioridades	
Prioridad	Problema
Alta	<code>toggle-btn</code> es <code><p></code> , no navegable por teclado
Alta	Botones Anterior/Siguiente son <code><div></code> , no navegables
Alta	Modal sin <code>role="dialog"</code> ni gestión de foco
Media	Botón cerrar modal es <code></code>
Media	Tabla de pedidos con <code><div></code> en vez de <code><table></code>
Baja	<code>alt=""</code> en imagen decorativa de perfil
Baja	<code>aria-label</code> en el <code><nav></code> del sidebar

Figura 96: Informe de errores de accesibilidad generado por Claude

Nos preguntó si queríamos implementar las mejoras y le pedimos que hiciera los cambios oportunos para solucionar y nos explicara qué estaba haciendo, nos dijo lo siguiente:

“En primer lugar, se han sustituido elementos no interactivos que actuaban como botones, concretamente el botón de despliegue del menú lateral y los controles de paginación, por elementos `<button>` semánticos, dotándolos de los atributos ARIA necesarios para que su estado y propósito sean comprensibles por lectores de pantalla y navegables mediante teclado. El componente modal de mensajes se ha enriquecido con `role="dialog"`, `aria-modal="true"` y `aria-labelledby`, asegurando que al abrirse el foco del usuario se desplace automáticamente a él y el lector de pantalla anuncie su título. La tabla de pedidos se ha reescrito utilizando elementos semánticos nativos de HTML, lo que permite que los lectores de pantalla asocien correctamente cada celda con su encabezado de columna. Las imágenes meramente decorativas se han marcado con `alt=""` y `aria-hidden="true"` para que sean ignoradas por las tecnologías de asistencia, evitando ruido innecesario en la navegación. Se ha incorporado la clase utilitaria `.sr-only` para proporcionar textos de ayuda visualmente ocultos pero accesibles, como indicaciones de formato en los campos de fecha y contraseña. Finalmente, se ha

mejorado la gestión del foco a lo largo de toda la interfaz, añadiendo soporte para la tecla *escape* en el cierre del menú lateral y del modal, y asegurando que todos los elementos interactivos dispongan de estilos de foco visibles mediante outline, en cumplimiento del criterio de éxito 2.4.7 de las WCAG”

3.14.7. Revisión por parte de los tutores

Los tutores han revisado los resultados de la iteración y aprendido/comprobado cómo Claude puede ser útil de apoyo al diseño e implementación. Concluyen que por el momento, es imprescindible la labor de una persona que tenga claro qué necesita y cómo pedirlo, y que luego supervise, compruebe, complete y modifique lo generado para cubrir completamente los requisitos funcionales y no funcionales. Las historias planificadas para esta iteración han sido implementadas con éxito

3.14.8. Retrospectiva de la iteración

Durante el desarrollo de esta iteración se han completado en su totalidad las HU planificadas. Además, se ha adquirido conocimiento y práctica con la utilización de la IA para desarrollo de software. Aprendiendo a: integrar en nuestro sistema la herramienta Claude, realizar el trabajo en la nube y pasar a ser un supervisor crítico con el trabajo realizado y no un programador al uso.

Por tanto, la planificación temporal ha sido la adecuada y los resultados obtenidos satisfactorios.

3.15. Iteración 11

Durante esta iteración, que abarca desde el 15 de Mayo hasta el 24 de Mayo, las tareas se centrarán principalmente en la optimización del código generado durante el desarrollo de las distintas historias de usuario. Vamos a comenzar por hacer una revisión exhaustiva de todo lo relativo al frontend, es decir, archivos HTML, CSS y JS. Para posteriormente repetir el proceso con el backend.

Se utilizará Claude como herramienta de apoyo para estas tareas. Gracias a la integración realizada en la iteración anterior el asistente puede analizar el código existente y proponer mejoras relacionadas con su rendimiento, mantenimiento y estructura.

En todo momento esas propuestas serán supervisadas, revisadas y validadas manualmente manteniendo un control sobre las decisiones técnicas finales.

3.15.1. Optimización frontend

Hemos empezado analizando los archivos que forman parte de nuestro frontend. Le hemos hecho a Claude una petición para que los revise en profundidad y nos proponga mejoras para optimizar rendimiento, mantenimiento y estructura. Nos ha indicado los siguientes problemas a solucionar:

- Duplicidad de la función `showModal` en cada archivo JS
- Estilos inline en JS, es decir, estilos CSS que se aplican directamente sobre los elementos.
- Mensajes de depuración en producción
- `localStorage` usado para `id_usuario_comercio` sin expiración
- Archivos CSS con reglas duplicadas y sin variables globales
- El header y el footer no son componentes reutilizables

- URL del backend definida como localhost en producción que nos lleva a no tener distinción entre entornos

Duplicidad de la función showModal

La función showModal, y su contraparte hideModal, está en nueve archivos JS diferentes, cada uno con variaciones diferentes. Esto suponía que: cualquier cambio del modal requiere editar nueve ficheros y las variaciones entre copias generan inconsistencias.

La solución es crear un archivo JS que contiene una única implementación de las funciones mencionadas. Además de la creación de este archivo, tenemos que actualizar los HTMLs que usan el modal para que carguen el nuevo archivo creado. Este nuevo script debe cargarse antes que cualquier otro porque el resto llamarán a showModal sin redefinirla localmente, si se cargara después nos daría un error al no estar definida.

Hacemos la petición a Claude de que realice las modificaciones expuestas anteriormente obteniendo como resultado un único archivo funcional, llamado utils.js, que contiene la lógica deseada. Gracias a esta optimización los futuros cambios en el aspecto de las notificaciones del sistema implicarán actualizar un solo archivo agilizando el proceso.

Estilos inline en JS

Uno de los principios fundamentales del desarrollo web es la separación de responsabilidades. Con HTML definimos la estructura, con CSS el estilo y con JS el comportamiento. Cuando JS aplica estilos directamente se viola este principio y se generan una serie de problemas:

- Los estilos se reparten en dos lugares diferentes, archivos CSS y JS, dificultando encontrar el origen de un estilo concreto
- Cambiar el diseño de un componente requiere modificar archivos JS, tarea que no es responsabilidad del diseñador
- Los estilos inline tiene máxima especificidad CSS, lo que hace que sea muy difícil sobreescribirlos desde la hoja de estilos
- Los strings HTML generados dinámicamente son más difíciles de leer y mantener

Tras pedirle a Claude que llevará a cabo una revisión de los archivos JavaScript que tenemos encontró una serie de estilos inline relacionados con la visualización del modal de las notificaciones del sistema.

La solución ha sido crear un archivo css, llamado components.css, que centraliza todos los estilos que se encontraban antes en los distintos JS. Con esto hemos conseguido que cuando se quieran realizar modificaciones solo se tenga que editar un fichero.

Mensajes de depuración en producción

Tener mensajes de depuración en producción pueden generar problemas de seguridad, rendimiento y pueden revelar la lógica interna del proyecto. Claude hizo una revisión de los archivos JS del frontend y se encontraron dos archivos con este tipo de mensajes. Tenemos dos posibilidades:

- **Opción A** - Eliminar directamente estos mensajes.
 - Ventajas:
 - Código limpio y sin lógica externa
 - Coste cero en tiempo de ejecución

- SOLución permanente que no requiere mantenimiento en el futuro
 - Desventajas
 - Tenemos que volver a añadir los mensajes de depuración si en un futuro se necesitan
 - Se pierde el rastro de errores que se revisaron
- **Opción B** - Crear una función debug() con el flag de entorno, consiste en que la llamada a esta función sustituya a todos los console.log/console.error
 - Ventajas
 - Los mensajes de depuración siguen estando disponibles en local
 - Es útil cuando el proyecto tiene muchos logs informativos distribuidos por el código
 - Desventajas
 - Añade complejidad
 - El beneficio solo se refleja si hay muchos mensajes que aporten valor real que preservar. Si son escasos o poco informativos el sistema es sobrediseño
 - La función debug() oculta el nivel del log, es decir el tipo, lo que hace menos preciso el filtrado de los mismos en las herramientas del desarrollador

Teniendo en cuenta que solo quedan dos llamadas a console.error en todo el frontend, ambas en el mismo archivo, que dichos mensajes no aportan valor informativo real y que este proyecto sigue en fase de desarrollo se ha optado por implementar la primera posibilidad: la eliminación de esos mensajes. La eliminación no es un borrado al uso, hemos sustituido la llamada al log por un mensaje por pantalla a modo de notificación haciendo uso de showModal, así no perdemos la información que aportaban dichos mensajes.

Archivos CSS con reglas duplicadas y sin variables globales

Claude estudió los archivos CSS con los que cuenta el proyecto por el momento encontrando un problema estructural: bloques de código duplicados en múltiples archivos y los valores concretos del diseño, como colores, fuentes o radios de borde, escritos como literales en cada regla, sin ningún punto centralizado.

Esta situación comprometía la mantenibilidad y aumentaba el riesgo de inconsistencias visuales. Como solución, se implementó una estrategia de centralización de estilos basada en variables globales y clases reutilizables, permitiendo un mantenimiento más eficiente y una mayor coherencia en la interfaz. Se puede encontrar en el archivo main.css que contiene diez secciones diferentes:

- **Sección 1 - variables css.** Se definen variables nativas para todos los valores que antes se contrataban repetidos. Ahora para cambiar el color primario del proyecto vale con cambiar la variable --color-primary en un único lugar, el cambio se propaga automáticamente a todos los elementos.
- **Sección 2 - Reset mínimo y base.** La propiedad *box-sizing: border-box* se añadió como configuración global para asegurar que el padding y los bordes se incluyan dentro de las dimensiones declaradas de cada elemento. Esto simplifica el cálculo de tamaños y evita desbordamientos o inconsistencias visuales en la maquetación.
- **Secciones 3 y 4 - Header y footer.** Pasamos a este archivo lo que se encontraba en estructura.css, quedando este vacío y por tanto eliminándolo del proyecto.

- **Sección 5 - Estilo base del botón.** Definimos un único estilo base a los botones.
- **Sección 6 - definición única del modal.** El bloque completo del estilo del modal se define una sola vez en este archivo, se toma la versión más completa y se eliminan las otras cinco copias
- **Sección 7 - Grid de formularios.** Se define con un ancho de 130px como valor base, los css de cada página que necesitan un ancho diferente se sobrescriben en una sola línea en lugar de redefinir todo un bloque de código
- **Sección 8 - Filas de las listas de gestión**
- **Sección 9 - header de las tablas**
- **Sección 10 - utilidades**

En el resto de CSSs encontramos las funcionalidades específicas de cada página. Además de los cambios en los distintos css, hemos tenido que actualizar los HTMLs para que incorporen la carga del nuevo archivo de estilos.

Tras aplicar todos estos cambios comprobamos que la visualización seguía siendo la adecuada, nos encontramos con el problema de que algunas páginas, como la landing page, se habían desestructurado viendo solapamiento entre los distintos contenidos de las mismas. Hicimos la petición a Claude de que volviera a revisar los archivos y las modificaciones aplicadas para que este problema desapareciera. Tras revisarlo se llegó a la conclusión de que el problema venía de la definición base del botón ya que incluía posicionamiento absoluto que específico de un contexto. La solución ha sido acotar dicho posicionamiento exclusivamente al header y no al contenido principal de la página, gracias a esto podemos volver a visualizar correctamente el contenido de la web.

Header y footer no reutilizables

Los elementos header y footer no son reutilizables ya que estan copiados manualmente en los archivos HTML que tenemos en el proyecto. Cualquier cambio que se realice requiere editar cada uno de esos archivos a mano. Nos enfrentamos al mismo problema que con la función showModal: código duplicado.

Las razones por las que estos elementos no son reutilizables son las siguientes:

- **El proyecto es HTML estático servido por PHP sin motor de plantillas** → servimos directamente los archivos .html. PHP tiene su propio sistema de inclusión pero solo funciona con archivos .php, al no tener archivos de esta extensión PHP no los procesa y no puede inyectar fragmentos en ellos en el lado del servidor.
- **Uso de jQuery puro sobre el HTML estático sin sistema de componentes** → frameworks como React o Vue tienen componentes nativos que permiten definir el encabezado una sola vez y usarlo en todas las páginas, jQuery no.

Estudiando el header de los archivos vemos que tenemos tres variantes: para las páginas que no requieren sesión de usuario, los dashboard de los distintos usuarios y el dashboard del validador (botón de cierre de sesión no está en el sidebar).

Sin embargo, el footer es idéntico en todos los archivos, este elemento es el más fácil de extraer como componente.

Frente a esto Claude recomienda varias opciones para convertirlos en elementos reutilizables:

- **Opción 1 - Carga dinámica con jQuery** → crear header.html y footer.html como fragmentos y cargarlos con jQuery en el `$(document).ready`
 - **Ventajas**: simple, sin dependencias nuevas, compatible con el stack actual.
 - **Desventajas**: el header y footer aparecen después de que la página carga. El `<title>` de cada página sigue siendo individual. Las variantes del header complican el fragmento único: necesitarán lógica condicional en JS o fragmentos múltiples.
 - **Valoración para ResurgeNet**: Viable para el footer (idéntico en todas las páginas). Más complejo para el header por sus tres variantes.
- **Opción 2 - Cambiar extensión a .php** → renombrar los archivos .html a .php y usar la inclusión nativa que ofrece PHP
 - **Ventajas**
 - El fragmento se inyecta en el servidor antes de enviar la respuesta, sin flash visual.
 - Soporte nativo en el stack.
 - Las variantes del header se manejan con variables PHP.
 - **Desventajas**: requiere renombrar los 16 ficheros y actualizar todas las referencias internas. Si el servidor web no está configurado para servir .php directamente desde la carpeta frontend/, hay que ajustar la configuración de Nginx/Apache.
 - **Valoración para ResurgeNet**: es la solución más limpia dado el stack. Pero implica una decisión arquitectural que va más allá de la optimización CSS/JS.

La solución que nos da Claude es: no implementar la reutilización de estos elementos actualmente. Las razones que nos da son:

1. El footer es idéntico en todas las páginas pero trivialmente simple (dos párrafos). El coste de mantenerlo duplicado es bajo; el coste de introducir una nueva dependencia de carga dinámica es desproporcionado.
2. El header tiene tres variantes con contenido dinámico. Un fragmento compartido necesitaría lógica condicional que añade complejidad sin aportar valor claro en esta fase.
3. La Opción 2 (renombrar a .php) es la única que resuelve el problema correctamente sin efectos secundarios visuales, pero es una decisión arquitectural que afecta al servidor.

Si en el futuro se decide implementarlo, la ruta recomendada es la Opción 2.

Tras analizar toda la información proporcionada por Claude se ha decidido posponer la implementación de la reutilización del header y el footer hasta completar el resto de optimizaciones planteadas para el sistema. Se llevará a cabo la Opción 2 ya que es la más adecuada a largo plazo. Esta solución permite centralizar los elementos compartidos de forma limpia, mejorar la mantenibilidad del proyecto y facilitar futuras iteraciones del sistema sin depender de lógica adicional en el cliente.

URL del backend definida como localhost en producción

La URL que tenemos actualmente está escrita de forma manual y es válida únicamente cuando el proyecto se ejecuta en el ordenador donde se desarrolló. En cualquier otro contexto todas las llamadas al backend fallarán y la aplicación quedaría inutilizada.

Tenemos varias opciones para solucionar este problema:

1. **URL relativa al origen usando `window.location.origin`** que nos devuelve el protocolo junto con el dominio y el puerto del frontend tal como el navegador lo ve en ese momento. La ventaja es que no requiere ningún cambio en el despliegue y funciona automáticamente en cualquier entorno. La desventaja para este proyecto es que tenemos el front y el backend separados, con esta solución la URL sería la misma para ambos. Tendríamos que configurar Nginx como proxy inverso para que fuera viable
2. **Usar una variable de entorno en HTML vía Nginx** ya que esta puede inyectar variables de entorno en archivos estáticos en el momento en que se arranca el contenedor. La ventaja es que es la solución estándar en entornos Docker ya que la URL se define en el archivo `docker-compose.yml` y no en el código. La desventaja es que requiere crear un Dockerfile para el servicio frontend y un script de arranque.
3. **Proxy inverso en Nginx que unifica front y back en el mismo origen** de esta forma el navegador solo ve un único origen. Las ventajas son:
 - a. Tenemos una URL relativa que funciona en cualquier entorno sin cambios
 - b. No se necesita hacer cambios en el código JS
 - c. El puerto 8080 del backend deja de estar expuesto al exterior
 - d. Es la arquitectura estándar para apps web con front estático y backend en API

Las desventajas son:

- a. Requiere la modificación de dos archivos `nginx/front.conf` y `config.js`

Esta es la solución más limpia, estándar y la que menos complejidad añade.

La solución que nos aconseja Claude es la tercera por los motivos ya expuestos, le hemos dejado que desarrolle esta solución y ha realizado los cambios en los archivos necesarios (`nginx/front.conf` y `config.js`)

En definitiva, la solución adoptada es la estándar para aplicaciones de este tipo, con frontend estático y backend en una API separados. Hace que el despliegue sea reproducible sin cambios en el código.

3.15.2. Optimización backend

Al igual que hicimos para el frontend hemos empezado analizando los archivos que forman parte de nuestro frontend. Le hemos hecho a Claude una petición para que los revise en profundidad y nos proponga mejoras para optimizar rendimiento, mantenimiento y estructura. Nos ha indicado los siguientes problemas a solucionar:

- No tenemos autenticación en las rutas de la API
- El registro de un consumidor se hace sin transacción de base de datos

Autenticación en las rutas de la API

Todas las rutas de nuestro sistema son completamente públicas, no tenemos ningún tipo de middleware de autenticación que las proteja. El único control de acceso existente en la web está en el frontend: el menú de administración solo aparece cuando se inicia la sesión de usuario. Con esto no es suficiente ya que el front es código que es ejecutado en el navegador de cada usuario y puede ser ignorado. Cualquier persona con conocimientos básicos puede abrir las herramientas de desarrollo del navegador y obtener las URLs del backend sin pasar por la interfaz. Debemos implementar protección en la parte del servidor.

Le hemos pedido a Claude como se resolvería esta brecha de seguridad del sistema y propone lo siguiente: Laravel incluye un paquete llamado Sanctum, que es el paquete oficial de autenticación ligera de esta herramienta, con el que ya contamos lo que nos da una solución estándar en tres pasos:

1. Agrupamos las rutas protegidas bajo el middleware de autenticación en el archivo `api.php`
2. Creamos un middleware que verifique el rol del usuario autenticado antes de dar acceso a la ruta
3. Usamos tokens de Sanctum en lugar de la sesión manual actual

Tras analizar la solución propuesta le hemos pedido que la implemente consiguiendo solucionar la brecha de seguridad que tenía nuestro sistema contando ahora con una solución más robusta.

Registro de consumidores

El registro de un consumidor requiere insertar datos en las tablas usuario y consumidor. En la tabla usuario guardamos la información relacionada con la autenticación mientras que en la de consumidor guardamos todos los datos del perfil. Actualmente se hacen las dos inserciones de forma independiente.

Si el primer INSERT, que se realiza en la tabla usuario, se realiza correctamente pero el segundo falla la base de datos queda en un estado inconsistente porque tendríamos registro en usuario sin su registro en consumidor.

Para resolver este problema Claude nos dice que debemos envolver ambas inserciones en una transacción, así si cualquier operación dentro del bloque falla Laravel realiza un rollback de todas las operaciones anteriores.

Gracias a este cambio, el registro de un nuevo consumidor deja consistente la base de datos tanto si se realiza correctamente como si tenemos algún fallo.

3.16. Problemas encontrados y soluciones adquiridas

A lo largo del desarrollo del sistema nos hemos encontrado con una serie de problemas imprevistos que hemos tenido que ir solucionando. Aunque los hemos ido comentando durante cada iteración, a continuación se detalla cuáles han sido los dos problemas más relevantes y cómo se han afrontado.

3.16.1. Envío de emails a los clientes

Cuando estábamos desarrollando las historias de usuario 12 y 13 durante la iteración 7 teníamos que permitir el envío y recepción de emails entre los validadores de comercio y los comercios. La primera idea fue hacer uso de un dominio de correo electrónico como Gmail pero bloqueaba el acceso si no se activaba la verificación en dos pasos y se usaba la contraseña de la aplicación. Como no queríamos depender del protocolo que usan estos servidores de correo la solución fue usar el registro de log que tenemos con Laravel durante el desarrollo. De esta manera podemos verificar que los correos se envían de forma adecuada aunque no sea de la manera habitual.

3.16.2. Configuración de Claude

Para realizar la última iteración, la diez, decidimos introducir inteligencia artificial por dos motivos: aprender a desenvolvernos en el uso de este recurso y ver su nivel de utilidad.

Se intentó llevar a cabo mediante GitHub Copilot Chat, que está integrado con Visual Studio Code, sin embargo debido al límite mensual de uso y la carga computacional en el entorno local tuvimos que desechar esta opción.

Posteriormente se intentó trabajar en la nube usando GitHub Code Spaces que nos proporcionaba una instancia de Visual Studio Code accesible desde el navegador. El problema que tuvimos con esta alternativa fue que al contar con el plan gratuito seguíamos teniendo problemas con el límite de uso del chat. Se intentó mejorar la suscripción pero el 20 de Abril GitHub pausó los nuevos registros para los planes de pago por la gran demanda de su modo agente.

Finalmente se optó por usar Claude ya que no tiene límites restrictivos de uso, funciona completamente en la nube, podemos integrarla fácilmente con GitHub y tiene capacidad de razonamiento sobre el proyecto completo.

3.16.3. Despliegue de la base de datos en cualquier entorno

Este proyecto utiliza Docker para orquestar todos los servicios como se ha descrito a lo largo de esta memoria. La configuración de dichos servicios se define en el fichero docker-compose.yml en la raíz del proyecto. Al inicio del proyecto se configuró la base de datos de forma local lo que impedía que cualquier persona que clonará el repositorio pudiera levantar el proyecto correctamente. Nos encontrábamos ante dos problemas:

- El volumen de datos era una carpeta local que no estaba en nuestro repositorio lo que implica que cualquier persona que clone el repositorio no tendrá los datos disponibles.
- El backup SQL existía en el repositorio pero no se utilizaba, al no contar con este archivo al clonar el repositorio nos llegaba la base de datos completamente vacía causando errores en todas las operaciones.

La solución que se tomó fue modificar el fichero docker-compose.yml añadiendo dos cambios:

- El volumen de datos pasa a gestionarlo Docker en lugar de la carpeta local , por lo que se encarga de crear y gestionar el espacio de almacenamiento internamente. No necesitamos ninguna carpeta en el proyecto y funciona desde cualquier máquina donde se clone el repositorio
- Montar el backup SQL para que se haga una inicialización automática. Con esto conseguimos que la primera vez que se despliega el proyecto con un volumen vacío se ejecutan automáticamente todos los ficheros .sql y .sh que tengamos inicializando la base de datos con un esquema y datos predefinidos. Esto solo pasará la primera vez, cuando el volumen esté vacío, posteriormente MySQL detecta que tiene datos en el volumen y este paso se omite. Los cambios que se realicen en la base de datos quedarán reflejados en el volumen gestionado por Docker, este persiste entre reinicios sin tener pérdida de datos. Cuando se realicen cambios significativos tendrá que actualizarse el backup para que se vean reflejados en el repositorio.

Gracias a la modificación realizada contamos con un entorno completamente portable, siendo posible desplegarlo en cualquier dispositivo.

CAPÍTULO 4 - Conclusiones y trabajos futuro

4.1. Conclusiones

4.1.1. Cumplimiento de objetivos

Este trabajo tenía como objetivo general contribuir a la recuperación de los comercios afectados por una catástrofe mediante la implementación de una plataforma web que les de visibilidad así como les permita mantener su actividad económica usando el e-commerce. A lo largo del proyecto se ha comenzado con el diseño y desarrollo una solución funcional que permite definir las funciones de los distintos usuarios del sistema para que tengan su propio espacio de interacción con la plataforma web.

Además de este objetivo general, se han alcanzado los distintos objetivos específicos planteados al inicio del proyecto. Sin embargo, debido a la limitación temporal, no se ha completado la totalidad de las historias de usuario planteadas en el apartado 3.3.3. En el punto 3.3.4 podemos ver como hemos distribuido las historias por orden de prioridad en distintas iteraciones atendiendo al tiempo disponible. Las historias de usuario que han quedado pendientes son las menos prioritarias y se plantean como trabajos futuros para próximas fases de desarrollo.

En primer lugar, se han aplicado los **conocimientos adquiridos durante el grado** en áreas como ingeniería del software, desarrollo web, bases de datos y gestión de proyectos para construir una web completa orientada al comercio electrónico. Este proyecto nos ha permitido revisar y adquirir **nuevos conocimientos en tecnologías** como Laravel, Docker, Nginx y Claude AI, aprendiendo su funcionamiento e integración dentro de una solución real. Con estas herramientas hemos podido construir una plataforma organizada, mantenible y portable.

Para organizar todo el proceso se ha usado **Scrum**, tal y como puede verse en el capítulo 3, mediante la definición de historias de usuario, priorización de tareas en distintas iteraciones y el seguimiento continuo de los avances.

En relación con la **accesibilidad**, se han incorporado criterios basados en las recomendaciones WCAG como una prueba de aceptación más de las historias de usuario con el propósito de mejorar la usabilidad y favorecer su uso por el mayor número de personas posibles.

También se han realizado diferentes **estudios**, tanto de las principales plataformas como de las tecnologías existentes en el ámbito del e-commerce. Este análisis nos ha permitido tomar decisiones justificadas y conscientes sobre las tecnologías usadas a la hora de implementar nuestra solución. Estos estudios pueden verse en las secciones 2.1 y 2.2 de este documento.

Desde el **punto de vista arquitectónico** hemos conseguido desarrollar una solución mantenible, escalable, portable, modular, etc. basada en la separación de responsabilidades entre front y backend y el uso de contenedores. Se puede ver más en detalle la explicación de la arquitectura en el punto 3.3.2.

Finalmente, el proyecto ha permitido profundizar en las **competencias** relacionadas con la **búsqueda, extracción, análisis y síntesis de información** técnica y académica, necesarias tanto para la elaboración de la memoria como para la toma de decisiones durante el desarrollo.

4.1.2. Objetivos de desarrollo sostenible

El desarrollo de este proyecto se alinea con varios de los Objetivos de Desarrollo Sostenible establecidos en la agenda de 2030 por Naciones Unidas [41], especialmente con los siguientes:

- **Objetivo 8 - Trabajo decente y crecimiento económico**, cuyo propósito es promover el crecimiento económico inclusivo y sostenible, así como apoyar el empleo y la actividad empresarial. La plataforma desarrollada busca proporcionar visibilidad y herramientas digitales a los comercios afectados facilitando la continuidad de su actividad económica y favoreciendo su recuperación.
- **Objetivo 9 - Industria, innovación e infraestructura**, centrado en fomentar la innovación y el desarrollo de infraestructuras sostenibles. Las tecnologías utilizadas para llevar a cabo este proyecto contribuyen al desarrollo de soluciones accesibles y adaptables a diferentes contextos y necesidades
- **Objetivo 11 - Ciudades y comunidades sostenibles**, ya que nuestra solución pretende apoyar a los comercios locales y favorecer la resiliencia de las comunidades afectadas.
- **Objetivo 17 - Alianzas para lograr los objetivos**, debido al propósito de trabajar junto a las administraciones públicas y las ONGs para facilitar la implantación y uso de la plataforma en escenarios reales. Podemos ver cual es el papel de las ONGs en el apartado 3.1 de este documento.

En definitiva, este proyecto no solo pretende ofrecer una solución tecnológica sino también contribuir al cumplimiento de distintos Objetivos de Desarrollo Sostenible mediante una solución digital orientada a las necesidades reales.

4.1.3. Valoración personal

La realización de este trabajo ha supuesto una experiencia positiva, ya que me ha permitido desarrollar una idea propia desde cero estando presente en todo el ciclo de desarrollo y manejando los imprevistos. Además, he aplicado los conocimientos adquiridos en un proyecto real especialmente en áreas de desarrollo software y sistemas de información.

No solo he podido aplicar los conocimientos previos sino que he adquirido nuevos que serán muy útiles para la etapa profesional como la integración y uso de Claude AI, Laravel o Docker entre otros, que se han mencionado anteriormente.

En general el desarrollo ha sido satisfactorio y contribuido significativamente a mi formación como ingeniera informática, permitiéndole afrontar retos similares a los que me pueda encontrar en el ámbito profesional.

La naturaleza de este proyecto, nacido de una idea propia para resolver necesidades reales, abre una vertiente hacia el emprendimiento. El proceso llevado a cabo durante la puesta en marcha de este proyecto plantea una posibilidad de negocio, por lo que la intención a futuro es continuar con el desarrollo del mismo. De hecho, esta propuesta fue presentada en el concurso de ideas del Centro de Emprendimiento de la UGR en el año 2024, lo que refleja el interés real por explorar su posible aplicación práctica y su potencial de evolución, no solo en el ámbito académico.

4.2. Trabajos futuros

La plataforma desarrollada cumple con los objetivos planteados inicialmente como se ha podido ver en el punto anterior, sin embargo, durante el desarrollo vimos que podrían incorporarse mejoras en futuras versiones.

Además del desarrollo de las historias de usuario que han quedado pendientes por los motivos explicados anteriormente, existen mejoras que podemos aportar al sistema:

Implementar una arquitectura basada en microservicios

Actualmente nuestra arquitectura se basa en un sistema cliente-servidor centralizado. Podríamos pasar de esto a un modelo basado en microservicios separando funcionalidades como autenticación o gestión de productos en servicios independientes. Esto permite mejorar la escalabilidad, el mantenimiento y la tolerancia a fallos del sistema.

Mejorar la seguridad

En versiones futuras se podrían incorporar mecanismos avanzados de seguridad como la autenticación multifactor, sistemas de detección de accesos no autorizados o herramientas de monitorización. Con esto reforzaremos la protección de datos aumentando la fiabilidad de la plataforma.

Pasar de una base de datos centralizada a una distribuida y replicada

Actualmente el sistema cuenta con una base de datos centralizada, si pasamos al uso de bases de datos distribuidas y replicadas aumentaremos la disponibilidad y tolerancia a fallos del sistema, especialmente en situaciones con un elevado número de usuarios y transacciones simultáneas.

Permitir el hospedaje de otras webs en nuestros contenedores

Podríamos ampliar la infraestructura existente para permitir el hospedaje de webs existentes de los comercios dentro del sistema. Esto facilita la reutilización de recursos y hace que la infraestructura sea más flexible

Incluir un chat entre los consumidores y los comercios

Podría añadirse un sistema de comunicación tipo chat en tiempo real entre consumidores y comercios. Esta funcionalidad permitirá resolver las dudas que surjan a los consumidores, así como mejorar la atención al cliente al contar con una interacción directa con el comercio.

Realización de pruebas con usuarios reales tanto técnicas como funcionales

Podríamos realizar pruebas funcionales y de usabilidad con usuarios reales, tanto comercios como consumidores. Estas pruebas permitirían recoger sugerencias de mejora y detectar problemas relacionados con la experiencia de usuario, accesibilidad y rendimiento del sistema en entornos reales.

Posible comercialización del producto

La plataforma desarrollada podría aplicarse a contextos reales por lo que evolucionaría hacia un modelo comercializable real. Como se expone en el apartado 4.1.3 la propuesta fue presentada en el concurso de ideas del Centro de Emprendimiento de la UGR en el año 2024, hecho que refleja el interés desde un primer momento por hacer esta solución viable y funcional en el mercado.

CAPÍTULO 5 - Bibliografía

5.1. Introducción

- [1] Oficina PATECO - Comercio y Territorio. (2025, abril). *II Informe. Los daños y pérdidas ocasionadas por la DANA en la actividad empresarial: los servicios y el comercio*. Consejo de Cámaras de Comercio de la Comunitat Valenciana.
<https://pateco.org/ii-informe-de-danos-en-el-sector-comercio-y-los-servicios-por-la-dana/>
- [2] Adeva. (2024). Software maintenance costs: A comprehensive guide. Adeva Blog.
<https://adevs.com/blog/software-maintenance-costs/>
- [3] Indeed. (2026). Sueldo de Desarrollador/a de software en la Provincia de Granada. Indeed España. <https://es.indeed.com/career/desarrollador-de-software/salaries/Granada-provincia>
- [4] OVHcloud. (s.f.). *Servidores VPS económicos de alto rendimiento*. Recuperado el 16 de junio de 2026, de <https://www.ovhcloud.com/es/vps/>
- [5] Microsoft Visual Studio. (s.f.). Precios de suscripción y planes de Visual Studio. Microsoft Docs. <https://visualstudio.microsoft.com/es/vs/pricing/>
- [6] Social Media Pymes. (2024). Tarifas de marketing digital en España: Guía de precios y servicios. Blog de Social Media Pymes.
<https://www.socialmediapymes.com/tarifas-de-marketing-digital-en-espana>

5.2. Dominio del problema

- [7] Rincón-Soto, I. B., & Córdoba-Mendoza, J. A. (2017). El comercio electrónico (e-commerce) como estrategia de competitividad en las PyMEs. *Revista Politécnica*, 13(24), 39-51.
http://www.scielo.org.co/scielo.php?pid=S2248-60462017000100041&script=sci_arttext
- [8] Amazon Web Services. (s.f.). Architecting a highly available serverless microservices-based e-commerce site. AWS Architecture Blog.
<https://aws.amazon.com/es/blogs/architecture/architecting-a-highly-available-serverless-microservices-based-ecommerce-site/>
- [9] Amazon Web Services. (s.f.). Microservices on AWS. AWS Whitepapers.
https://docs.aws.amazon.com/es_es/whitepapers/latest/microservices-on-aws/microservices-on-aws.html
- [10] Hostinger. (s.f.). Sobre nosotros. Hostinger España.
<https://www.hostinger.com/es/sobre-nosotros>
- [11] Hostinger. (s.f.). ¿Qué es un hosting web y cómo funciona? Tutoriales Hostinger.
<https://www.hostinger.com/es/tutoriales/que-es-un-hosting>
- [12] Hostinger. (s.f.). Tutorial de hPanel: cómo gestionar tu cuenta. Tutoriales Hostinger.
<https://www.hostinger.com/es/tutoriales/tutorial-hpanel>
- [13] Hostinger. (s.f.). Precios de hosting web. Hostinger España.
<https://www.hostinger.com/es/precios>

- [14] HostingExperto. (2026). Opiniones sobre Hostinger: Pros y contras. <https://www.hostingexperto.es/opiniones/hostinger/#pros-y-contras>
- [15] Stockabee. (s.f.). Ventajas y desventajas de WooCommerce. Blog de Stockabee. <https://stockabee.com/ventajas-y-desventajas-de-woocommerce/>
- [16] WooCommerce. (s.f.). Enterprise Ecommerce Solutions. WooCommerce Official Site. <https://woocommerce.com/enterprise-ecommerce/>
- [17] Yupop. (s.f.). Vender en Amazon: Ventajas y desventajas para tu negocio. Blog de Yupop. <https://www.yupop.com/blog/vender-amazon-ventajas-desventajas>

5.3. Tecnologías potenciales e infraestructura de desarrollo

- [18] Hostinger. (2026, 15 de enero). *Los 11 mejores lenguajes de programación para aprender*. Tutoriales de Hostinger. <https://www.hostinger.es/tutoriales/mejores-lenguajes-de-programacion>
- [19] Hostinger. (2025a, 18 diciembre). *¿Qué es HTML? El lenguaje de marcado de hipertexto explicado para principiantes*. Tutoriales de Hostinger. <https://www.hostinger.es/tutoriales/que-es-html>
- [20] Hostinger. (2025b, 18 diciembre). *¿Qué es CSS? El lenguaje de hojas de estilo explicado para principiantes*. Tutoriales de Hostinger. <https://www.hostinger.es/tutoriales/que-es-css>
- [21] Amazon Web Services. (s.f.-a). *¿Cuál es la diferencia entre MySQL y PostgreSQL?* AWS Compare. <https://aws.amazon.com/es/compare/the-difference-between-mysql-vs-postgresql/>
- [22] Amazon Web Services. (s.f.-c). *¿Cuál es la diferencia entre Cassandra y MongoDB?* AWS Compare. <https://aws.amazon.com/es/compare/the-difference-between-cassandra-and-mongodb/>
- [23] Atlassian. (s.f.). Arquitectura de microservicios: Kubernetes vs. Docker. Atlassian Agile Coach. <https://www.atlassian.com/es/microservices/microservices-architecture/kubernetes-vs-docker>
- [24] Back4App. (2024). Los 10 mejores marcos de desarrollo frontend y backend. Back4App Blog. <https://blog.back4app.com/es/los-10-mejores-marcos-de-frontend-y-backend/>
- [25] Amazon Web Services. (s.f.). Diferencias entre bases de datos relacionales y no relacionales (NoSQL). AWS Cloud Camp. <https://aws.amazon.com/es/compare/the-difference-between-relational-and-non-relational-databases/>
- [26] Hostinger. (s.f.). Nginx vs. Apache: ¿Cuál es el mejor servidor web? Tutoriales Hostinger. <https://www.hostinger.es/tutoriales/que-usar-nginx-vs-apache>
- [27] Stackscale. (s.f.). Contenerización y orquestación de contenedores. Blog de Stackscale. <https://www.stackscale.com/es/blog/contenerizacion-orquestacion-contenedor/>

5.4. Accesibilidad

Material docente y guías de prácticas de las asignaturas DGP y MDA [Apuntes de clase].
Universidad de Granada (UGR).

5.5. Inteligencia Artificial e integración con el sistema

[28] Anthropic. (2024, noviembre). Introducing the Model Context Protocol. Anthropic Blog.
<https://www.anthropic.com/news/model-context-protocol>

[29] Anthropic. (2026). Claude [Herramienta de inteligencia artificial, versión Sonnet 4.6].
<https://claude.ai>

[30] Anthropic. (2026). Claude documentation. Anthropic Docs. <https://docs.anthropic.com>

[31] Binder, J. (2026, 20 de abril). Changes to GitHub Copilot Individual plans. The GitHub Blog.
<https://github.blog/news-insights/company-news/changes-to-github-copilot-individual-plans/>

[32] GitHub. (2026, 10 de abril). Pausing new GitHub Copilot Pro trials. GitHub Changelog.
<https://github.blog/changelog/2026-04-10-pausing-new-github-copilot-pro-trials/>

[33] GitHub. (2026, 20 de abril). Changes to GitHub Copilot plans for individuals. GitHub Changelog.
<https://github.blog/changelog/2026-04-20-changes-to-github-copilot-plans-for-individuals/>

[34] GitHub. (2026, 27 de abril). GitHub Copilot is moving to usage-based billing. The GitHub Blog.
<https://github.blog/news-insights/company-news/github-copilot-is-moving-to-usage-based-billing/>

[35] GitHub. (2026). Plans for GitHub Copilot. GitHub Docs.
<https://docs.github.com/en/copilot/get-started/plans>

[36] GitHub. (2026). About individual GitHub Copilot plans and benefits. GitHub Docs.
<https://docs.github.com/en/copilot/concepts/billing/individual-plans>

[37] GitHub Community. (2026, 24 de abril). Announcement & FAQ: Changes to GitHub Copilot Individual Plans [Discusión en foro]. <https://github.com/orgs/community/discussions/192963>

[38] Hardt, D. (Ed.). (2012, octubre). RFC 6749: The OAuth 2.0 Authorization Framework. Internet Engineering Task Force (IETF). <https://www.rfc-editor.org/rfc/rfc6749>

[39] Model Context Protocol. (2025). Specification — Model Context Protocol (versión 2025-11-25). <https://modelcontextprotocol.io/specification/2025-11-25>

[40] Model Context Protocol. (s.f.). Repositorio oficial de especificación y documentación [Repositorio GitHub]. <https://github.com/modelcontextprotocol/modelcontextprotocol>

5.6. Conclusiones - Objetivos de desarrollo sostenible

[41] Objetivos de desarrollo sostenible UNESCO.
<https://www.unesco.org/en/sdgs?hub=84636&utm>

ANEXO

Matriz de problemas encontrados y soluciones adoptadas

A continuación se presenta una tabla resumen que sintetiza los problemas que hemos tenido a lo largo del desarrollo del proyecto, detallando sus indicios, el diagnóstico técnico, las alternativas evaluadas y la solución adoptada en cada caso

ID	Componente	Indicios	Diagnóstico	Soluciones posibles	Solución adoptada
PR-01	Módulo de notificaciones	Bloqueo del envío de correos de prueba y errores de autenticación	Restricciones estrictas de seguridad de Gmail que exige obligatoriamente factores de doble autenticación y contraseñas específicas	Contar con un servicio SMTP dedicado Desviar el tráfico de correo al sistema de logs	Uso del driver log de Laravel. Se redirigió el tráfico de emails en el entorno de desarrollo hacia el archivo <code>laravel.log</code> . Esto nos permitió validar el envío de los mails sin dependencias externas
PR-02	Entorno de desarrollo con IA	Bloqueos por fin de cuota mensual y limitaciones del equipo local al procesar el contexto global del código.	Limitaciones del plan gratuito de GitHub Copilot y la suspensión temporal de altas de pago en GitHub Code Spaces (abril de 2026) por alta demanda.	Usar GitHub Copilot Chat en Visual Studio Code Trabajar en la nube con GitHub Code Spaces Migrar a un asistente en la nube sin restricciones.	Integración de Claude. Se seleccionó esta plataforma por su ejecución en la nube (liberando de carga al hardware local), su capacidad de razonamiento global del proyecto completo y la ausencia de bloqueos de registro en sus planes.
PR-03	Despliegue del sistema	Errores en operaciones por base de datos vacía al clonar el repositorio en una nueva máquina.	Dependencia de una ruta de volumen local absoluta (fuera del repositorio) y falta de automatización en la carga del script SQL de respaldo	Modificar la orquestación del contenedor.	Rediseño del archivo <code>docker-compose.yml</code>. Se delegó la gestión del volumen de datos a los volúmenes internos de Docker . Además, se configuró el montaje del <i>backup</i> SQL en el directorio de inicialización automática de MySQL

Enlace al repositorio de GitHub

A continuación se proporciona el enlace para acceder al repositorio de GitHub donde pueden encontrar todo el código para desplegar el programa:

<https://github.com/angelamgr/ResurgeNet#>